

JAVA 17 OCP · YDS STUDY SERIES

Unit 09 · Collections and Generics · Bilingual Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu ana kaynak, [OCP_Java_SE17_Chapter1den_Itibaren.pdf](#) dosyasındaki Chapter 9 sayfaları 463–530'u kaynak sırasını bozmadan kapsar. Tekrarlanan running header/footer ve basılı sayfa numaraları içerik sayılmamış; chapter hedefleri, bütün paragraflar, listeler, 14 tablo, 8 figure/caption, kodlar, çıktılar, Summary, Exam Essentials ve Review Questions 1–20 korunmuştur. Appendix'teki resmî cevap açıklamaları PDF sayfaları 939–942'den ayrıca eklenmiştir.

[Vocabulary](#) · [Grammar notes](#) · [Teknik hafıza notu](#)

Kaynak kapsam manifesti

- Kaynak: [exam_lecture/OCP_Java_SE17_Chapter1den_Itibaren.pdf](#)
- Chapter: 9 · Collections and Generics
- Chapter PDF sayfaları: 463–530
- Appendix cevap sayfaları: 939–942 (yalnız Chapter 9, Answers 1–20)
- Beklenen sayfa marker'ı: 68
- Beklenen resmî cevap: 20
- Eşleme biçimi: English paragraf → hemen altında Türkçe çeviri → varsa kod

İçindekiler

- 1 [Using Common Collection APIs](#)
- 2 [Using the List Interface](#)
- 3 [Using the Set Interface](#)
- 4 [Using the Queue and Deque Interfaces](#)
- 5 [Using the Map Interface](#)
- 6 [Comparing Collection Types](#)
- 7 [Sorting Data](#)
- 8 [Working with Generics](#)
- 9 [Summary / Özet](#)
- 10 [Exam Essentials / Sınav İçin Temel Noktalar](#)
- 11 [Review Questions / Gözden Geçirme Soruları](#)
- 12 [Official Review Question Answers / Resmî Cevaplar](#)

Chapter 9 · Collections and Generics · Eksiksiz çift dilli kaynak

Chapter 9 · Collections and Generics

English: OCP exam objectives covered in this chapter: Working with Arrays and Collections. Create Java arrays, `List`, `Set`, `Map`, and `Deque` collections, and add, remove, update, retrieve, and sort their elements. **Türkçe:** Bu bölümde kapsanan OCP sınav hedefleri: Arrays ve Collections ile çalışmak. Java array'leri ile `List`, `Set`, `Map` ve `Deque` collection'larını oluşturmak; bunların element'lerini eklemek, kaldırmak, güncellemek, getirmek ve sıralamak.

English: In this chapter, we introduce the Java Collections Framework classes and interfaces you need to know for the exam. The thread-safe collection types are discussed in Chapter 13, "Concurrency." **Türkçe:** Bu bölümde sınav için bilmeniz gereken Java Collections Framework class ve interface'lerini tanıtıyoruz. Thread-safe collection type'ları Chapter 13, "Concurrency" bölümünde ele alınmaktadır.

English: As you may remember from Chapter 8, "Lambdas and Functional Interfaces," we covered lambdas, method references, and built-in functional interfaces. Many of these are used in this chapter. Please go back and review Table 8.4 if the functional interfaces are unfamiliar. **Türkçe:** Chapter 8, "Lambdas and Functional Interfaces" bölümünden hatırlayabileceğiniz gibi lambda'ları, method reference'ları ve built-in functional interface'leri ele aldık. Bunların çoğu bu bölümde kullanılır. Functional interface'ler size yabancı geliyorsa Table 8.4'ü yeniden inceleyin.

English: Next, we cover details about `Comparator` and `Comparable`. Finally, we discuss how to create your own classes and methods that use generics so that the same class can be used with many types. **Türkçe:** Ardından `Comparator` ve `Comparable` hakkındaki ayrıntıları ele alıyoruz. Son olarak aynı class'ın birçok type ile kullanılabilmesi için generics kullanan kendi class ve method'larınızı nasıl oluşturacağınızı tartışıyoruz.

Using Common Collection APIs

English: A collection is a group of objects contained in a single object. The Java Collections Framework is a set of classes in `java.util` for storing collections. There are four main interfaces in the Java Collections Framework. **Türkçe:** Collection, tek bir object içinde bulunan object grubudur. Java Collections Framework, collection'ları saklamak için `java.util` içindeki class kümesidir. Java Collections Framework'te dört ana interface vardır.

English: List: A list is an ordered collection of elements that allows duplicate entries. Elements in a list can be accessed by an `int` index. **Türkçe:** List: List, duplicate entry'lere izin veren ordered element collection'ıdır. Bir list'teki element'lara `int` index ile erişilebilir.

English: Set: A set is a collection that does not allow duplicate entries. **Türkçe:** Set: Set, duplicate entry'lere izin vermeyen bir collection'dır.

English: Queue: A queue is a collection that orders its elements in a specific order for processing. A `Deque` is a subinterface of `Queue` that allows access at both ends. **Türkçe:** Queue: Queue, işleme amacıyla element'larını belirli bir sıraya koyan collection'dır. `Deque`, iki uçtan da erişime izin veren bir `Queue` subinterface'idir.

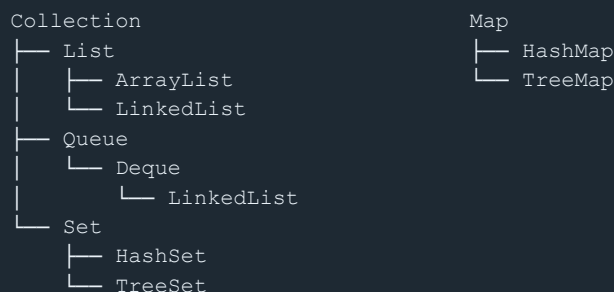
English: Map: A map is a collection that maps keys to values, with no duplicate keys allowed. The elements in a map are key/value pairs. **Türkçe:** Map: Map, duplicate key'e izin vermeden key'leri value'lara eşleyen collection'dır. Bir map'teki element'lar key/value pair'leridir.

English: Figure 9.1 shows the `Collection` interface, its subinterfaces, and some classes that implement the interfaces that you should know for the exam. The interfaces are shown in rectangles, with the classes in rounded boxes. **Türkçe:** Figure 9.1, `Collection` interface'ini, subinterface'lerini ve sınav için bilmeniz gereken bu interface'leri implement eden bazı class'ları gösterir. Interface'ler dikdörtgenlerde, class'lar yuvarlatılmış kutularda gösterilmiştir.

English: Notice that `Map` doesn't implement the `Collection` interface. It is considered part of the Java Collections Framework even though it isn't technically a `Collection`. It is a collection (note the lowercase), though, in that it contains a group of objects. The reason maps are treated differently is that they need different methods due to being key/value pairs. **Türkçe:** `Map`in `Collection` interface'ini implement etmediğine dikkat edin. Teknik olarak bir `Collection` olmamasına rağmen Java Collections Framework'ün parçası kabul edilir. Yine de bir object grubu içerdiği için küçük harfle collection'dır. Map'lerin farklı ele alınmasının nedeni, key/value pair olmaları yüzünden farklı method'lara gereksinim duymalarıdır.

Figure 9.1 · Java Collections Framework / Java Collections Framework

English — figure structure: `Collection` has the `List`, `Queue`, and `Set` subinterfaces. `Deque` extends `Queue`. `ArrayList` implements `List`; `LinkedList` implements both `List` and `Deque`; `HashSet` and `TreeSet` implement `Set`. Separately, `HashMap` and `TreeMap` implement `Map`. **Türkçe — şekil yapısı:** `Collection`; `List`, `Queue` ve `Set` subinterface'lerine sahiptir. `Deque`, `Queue`yu extend eder. `ArrayList`, `List`i; `LinkedList`, hem `List`i hem `Deque`u; `HashSet` ve `TreeSet`, `Set`i implement eder. Ayrı olarak `HashMap` ve `TreeMap`, `Map`i implement eder.



English: In this section, we discuss the common methods that the Collections API provides to the implementing classes. Many of these methods are convenience methods that could be implemented in other ways but make your code easier to write and read. This is why they are convenient. **Türkçe:** Bu kısımda Collections API'nin implementing class'lara sağladığı ortak method'ları tartışıyoruz. Bunların çoğu başka yollarla da implement edilebilecek, fakat kodun yazılmasını ve okunmasını kolaylaştıran convenience method'lardır. Kullanışlı olmalarının nedeni budur.

English: In this section, we use `ArrayList` and `HashSet` as our implementation classes, but they can apply to any class that inherits the `Collection` interface. We cover the specific properties of each `Collection` class in the next section. **Türkçe:** Bu kısımda implementation class'larımız olarak `ArrayList` ve `HashSet` kullanıyoruz; ancak method'lar `Collection` interface'ini inherit eden her class'a uygulanabilir. Her `Collection` class'ının kendine özgü özelliklerini sonraki kısımda ele alıyoruz.

Editor note: A Java class *implements* an interface; an interface may *extend* another interface. The source's informal phrase “class that inherits the `Collection` interface” means a class that implements `Collection` directly or through a subinterface. **Editör notu:** Java'da class bir interface'i *implements* eder; bir interface başka bir interface'i *extends* edebilir. Kaynaktaki “`Collection` interface'ini inherit eden class” ifadesi, `Collection` doğrudan veya bir subinterface üzerinden implement eden class anlamındadır.

Using the Diamond Operator

English: When constructing a Java Collections Framework, you need to specify the type that will go inside. We could write code using generics like the following: **Türkçe:** Bir Java Collections Framework nesnesi oluştururken içine girecek type'ı belirtmeniz gerekir. Generics kullanan kodu şöyle yazabiliriz:

```
List<Integer> list = new ArrayList<Integer>();
```

English: You might even have generics that contain other generics, such as this: **Türkçe:** Hatta aşağıdaki gibi başka generic'leri içeren generic'leriniz olabilir:

```
Map<Long, List<Integer>> mapLists = new HashMap<Long, List<Integer>>();
```

English: That's a lot of duplicate code to write! Luckily, the diamond operator (`<>`) is a shorthand notation that allows you to omit the generic type from the right side of a statement when the type can be inferred. It is called the diamond operator because `<>` looks like a diamond. **Türkçe:** Bu, yazılacak çok fazla tekrarlı kod demektir! Neyse ki diamond operator (`<>`), type çıkarılabildiğinde statement'ın sağ tarafındaki generic type'ı yazmamanıza olanak tanıyan kısa gösterimdir. `<>` elmasa benzediği için diamond operator olarak adlandırılır.

English: Compare the previous declarations with these new, much shorter versions: **Türkçe:** Önceki declaration'ları şu yeni ve çok daha kısa sürümlerle karşılaştırın:

```
List<Integer> list = new ArrayList<>();  
Map<Long, List<Integer>> mapOfLists = new HashMap<>();
```

English: To the compiler, both these declarations and our previous ones are equivalent. To us, though, the latter is a lot shorter and easier to read. **Türkçe:** Compiler açısından bu declaration'lar önceki declaration'larla eşdeğerdir. Bizim açımızdan ise sonrakiler çok daha kısa ve kolay okunur.

English: The diamond operator cannot be used as the type in a variable declaration. It can be used only on the right side of an assignment operation. For example, neither of the following compiles: **Türkçe:** Diamond operator bir variable declaration'da type olarak kullanılamaz. Yalnız bir assignment operation'ın sağ tarafında kullanılabilir. Örneğin aşağıdakilerin hiçbirini derlenmez:

```
List<> list = new ArrayList<Integer>(); // DOES NOT COMPILE

class InvalidUse {
    void use(List<> data) {}           // DOES NOT COMPILE
}
```

Adding Data

English: The `add()` method inserts a new element into the `Collection` and returns whether it was successful. The method signature is as follows: **Türkçe:** `add()` method'u `Collection` a yeni bir element ekler ve işlemin başarılı olup olmadığını döndürür. Method signature şöyledir:

```
public boolean add(E element)
```

English: Remember that the Collections Framework uses generics. You will see `E` appear frequently. It means the generic type that was used to create the collection. For some `Collection` types, `add()` always returns `true`. For other types, there is logic as to whether the `add()` call was successful. The following shows how to use this method: **Türkçe:** Collections Framework'ün generics kullandığını unutmayın. `E` ile sık karşılaşacaksınız. Bu harf, collection oluşturulurken kullanılan generic type'ı ifade eder. Bazı `Collection` type'larında `add()` her zaman `true` döndürür. Diğerlerinde `add()` çağrısının başarılı olup olmadığını belirleyen bir mantık vardır. Aşağıdaki kod bu method'un kullanımını gösterir:

```
3: Collection<String> list = new ArrayList<>();
4: System.out.println(list.add("Sparrow")); // true
5: System.out.println(list.add("Sparrow")); // true
6:
7: Collection<String> set = new HashSet<>();
8: System.out.println(set.add("Sparrow")); // true
9: System.out.println(set.add("Sparrow")); // false
```

English: A `List` allows duplicates, making the return value `true` each time. A `Set` does not allow duplicates. On line 9, we tried to add a duplicate so that Java returns `false` from the `add()` method.

Türkçe: `List` duplicate'lere izin verdiğinden return value her seferinde `true` olur. `Set` duplicate'lere izin vermez. Line 9'da duplicate eklemeye çalıştığımız için Java, `add()` method'undan `false` döndürür.

Removing Data

English: The `remove()` method removes a single matching value in the `Collection` and returns whether it was successful. The method signature is as follows: **Türkçe:** `remove()` method'u `Collection` içindeki eşleşen tek bir value'yu kaldırır ve işlemin başarılı olup olmadığını döndürür. Method signature şöyledir:

```
public boolean remove(Object object)
```

English: This time, the `boolean` return value tells us whether a match was removed. The following shows how to use this method: **Türkçe:** Bu kez `boolean` return value, eşleşen bir öğenin kaldırılıp kaldırılmadığını söyler. Aşağıdaki kod method'un kullanımını gösterir:

```
3: Collection<String> birds = new ArrayList<>();
4: birds.add("hawk"); // [hawk]
```

```
5: birds.add("hawk"); // [hawk, hawk]
6: System.out.println(birds.remove("cardinal")); // false
7: System.out.println(birds.remove("hawk")); // true
8: System.out.println(birds); // [hawk]
```

English: Line 6 tries to remove an element that is not in `birds`. It returns `false` because no such element is found. Line 7 tries to remove an element that is in `birds`, so it returns `true`. Notice that it removes only one match. **Türkçe:** Line 6, `birds` içinde bulunmayan bir element'i kaldırmaya çalışır. Böyle bir element bulunmadığı için `false` döndürür. Line 7, `birds` içinde bulunan bir element'i kaldırır; bu nedenle `true` döndürür. Yalnızca bir eşleşmenin kaldırıldığına dikkat edin.

Counting Elements

English: The `isEmpty()` and `size()` methods look at how many elements are in the `Collection`. The method signatures are as follows: **Türkçe:** `isEmpty()` ve `size()` method'ları `Collection` içinde kaç element bulunduğuna bakar. Method signature'ları şöyledir:

```
public boolean isEmpty()
public int size()
```

English: The following shows how to use these methods: **Türkçe:** Aşağıdaki kod bu method'ların kullanımını gösterir:

```
Collection<String> birds = new ArrayList<>();
System.out.println(birds.isEmpty()); // true
System.out.println(birds.size()); // 0
birds.add("hawk"); // [hawk]
birds.add("hawk"); // [hawk, hawk]
System.out.println(birds.isEmpty()); // false
System.out.println(birds.size()); // 2
```

English: At the beginning, `birds` has a size of 0 and is empty. It has a capacity that is greater than 0. After we add elements, the size becomes positive, and it is no longer empty. **Türkçe:** Başlangıçta `birds`'in size'i 0'dır ve collection boştur. Capacity'si ise 0'dan büyüktür. Element ekledikten sonra size pozitif olur ve artık boş değildir.

Clearing the Collection

English: The `clear()` method provides an easy way to discard all elements of the `Collection`. The method signature is as follows: **Türkçe:** `clear()` method'u `Collection`'in bütün element'lerini atmanın kolay bir yolunu sağlar. Method signature şöyledir:

```
public void clear()
```

English: The following shows how to use this method: **Türkçe:** Aşağıdaki kod method'un kullanımını gösterir:

```
Collection<String> birds = new ArrayList<>();
birds.add("hawk");           // [hawk]
birds.add("hawk");           // [hawk, hawk]
System.out.println(birds.isEmpty()); // false
System.out.println(birds.size());    // 2
birds.clear();                // []
```

```
System.out.println(birds.isEmpty()); // true
System.out.println(birds.size());    // 0
```

English: After calling `clear()`, `birds` is back to being an empty `ArrayList` of size 0. **Türkçe:** `clear()` çağrısından sonra `birds`, size'i 0 olan boş bir `ArrayList` durumuna döner.

Check Contents

English: The `contains()` method checks whether a certain value is in the `Collection`. The method signature is as follows: **Türkçe:** `contains()` method'u belirli bir value'nun `Collection` içinde bulunup bulunmadığını kontrol eder. Method signature şöyledir:

```
public boolean contains(Object object)
```

English: The following shows how to use this method: **Türkçe:** Aşağıdaki kod method'un kullanımını gösterir:

```
Collection<String> birds = new ArrayList<>();
birds.add("hawk");           // [hawk]
System.out.println(birds.contains("hawk")); // true
System.out.println(birds.contains("robin")); // false
```

English: The `contains()` method calls `equals()` on elements of the `ArrayList` to see whether there are any matches. **Türkçe:** `contains()` method'u bir eşleşme olup olmadığını görmek için `ArrayList` element'ları üzerinde `equals()` çağırır.

Removing with Conditions

English: The `removeIf()` method removes all elements that match a condition. We can specify what should be deleted using a block of code or even a method reference. **Türkçe:** `removeIf()` method'u bir condition ile eşleşen bütün element'ları kaldırır. Neyin silineceğini bir code block veya method reference kullanarak belirtebiliriz.

English: The method signature looks like the following. (We explain what the `? super` means in the “Working with Generics” section later in this chapter.) **Türkçe:** Method signature aşağıdaki gibidir. (`? super` ifadesinin ne anlama geldiğini bu bölümün ilerleyen kısmındaki “Working with Generics” başlığında açıklıyoruz.)

```
public boolean removeIf(Predicate<? super E> filter)
```

English: It uses a `Predicate`, which takes one parameter and returns a `boolean`. Let's take a look at an example: **Türkçe:** Bir parameter alan ve `boolean` döndüren `Predicate` kullanır. Bir örneğe bakalım:

```
4: Collection<String> list = new ArrayList<>();
5: list.add("Magician");
6: list.add("Assistant");
7: System.out.println(list);           // [Magician, Assistant]
8: list.removeIf(s -> s.startsWith("A"));
9: System.out.println(list);           // [Magician]
```

English: Line 8 shows how to remove all of the `String` values that begin with the letter A. This allows us to make the Assistant disappear. Let's try an example with a method reference: **Türkçe:** Line 8, A harfiyle başlayan bütün `String` value'ların nasıl kaldırılacağını gösterir. Böylece Assistant'ı ortadan kaldırabiliriz. Bir method reference örneği deneyelim:

```
11: Collection<String> set = new HashSet<>();
12: set.add("Wand");
13: set.add("");
```

```
14: set.removeIf(String::isEmpty); // s -> s.isEmpty()
15: System.out.println(set);       // [Wand]
```

English: On line 14, we remove any empty `String` objects from `set`. The comment on that line shows the lambda equivalent of the method reference. Line 15 shows that the `removeIf()` method successfully removed one element from `list`. **Türkçe:** Line 14'te boş olan bütün `String` object'lerini `set`ten kaldırıyoruz. O satırdaki comment, method reference'ın lambda eşdeğerini gösterir. Line 15, `removeIf()` method'unun `list`ten bir element'ı başarıyla kaldırdığını gösterir.

Editor note: The source says “removed one element from `list`” although this example variable is named `set`; the source wording is retained here. **Editör notu:** Kaynak, bu örnekteki variable adı `set` olduğu hâlde “`list`ten bir element kaldırdı” der; kaynak ifadesi burada korunmuştur.

Iterating

English: There’s a `forEach()` method that you can call on a `Collection` instead of writing a loop. It uses a `Consumer` that takes a single parameter and doesn’t return anything. The method signature is as follows: **Türkçe:** Loop yazmak yerine `Collection` üzerinde çağırabileceğiniz bir `forEach()` method'u vardır. Tek parameter alan ve hiçbir şey döndürmeyen bir `Consumer` kullanır. Method signature şöyledir:

```
public void forEach(Consumer<? super T> action)
```

English: Cats like to explore, so let’s print out two of them using both method references and lambdas: **Türkçe:** Kediler keşfetmeyi sever; öyleyse ikisinin adını hem method reference hem lambda kullanarak yazdıralım:

```
Collection<String> cats = List.of("Annie", "Ripley");  
cats.forEach(System.out::println);  
cats.forEach(c -> System.out.println(c));
```

English: The cats have discovered how to print their names. Now they have more time to play (as do we)! **Türkçe:** Kediler adlarını nasıl yazdıracaklarını keşfettiler. Artık onların da bizim de oynamak için daha fazla zamanı var!

Other Iteration Approaches

English: There are other ways to iterate through a `Collection`. For example, in Chapter 3, “Making Decisions,” you saw how to loop through a list using an enhanced `for` loop. **Türkçe:** Bir `Collection` üzerinde iterate etmenin başka yolları da vardır. Örneğin Chapter 3, “Making Decisions” bölümünde enhanced `for` loop ile bir list üzerinde nasıl dolaşıldığını gördünüz.

```
for (String element : coll)  
    System.out.println(element);
```

English: You may see another older approach used. **Türkçe:** Daha eski başka bir yaklaşımın kullanıldığını görebilirsiniz.

```
Iterator<String> iter = coll.iterator();  
while (iter.hasNext()) {  
    String string = iter.next();  
    System.out.println(string);  
}
```

English: Pay attention to the difference between these techniques. The `hasNext()` method checks whether there is a next value. In other words, it tells you whether `next()` will execute without throwing an exception. The `next()` method actually moves the `Iterator` to the next element. **Türkçe:** Bu teknikler arasındaki farka dikkat edin. `hasNext()` method'u sıradaki bir value'nun bulunup bulunmadığını kontrol eder; başka bir deyişle `next()` in exception fırlatmadan çalışıp çalışmayacağını söyler. `next()` method'u ise `Iterator`ı gerçekten sonraki element'a ilerletir.

Determining Equality

English: There is a custom implementation of `equals()` so you can compare two `Collections` to compare the type and contents. The implementation will vary. For example, `ArrayList` checks order, while `HashSet` does not. **Türkçe:** İki `Collection`ın type ve contents'ini karşılaştırabilmeniz için custom bir `equals()` implementation'ı vardır. Implementation değişiklik gösterir. Örneğin `ArrayList` order'ı kontrol ederken `HashSet` etmez.

```
boolean equals(Object object)
```

English: The following shows an example: **Türkçe:** Aşağıda bir örnek gösterilmektedir:

```
23: var list1 = List.of(1, 2);
24: var list2 = List.of(2, 1);
25: var set1 = Set.of(1, 2);
26: var set2 = Set.of(2, 1);
27:
28: System.out.println(list1.equals(list2)); // false
29: System.out.println(set1.equals(set2)); // true
30: System.out.println(list1.equals(set1)); // false
```

English: Line 28 prints `false` because the elements are in a different order, and a `List` cares about order. By contrast, line 29 prints `true` because a `Set` is not sensitive to order. Finally, line 30 prints `false` because the types are different. **Türkçe:** Element'lar farklı sırada olduğu ve `List` order'ı önemseydiği için line 28 `false` yazdırır. Buna karşılık `Set` order'a duyarlı olmadığından line 29 `true` yazdırır. Son olarak type'lar farklı olduğu için line 30 `false` yazdırır.

Unboxing nulls

English: Java protects us from many problems with Collections. However, it is still possible to write a `NullPointerException`: **Türkçe:** Java bizi Collections ile ilgili birçok sorundan korur. Bununla birlikte `NullPointerException` üreten kod yazmak yine de mümkündür:

```
3: var heights = new ArrayList<Integer>();
4: heights.add(null);
5: int h = heights.get(0); // NullPointerException
```

English: On line 4, we add a `null` to the list. This is legal because a `null` reference can be assigned to any reference variable. On line 5, we try to unbox that `null` to an `int` primitive. This is a problem. Java tries to get the `int` value of `null`. Since calling any method on `null` gives a `NullPointerException`, that is just what we get. Be careful when you see `null` in relation to autoboxing. **Türkçe:** Line 4'te list'e `null` ekliyoruz. Bir `null` reference herhangi bir reference variable'a atanabildiği için bu yasaldir. Line 5'te bu `null` değeri `int` primitive'e unbox etmeye çalışıyoruz. Sorun burada doğar. Java, `null`'in `int` value'sunu almaya çalışır. `null` üzerinde herhangi bir method çağırmak `NullPointerException` verdiği için tam olarak bu exception'ı alırız. Autoboxing ile birlikte `null` gördüğünüzde dikkatli olun.

Using the List Interface

English: Now that you're familiar with some common `Collection` interface methods, let's move on to specific interfaces. You use a list when you want an ordered collection that can contain duplicate entries. For example, a list of names may contain duplicates, as two animals can have the same name. Items can be retrieved and inserted at specific positions in the list based on an `int` index, much like an array. Unlike an array, though, many `List` implementations can change in size after they are declared. **Türkçe:** Bazı ortak `Collection` interface method'larını öğrendiğimize göre belirli interface'lere geçelim. Duplicate entry içerebilen ordered bir collection istediğinizde list kullanırsınız. Örneğin iki hayvan aynı ada sahip olabileceğinden ad listesi duplicate içerebilir. Öğeler, array'e benzer biçimde `int` index'e göre list'in belirli konumlarından getirilebilir ve bu konumlara eklenebilir. Ancak array'den farklı olarak birçok `List` implementation'ının size'ı declaration'dan sonra değişebilir.

English: Lists are commonly used because there are many situations in programming where you need to keep track of a list of objects. For example, you might make a list of what you want to see at the zoo: first, see the lions, because they go to sleep early; second, see the pandas, because there is a long line later in the day; and so forth. **Türkçe:** Programlamada bir object listesini izlemeniz gereken birçok durum bulunduğu için list'ler yaygın olarak kullanılır. Örneğin hayvanat bahçesinde görmek istediklerinizin listesini yapabilirsiniz: önce erken uyudukları için aslanları, ikinci olarak günün ilerleyen saatlerinde kuyruk uzadığı için pandaları görmek ve bu şekilde devam etmek.

English: Figure 9.2 shows how you can envision a `List`. Each element of the `List` has an index, and the indexes begin with zero. **Türkçe:** Figure 9.2 bir `List`i zihninizde nasıl canlandırabileceğinizi gösterir. `List`in her element'ının bir index'i vardır ve index'ler sıfırdan başlar.

Figure 9.2 · Example of a List / List örneği

Ordered index / Sıralı index	Data / Veri
0	lions
1	pandas
2	zebras
...	...

Türkçe şekil özeti: Ordered index sütunu 0, 1, 2, ... diye ilerler; karşılık gelen data sırasıyla `lions`, `pandas`, `zebras`, ... değerleridir.

English: Sometimes you don't care about the order of elements in a list. `List` is like the "go to" data type. When we make a shopping list before going to the store, the order of the list happens to be the order in which we thought of the items. We probably aren't attached to that particular order, but it isn't hurting anything. **Türkçe:** Bazen bir list'teki element'ların order'ını önemsemezsiniz. `List`, başvuru varsayılan data type gibidir. Mağazaya gitmeden önce alışveriş listesi yaptığımızda list'in order'ı, öğelerin aklımıza gelme sırasıdır. Muhtemelen bu belirli sıraya bağlı değildir; yine de bu sıra bir zarar vermez.

English: While the classes implementing the `List` interface have many methods, you need to know only the most common ones. Conveniently, these methods are the same for all of the implementations that might show up on the exam. **Türkçe:** `List` interface'ini implement eden class'ların birçok method'u olsa da yalnız en yaygın olanları bilmeniz gerekir. Kullanışlı biçimde bu method'lar, sınavda karşınıza çıkabilecek bütün implementation'larda aynıdır.

English: The main thing all `List` implementations have in common is that they are ordered and allow duplicates. Beyond that, they each offer different functionality. We look at the implementations that you need to know and the available methods. **Türkçe:** Bütün `List` implementation'larının ortak temel özelliği ordered olmaları ve duplicate'lere izin vermeleridir. Bunun ötesinde her biri farklı functionality sunar. Bilmeniz gereken implementation'lara ve kullanılabilir method'lara bakacağız.

English — exam note: Pay special attention to which names are classes and which are interfaces. The exam may ask you which is the best class or which is the best interface for a scenario. **Türkçe — sınav notu:** Hangi adların class, hangilerinin interface olduğuna özellikle dikkat edin. Sınav, belirli bir scenario için en iyi class'ın veya en iyi interface'in hangisi olduğunu sorabilir.

Comparing List Implementations

English: An `ArrayList` is like a resizable array. When elements are added, the `ArrayList` automatically grows. When you aren't sure which collection to use, use an `ArrayList`. **Türkçe:** `ArrayList`, yeniden boyutlandırılabilen bir array gibidir. Element eklendiğinde `ArrayList` otomatik büyür. Hangi collection'ı kullanacağınızdan emin değilseniz `ArrayList` kullanın.

English: The main benefit of an `ArrayList` is that you can look up any element in constant time. Adding or removing an element is slower than accessing an element. This makes an `ArrayList` a good choice when you are reading more often than (or the same amount as) writing to the `ArrayList`. **Türkçe:** `ArrayList`'in ana yararı, herhangi bir element'ı constant time'da bulabilmenizdir. Element eklemek veya kaldırmak, element'a erişmekten daha yavaştır. Bu nedenle `ArrayList`'ten, ona yazdığınızdan daha sık veya yazma sıklığınız kadar okuma yapıyorsanız `ArrayList` iyi bir seçimdir.

English: A `LinkedList` is special because it implements both `List` and `Deque`. It has all the methods of a `List`. It also has additional methods to facilitate adding or removing from the beginning and/or end of the list. **Türkçe:** `LinkedList`, hem `List` hem `Deque` implement ettiği için özeldir. Bir `List`'in bütün method'larına sahiptir. Ayrıca list'in başlangıcından ve/veya sonundan ekleme ya da kaldırmayı kolaylaştıran ek method'ları vardır.

English: The main benefits of a `LinkedList` are that you can access, add to, and remove from the beginning and end of the list in constant time. The trade-off is that dealing with an arbitrary index takes linear time. This makes a `LinkedList` a good choice when you'll be using it as `Deque`. As you saw in Figure 9.1, a `LinkedList` implements both the `List` and `Deque` interfaces. **Türkçe:** `LinkedList`'in ana yararı, list'in başlangıcına ve sonuna constant time'da erişebilmeniz, ekleme yapabilmeniz ve bu konumlardan kaldırabilmenizdir. Karşılığında arbitrary bir index ile çalışmak linear time alır. Bu, `LinkedList`'i `Deque` olarak kullanacağınızda iyi bir seçim yapar. Figure 9.1'de gördüğünüz gibi `LinkedList`, hem `List` hem `Deque` interface'ini implement eder.

Creating a List with a Factory

English: When you create a `List` of type `ArrayList` or `LinkedList`, you know the type. There are a few special methods where you get a `List` back but don't know the type. These methods let you create a `List` including data in one line using a factory method. This is convenient, especially when testing. Some of these methods return an immutable object. As we saw in Chapter 6, "Class Design," an immutable object cannot be changed or modified. Table 9.1 summarizes these three lists. **Türkçe:** `ArrayList` veya `LinkedList` type'ında bir `List` oluşturduğunuzda type'ı bilirsiniz. Geriye bir `List` aldığınız fakat type'ını bilmediğiniz birkaç özel method vardır. Bu method'lar bir factory method kullanarak data içeren `List`'i tek satırda oluşturmanızı sağlar. Bu, özellikle test sırasında kullanışlıdır. Bu method'lardan bazıları immutable object döndürür. Chapter 6, "Class Design" bölümünde gördüğümüz gibi immutable object değiştirilemez. Table 9.1 bu üç list'i özetler.

Table 9.1 · Factory methods to create a List / List oluşturan factory method'lar

Method	Description / Açıklama	Can add elements? / Element eklenebilir mi?	Can replace elements? / Element değiştirilebilir mi?	Can delete elements? / Element silinebilir mi?
<code>Arrays.asList(varargs)</code>	Returns fixed-size list backed by an array / Array tarafından desteklenen fixed-size list döndürür	No / Hayır	Yes / Evet	No / Hayır
<code>List.of(varargs)</code>	Returns immutable list / Immutable list döndürür	No / Hayır	No / Hayır	No / Hayır
<code>List.copyOf(collection)</code>	Returns immutable list with copy of original collection's values / Özgün collection value'larının kopyasını içeren immutable list döndürür	No / Hayır	No / Hayır	No / Hayır

Türkçe tablo özeti: `Arrays.asList()` array tarafından desteklenen fixed-size bir list döndürür; ekleme ve silme yapılamaz, replacement yapılabilir. `List.of()` ve `List.copyOf()` immutable list döndürür; ekleme, replacement ve silme yapılamaz. `copyOf()`, özgün collection value'larının kopyasını alır.

English: Let's take a look at an example of these three methods: **Türkçe:** Bu üç method'un bir örneğine bakalım:

```
16: String[] array = new String[] { "a", "b", "c" };
17: List<String> asList = Arrays.asList(array); // [a, b, c]
18: List<String> of = List.of(array);          // [a, b, c]
19: List<String> copy = List.copyOf(asList);   // [a, b, c]
20:
21: array[0] = "z";
22:
23: System.out.println(asList); // [z, b, c]
24: System.out.println(of);     // [a, b, c]
25: System.out.println(copy);   // [a, b, c]
26:
27: asList.set(0, "x");
28: System.out.println(Arrays.toString(array)); // [x, b, c]
29:
30: copy.add("y"); // UnsupportedOperationException
```

English: Line 17 creates a `List` that is backed by an array. Line 21 changes the array, and line 23 reflects that change. Lines 27 and 28 show the other direction where changing the `List` updates the underlying array. Lines 18 and 19 create an immutable `List`. Line 30 shows it is immutable by throwing an exception when trying to add a value. All three lists would throw an exception when adding or removing a value. The `of` and `copy` lists would also throw one on trying to update an element. **Türkçe:** Line 17, bir array tarafından desteklenen `List` oluşturur. Line 21 array'i değiştirir ve line 23 bu değişikliği yansıtır. Lines 27–28, diğer yönde `List` değiştirildiğinde underlying array'in güncellendiğini gösterir. Lines 18–19 immutable `List` oluşturur. Line 30, value ekleme denemesinde exception fırlatarak immutable olduğunu gösterir. Üç list'in tümü value ekleme veya kaldırma girişiminde exception fırlatır. `of` ve `copy` list'leri bir element güncellenmeye çalışıldığında da exception fırlatır.

Creating a List with a Constructor

English: Most Collections have two constructors that you need to know for the exam. The following shows them for `LinkedList`: **Türkçe:** Çoğu Collection'ın sınav için bilmeniz gereken iki constructor'ı vardır. Aşağıda `LinkedList` için bunlar gösterilmektedir:

```
var linked1 = new LinkedList<String>();
var linked2 = new LinkedList<String>(linked1);
```

English: The first says to create an empty `LinkedList` containing all the defaults. The second tells Java that we want to make a copy of another `LinkedList`. Granted, `linked1` is empty in this example, so it isn't particularly interesting. **Türkçe:** İlki bütün default'ları taşıyan boş bir `LinkedList` oluşturmayı söyler. İkincisi Java'ya başka bir `LinkedList`in kopyasını istediğimizi bildirir. Bu örnekte `linked1` boş olduğundan kuşkusuz pek ilgi çekici değildir.

English: `ArrayList` has an extra constructor you need to know. We now show the three constructors: **Türkçe:** `ArrayList`in bilmeniz gereken ek bir constructor'ı vardır. Şimdi üç constructor'ı gösteriyoruz:

```
var list1 = new ArrayList<String>();  
var list2 = new ArrayList<String>(list1);  
var list3 = new ArrayList<String>(10);
```

English: The first two are the common constructors you need to know for all Collections. The final example says to create an `ArrayList` containing a specific number of slots, but again not to assign any. You can think of this as the size of the underlying array. **Türkçe:** İlk ikisi bütün Collections için bilmeniz gereken ortak constructor'lardır. Son örnek, belirli sayıda slot içeren fakat yine hiçbirine değer atanmamış bir `ArrayList` oluşturmayı söyler. Bunu underlying array'in size'ı olarak düşünebilirsiniz.

Using `var` with `ArrayList`

English: Consider this code, which mixes `var` and generics: **Türkçe:** `var` ile generics'i birlikte kullanan şu kodu inceleyin:

```
var strings = new ArrayList<String>();  
strings.add("a");  
for (String s : strings) { }
```

English: The type of `var` is `ArrayList<String>`. This means you can add a `String` or loop through the `String` objects. What if we use the diamond operator with `var`? **Türkçe:** `var`'ın type'ı `ArrayList<String>`dir. Bu, bir `String` ekleyebileceğiniz veya `String` object'leri üzerinde loop yapabileceğiniz anlamına gelir. `var` ile diamond operator kullanırsak ne olur?

```
var list = new ArrayList<>();
```

English: Believe it or not, this does compile. The type of the `var` is `ArrayList<Object>`. Since there isn't a type specified for the generic, Java has to assume the ultimate superclass. This is a bit silly and unexpected, so please don't write it. But if you see it on the exam, you'll know what to expect. Now can you figure out why this doesn't compile? **Türkçe:** İster inanın ister inanmayın, bu kod derlenir. `var`'ın type'ı `ArrayList<Object>`tir. Generic için bir type belirtilmediğinden Java ultimate superclass'ı varsaymak zorundadır. Bu biraz anlamsız ve beklenmedik olduğu için böyle yazmayın. Ancak sınavda görürseniz ne bekleyeceğinizi biliyorsunuz. Şimdi aşağıdaki kodun neden derlenmediğini bulabilir misiniz?

```
var list = new ArrayList<>();  
list.add("a");  
for (String s : list) { } // DOES NOT COMPILE
```

English: The type of `var` is `ArrayList<Object>`. Since there isn't a type in the diamond operator, Java has to assume the most generic option it can. Therefore, it picks `Object`, the ultimate superclass. Adding a `String` to the list is fine. You can add any subclass of `Object`. However, in the loop, we need to use the `Object` type rather than `String`. **Türkçe:** `var`'ın type'ı `ArrayList<Object>`tir. Diamond operator içinde type bulunmadığından Java mümkün olan en generic seçeneği varsaymak zorundadır; bu nedenle ultimate superclass olan `Object`i seçer. List'e `String` eklemek geçerlidir; `Object`'in herhangi bir subclass'ını ekleyebilirsiniz. Ancak loop'ta `String` yerine `Object` type'ını kullanmamız gerekir.

Working with List Methods

English: The methods in the `List` interface are for working with indexes. In addition to the inherited `Collection` methods, the method signatures that you need to know are in Table 9.2. **Türkçe:** `List` interface'indeki method'lar index'lerle çalışmak içindir. Inherited `Collection` method'larına ek olarak bilmeniz gereken method signature'ları Table 9.2'dedir.

Table 9.2 · List methods / List method'ları · Part / Kısım 1

Method	Description / Açıklama
<code>public boolean add(E element)</code>	Adds element to end (available on all Collection APIs) / Element'i sona ekler (bütün Collection API'lerinde vardır)
<code>public void add(int index, E element)</code>	Adds element at index and moves the rest toward the end / Element'i index'e ekler, kalanları sona doğru kaydırır
<code>public E get(int index)</code>	Returns element at index / Index'teki element'i döndürür
<code>public E remove(int index)</code>	Removes element at index and moves the rest toward the front / Index'teki element'i kaldırır, kalanları öne doğru kaydırır

Türkçe tablo özeti: `add(E)` sona ekler; `add(index, E)` belirtilen index'e ekleyip kalanları sona doğru kaydırır. `get(index)` o index'teki element'i döndürür. `remove(index)` element'i kaldırıp kalanları öne doğru kaydırır.

Table 9.2 · List methods / List method'ları · Part / Kısım 2

Method	Description / Açıklama
<code>public default void replaceAll(UnaryOperator<E> op)</code>	Replaces each element in list with result of operator / List'teki her element'i operator sonucuyla değiştirir
<code>public E set(int index, E e)</code>	Replaces element at index and returns original; throws <code>IndexOutOfBoundsException</code> if index is invalid / Index'teki element'i değiştirip özgün value'yu döndürür; index geçersizse <code>IndexOutOfBoundsException</code> fırlatır
<code>public default void sort(Comparator<? super E> c)</code>	Sorts list; covered later in “Sorting Data” / List'i sıralar; ileride “Sorting Data” kısmında ele alınır

Türkçe tablo özeti: `replaceAll()` her element'i operator sonucuyla değiştirir. `set()` index'teki element'i değiştirip eski value'yu döndürür; geçersiz index'te `IndexOutOfBoundsException` fırlatır. `sort()` list'i sıralar; ileride “Sorting Data” kısmında ele alınır.

English: The following statements demonstrate most of these methods for working with a `List`: **Türkçe:** Aşağıdaki statement'lar bir `List` ile çalışırken kullanılan bu method'ların çoğunu gösterir:

```
3: List<String> list = new ArrayList<>();
4: list.add("SD");           // [SD]
5: list.add(0, "NY");        // [NY, SD]
6: list.set(1, "FL");        // [NY, FL]
7: System.out.println(list.get(0)); // NY
8: list.remove("NY");        // [FL]
9: list.remove(0);           // []
10: list.set(0, "?");        // IndexOutOfBoundsException
```

English: On line 3, `list` starts out empty. Line 4 adds an element to the end of the list. Line 5 adds an element at index 0 that bumps the original index 0 to index 1. Notice how the `ArrayList` is now automatically one larger. Line 6 replaces the element at index 1 with a new value. **Türkçe:** Line 3'te `list` boş başlar. Line 4, list'in sonuna bir element ekler. Line 5, index 0'a bir element ekleyerek eski index 0 ögesini index 1'e iter. `ArrayList`in otomatik olarak bir element büyüdüğüne dikkat edin. Line 6, index 1'deki element'i yeni bir value ile değiştirir.

English: Line 7 uses the `get()` method to print the element at a specific index. Line 8 removes the element matching `NY`. Finally, line 9 removes the element at index 0, and `list` is empty again. Line 10 throws an `IndexOutOfBoundsException` because there are no elements in the `List`. Since there are no elements to replace, even index 0 isn't allowed. If line 10 were moved up between lines 4 and 5, the call would succeed. **Türkçe:** Line 7 belirli index'teki element'i yazdırmak için `get()` method'unu kullanır. Line 8, `NY` ile eşleşen element'i kaldırır. Son olarak line 9 index 0'daki element'i kaldırır ve `list` yeniden boş olur. `List` içinde element bulunmadığından line 10 `IndexOutOfBoundsException` fırlatır. Değiştirilecek element olmadığı için index 0 bile geçerli değildir. Line 10, lines 4–5 arasına taşınıydı çağrı başarılı olurdu.

English: The output would be the same if you tried these examples with `LinkedList`. Although the code would be less efficient, it wouldn't be noticeable until you had very large lists. **Türkçe:** Bu örnekleri `LinkedList` ile deneseydiniz output aynı olurdu. Kod daha az efficient olsa da çok büyük list'leriniz olana kadar fark edilmezdi.

English: Now let's take a look at the `replaceAll()` method. It uses a `UnaryOperator` that takes one parameter and returns a value of the same type. **Türkçe:** Şimdi `replaceAll()` method'una bakalım. Bir parameter alan ve aynı type'ta value döndüren `UnaryOperator` kullanır:

```
var numbers = Arrays.asList(1, 2, 3);
numbers.replaceAll(x -> x * 2);
System.out.println(numbers); // [2, 4, 6]
```

English: This lambda doubles the value of each element in the list. The `replaceAll()` method calls the lambda on each element of the list and replaces the value at that index. **Türkçe:** Bu lambda list'teki her element'in value'sunu iki katına çıkarır. `replaceAll()` method'u lambda'yı list'in her element'i üzerinde çağırır ve o index'teki value'yu değiştirir.

Overloaded `remove()` Methods

English: We've now seen two overloaded `remove()` methods. The one from `Collection` removes an object that matches the parameter. By contrast, the one from `List` removes an element at a specified index. **Türkçe:** Artık iki overloaded `remove()` method'u gördük. `Collection`dan gelen sürüm parameter ile eşleşen object'i kaldırır. Buna karşılık `List`ten gelen sürüm belirtilen index'teki element'i kaldırır.

English: This gets tricky when you have an `Integer` type. What do you think the following prints?

Türkçe: `Integer` type'ınız olduğunda bu durum zorlaşır. Aşağıdaki kod sizce ne yazdırır?

```
31: var list = new LinkedList<Integer>();
32: list.add(3);
33: list.add(2);
34: list.add(1);
35: list.remove(2);
36: list.remove(Integer.valueOf(2));
37: System.out.println(list);
```

English: The correct answer is `[3]`. Let's look at how we got there. At the end of line 34, we have `[3, 2, 1]`. Line 35 passes a primitive, which means we are requesting deletion of the element at index 2. This leaves us with `[3, 2]`. Then line 36 passes an `Integer` object, which means we are deleting the value 2. That brings us to `[3]`. **Türkçe:** Doğru cevap `[3]`tür. Buna nasıl ulaştığımıza bakalım. Line 34'ün sonunda `[3, 2, 1]` vardır. Line 35 primitive geçer; bu, index 2'deki element'in silinmesini istediğimiz anlamına gelir ve `[3, 2]` kalır. Ardından line 36 bir `Integer` object geçirir; bu kez value 2'yi sileriz ve `[3]` kalır.

English — exam note: Since calling `remove()` with an `int` uses the index, an index that doesn't exist will throw an exception. For example, `list.remove(100)` throws an `IndexOutOfBoundsException`.

Türkçe — sınav notu: `remove()` u `int` ile çağırmak index kullandığından, var olmayan index exception fırlatır. Örneğin `list.remove(100)`, `IndexOutOfBoundsException` fırlatır.

Converting from List to an Array

English: Since an array can be passed as a vararg, Table 9.1 covered how to convert an array to a `List`. You should also know how to do the reverse. Let's start with turning a `List` into an array: **Türkçe:** Bir array vararg olarak geçirilebildiğinden Table 9.1, array'in `List`e nasıl dönüştürüleceğini ele aldı. Tersini de bilmelisiniz. Bir `List`i array'e dönüştürmekle başlayalım:

```
13: List<String> list = new ArrayList<>();
14: list.add("hawk");
15: list.add("robin");
16: Object[] objectArray = list.toArray();
17: String[] stringArray = list.toArray(new String[0]);
18: list.clear();
19: System.out.println(objectArray.length); // 2
20: System.out.println(stringArray.length); // 2
```

English: Line 16 shows that a `List` knows how to convert itself to an array. The only problem is that it defaults to an array of class `Object`. This isn't usually what you want. Line 17 specifies the type of the array and does what we want. The advantage of specifying a size of 0 **Türkçe:** Line 16, bir `List`in kendisini array'e dönüştürebildiğini gösterir. Tek sorun, default olarak `Object` class'ına ait bir array oluşturmasıdır; bu genellikle istediğiniz şey değildir. Line 17 array type'ını belirtir ve istediğimiz sonucu verir. Parameter için size 0 belirtmenin avantajı

English: for the parameter is that Java will create a new array of the proper size for the return value. If you like, you can suggest a larger array to be used instead. If the `List` fits in that array, it will be returned. Otherwise, a new array will be created. **Türkçe:** Java'nın return value için uygun size'da yeni bir array oluşturmasıdır. İsterseniz bunun yerine kullanılmak üzere daha büyük bir array önerebilirsiniz. `List` bu array'e sığarsa o array döndürülür; aksi hâlde yeni bir array oluşturulur.

English: Also, notice that line 18 clears the original `List`. This does not affect either array. The array is a newly created object with no relationship to the original `List`. It is simply a copy. **Türkçe:** Ayrıca line 18'in original `List`i temizlediğine dikkat edin. Bu, iki array'i de etkilemez. Array, original `List` ile ilişkisi olmayan yeni oluşturulmuş bir object'tir; yalnızca bir kopyadır.

Using the Set Interface

English: You use a `Set` when you don't want to allow duplicate entries. For example, you might want to keep track of the unique animals that you want to see at the zoo. You aren't concerned with the order in which you see these animals, but there isn't time to see them more than once. You just want to make sure you see the ones that are important to you and remove them from the set of outstanding animals to see after you see them. **Türkçe:** Duplicate entry'lere izin vermek istemediğinizde `Set` kullanırsınız. Örneğin hayvanat bahçesinde görmek istediğiniz unique hayvanları izlemek isteyebilirsiniz. Bu hayvanları hangi sırayla gördüğünüzü önemsemezsiniz, ancak onları birden fazla görmeye zaman yoktur. Sizin için önemli olanları gördüğünüzden emin olmak ve gördükten sonra bekleyen hayvanlar set'inden kaldırmak istersiniz.

English: Figure 9.3 shows how you can envision a `Set`. The main thing that all `Set` implementations have in common is that they do not allow duplicates. We look at each implementation that you need to know for the exam and how to write code using `Set`. **Türkçe:** Figure 9.3 bir `Set`i nasıl canlandırabileceğinizi gösterir. Bütün `Set` implementation'larının ortak temel özelliği duplicate'lere izin vermemeleridir. Sınav için bilmeniz gereken her implementation'a ve `Set` kullanan kodun nasıl yazıldığına bakacağız.

Figure 9.3 · Example of a Set / Set örneği

```
Set
┌───┬───┬───┐
│pandas│lions│zebras│
```

Türkçe şekil özeti: Set içinde index veya zorunlu bir order gösterilmeden `pandas`, `lions` ve `zebras` unique element'ları bulunur.

Comparing Set Implementations

English: A `HashSet` stores its elements in a hash table, which means the keys are a hash and the values are an `Object`. This means that the `HashSet` uses the `hashCode()` method of the objects to retrieve them more efficiently. Remember that a valid `hashCode()` doesn't mean every object will get a unique value, but the method is often written so that hash values are spread out over a large range to reduce collisions. **Türkçe:** `HashSet` element'lerini hash table'da saklar; burada key'ler hash, value'lar `Object` tir. Dolayısıyla `HashSet`, object'leri daha efficient getirmek için onların `hashCode()` method'unu kullanır. Geçerli bir `hashCode()` un her object'in unique value alacağı anlamına gelmediğini unutmayın; ancak method çoğunlukla collision'ları azaltmak amacıyla hash value'ları geniş bir range'e dağıtacak şekilde yazılır.

English: The main benefit is that adding elements and checking whether an element is in the set both have constant time. The trade-off is that you lose the order in which you inserted the elements. Most of the time, you aren't concerned with this in a `Set` anyway, making `HashSet` the most common set. **Türkçe:** Ana yararı, hem element eklemenin hem bir element'in set içinde bulunup bulunmadığını kontrol etmenin constant time almasıdır. Karşılığında element'ları eklediğiniz order'ı kaybedersiniz. Çoğu zaman bir `Set` te bu order'ı zaten önemsemezsiniz; bu da `HashSet` i en yaygın set yapar.

English: A `TreeSet` stores its elements in a sorted tree structure. The main benefit is that the set is always in sorted order. The trade-off is that adding and checking whether an element exists takes longer than with a `HashSet`, especially as the tree grows larger. **Türkçe:** `TreeSet` element'lerini sorted tree structure içinde saklar. Ana yararı set'in her zaman sorted order'da olmasıdır. Karşılığında, özellikle tree büyüdükçe element eklemek ve bir element'in varlığını kontrol etmek `HashSet` e göre daha uzun sürer.

English: Figure 9.4 shows how you can envision `HashSet` and `TreeSet` being stored. `HashSet` is more complicated in reality, but this is fine for the purpose of the exam. **Türkçe:** Figure 9.4, `HashSet` ve `TreeSet` in nasıl saklandığını canlandırabileceğiniz biçimde gösterir. `HashSet` gerçekte daha karmaşıktır; ancak bu gösterim sınavın amacı için yeterlidir.

Figure 9.4 · Examples of a HashSet and TreeSet / HashSet ve TreeSet örnekleri

HashSet hashCode() value / değeri	Data / Veri
-995544615	pandas
...	...
-705903059	zebras
...	...
102978519	lions

```

TreeSet
  pandas
  /    \
lions  zebras

```

Türkçe şekil özeti: `HashSet` tarafında `pandas`, `zebras` ve `lions` hashCode value'larıyla ilişkilidir; `TreeSet` tarafı aynı data'yı sorted tree structure içinde gösterir.

English: For the exam, you don't need to know how to create a hash or tree set (the implementation can be complex). Phew! You just need to know how to use them! **Türkçe:** Sınav için hash set veya tree set'in nasıl oluşturulduğunu bilmeniz gerekmez; implementation karmaşık olabilir. Neyse! Yalnızca bunları nasıl kullanacağınızı bilmelisiniz.

Working with Set Methods

English: Like a `List`, you can create an immutable `Set` in one line or make a copy of an existing one.

Türkçe: `List`'te olduğu gibi tek satırda immutable `Set` oluşturabilir veya var olan bir set'in kopyasını alabilirsiniz.

```
Set<Character> letters = Set.of('z', 'o', 'o');  
Set<Character> copy = Set.copyOf(letters);
```

Editor note — Java 17 result: The first line throws `IllegalArgumentException` because `Set.of()` rejects duplicate elements. Execution therefore never reaches `Set.copyOf(letters)`. **Editör notu — Java 17 sonucu:** `Set.of()` duplicate element'ları reddettiği için ilk satır `IllegalArgumentException` fırlatır. Bu nedenle execution, `Set.copyOf(letters)` satırına hiç ulaşmaz.

English: Those are the only extra methods you need to know for the `Set` interface for the exam! You do have to know how sets behave with respect to the traditional `Collection` methods. You also have to know the differences between the types of sets. Let's start with `HashSet`. **Türkçe:** Sınav için `Set` interface'ine özgü bilmeniz gereken ek method'lar yalnızca bunlardır! Set'lerin geleneksel `Collection` method'larına göre nasıl davrandığını ve set type'ları arasındaki farkları da bilmeniz gerekir. `HashSet` ile başlayalım:

```
3: Set<Integer> set = new HashSet<>();  
4: boolean b1 = set.add(66); // true  
5: boolean b2 = set.add(10); // true  
6: boolean b3 = set.add(66); // false  
7: boolean b4 = set.add(8); // true  
8: set.forEach(System.out::println);
```

English: This code prints three lines: **Türkçe:** Bu kod üç satır yazdırır:

```
66  
8  
10
```

English: The `add()` methods should be straightforward. They return `true` unless the `Integer` is already in the set. Line 6 returns `false`, because we already have 66 in the set, and a set must preserve uniqueness. Line 8 prints the elements of the set in an arbitrary order. In this case, it happens not to be sorted order or the order in which we added the elements. **Türkçe:** `add()` method'ları açık olmalıdır. `Integer` zaten set içinde değilse `true` döndürürler. Set'te 66 bulunduğu ve set uniqueness'i korumak zorunda olduğu için line 6 `false` döndürür. Line 8 set'in element'larını arbitrary order'da yazdırır. Bu örnekte sıra ne sorted order ne de element'ları eklediğimiz order'dır.

English: Remember that the `equals()` method is used to determine equality. The `hashCode()` method is used to know which bucket to look in so that Java doesn't have to look through the whole set to find out whether an object is there. The best case is that hash codes are unique and Java has to call `equals()` on only one object. The worst case is that all implementations return the same `hashCode()` and Java has to call `equals()` on every element of the set anyway. **Türkçe:** Equality'yi belirlemek için `equals()` method'unun kullanıldığını unutmayın. Java'nın bir object'in varlığını bulmak için bütün set'e bakmak zorunda kalmaması amacıyla hangi bucket'a bakılacağını bilmek için `hashCode()` kullanılır. En iyi durumda hash code'lar unique'tir ve Java yalnız bir object üzerinde `equals()` çağırır. En kötü durumda bütün implementation'lar aynı `hashCode()` u döndürür ve Java yine set'in her element'ında `equals()` çağırarak zorunda kalır.

English: Now let's look at the same example with `TreeSet`. **Türkçe:** Şimdi aynı örneğe `TreeSet` ile bakalım:

```
3: Set<Integer> set = new TreeSet<>();
4: boolean b1 = set.add(66); // true
5: boolean b2 = set.add(10); // true
6: boolean b3 = set.add(66); // false
7: boolean b4 = set.add(8); // true
8: set.forEach(System.out::println);
```

English: This time, the code prints the following: **Türkçe:** Bu kez kod şunları yazdırır:

```
8
10
66
```

English: The elements are printed out in their natural sorted order. Numbers implement the `Comparable` interface in Java, which is used for sorting. Later in the chapter, you learn how to create your own `Comparable` objects. **Türkçe:** Element'lar natural sorted order'larında yazdırılır. Number type'ları Java'da sorting için kullanılan `Comparable` interface'ini implement eder. Bölümün ilerleyen kısmında kendi `Comparable` object'lerinizi nasıl oluşturacağınızı öğreneceksiniz.

Using the Queue and Deque Interfaces

English: You use a `Queue` when elements are added and removed in a specific order. You can think of a queue as a line. For example, when you want to enter a stadium and someone is waiting in line, you get in line behind that person. And if you are British, you get in the queue behind that person, making this really easy to remember! This is a FIFO (first-in, first-out) queue. **Türkçe:** Element'lar belirli bir order'da eklenip kaldırıldığında `Queue` kullanırsınız. Queue'yu bir sıra olarak düşünebilirsiniz. Örneğin stadyuma girmek istediğinizde biri sırada bekliyorsa onun arkasına geçersiniz. Britanya İngilizcesinde de onun arkasında queue'ya girersiniz; bu da hatırlamayı kolaylaştırır. Bu FIFO (first-in, first-out) queue'dur.

English: A **Deque** (double-ended queue), often pronounced “deck,” is different from a regular queue in that you can insert and remove elements from both the front (head) and back (tail). Think, “Dr. Woodie Flowers, come right to the front! You are the only one who gets this special treatment. Everyone else will have to start at the back of the line.” **Türkçe:** Genellikle “deck” diye telaffuz edilen **Deque** (double-ended queue), hem front’tan (head) hem back’ten (tail) element ekleyip kaldırabilmeniz bakımından normal queue’dan farklıdır. “Dr. Woodie Flowers, doğrudan öne gelin! Bu özel muameleyi yalnız siz görüyorsunuz. Diğer herkes sıranın arkasından başlamak zorunda” diye düşünün.

English: You can envision a double-ended queue as shown in Figure 9.5. **Türkçe:** Double-ended queue’yu Figure 9.5’teki gibi canlandırabilirsiniz.

Figure 9.5 · Example of a Deque / Deque örneği

Front (head) / Ön (baş) → [Rover] — [Spot] — [Bella] ← Back (tail) / Arka (son)

Türkçe şekil özeti: **Rover** front/head tarafında, **Spot** ortada ve **Bella** back/tail tarafındadır.

English: Supposing we are using this as a FIFO queue. Rover is first, which means he was first to arrive. Bella is last, which means she was last to arrive and has the longest wait remaining. **Türkçe:** Bunu FIFO queue olarak kullandığımızı varsayalım. Rover ilk sıradadır; yani ilk gelen odur. Bella sondadır; yani en son gelmiştir ve kalan bekleme süresi en uzundur.

English: All queues have specific requirements for adding and removing the next element. Beyond that, they each offer different functionality. We look at the implementations you need to know and the available methods. **Türkçe:** Bütün queue’ların sıradaki element’i eklemek ve kaldırmak için belirli gereksinimleri vardır. Bunun ötesinde her biri farklı functionality sunar. Bilmeniz gereken implementation'lara ve kullanılabilir method'lara bakacağız.

Comparing Deque Implementations

English: You saw **LinkedList** earlier in the **List** section. In addition to being a list, it is a **Deque**. **Türkçe:** **LinkedList**i daha önce **List** kısmında gördünüz. List olmasının yanında bir **Deque**’dur.

English: The main benefit of a **LinkedList** is that it implements both the **List** and **Deque** interfaces. The trade-off is that it isn’t as efficient as a “pure” queue. You can use the **ArrayDeque** class if you don’t need the **List** methods. **Türkçe:** **LinkedList**in ana yararı hem **List** hem **Deque** interface’ini implement etmesidir. Karşılığında “pure” queue kadar efficient değildir. **List** method’larına gereksiniminiz yoksa **ArrayDeque** class’ını kullanabilirsiniz.

Working with Queue and Deque Methods

English: The `Queue` interface contains six methods, shown in Table 9.3. There are three pieces of functionality and versions of the methods that throw an exception or use the return type, such as `null`, for all information. We've bolded the ones that throw an exception when something goes wrong, like trying to read from an empty `Queue`. **Türkçe:** `Queue` interface'i Table 9.3'te gösterilen altı method içerir. Üç functionality ve bunların, sorun olduğunda exception fırlatan veya bütün bilgiyi `null` gibi return type üzerinden veren method sürümleri vardır. Boş `Queue`dan okumaya çalışmak gibi bir sorun olduğunda exception fırlatanlar kaynakta kalın gösterilmiştir.

Table 9.3 · Queue methods / Queue method'ları

Functionality / İşlev	Throws an exception / Exception fırlatır	Uses return value / Return value kullanır
Add to back / Back'e ekle	<code>public boolean add(E e)</code>	<code>public boolean offer(E e)</code>
Read from front / Front'tan oku	<code>public E element()</code>	<code>public E peek()</code>
Get and remove from front / Front'tan getir ve kaldır	<code>public E remove()</code>	<code>public E poll()</code>

Türkçe tablo özeti: Back'e eklemede `add()` exception sürümü, `offer()` return-value sürümüdür. Front'tan okumada `element()` exception, `peek()` return-value sürümüdür. Front'tan getirip kaldırmada `remove()` exception, `poll()` return-value sürümüdür.

English: Let's show a simple queue example: **Türkçe:** Basit bir queue örneği gösterelim:

```
4: Queue<Integer> queue = new LinkedList<>();
5: queue.add(10);
6: queue.add(4);
7: System.out.println(queue.remove()); // 10
8: System.out.println(queue.peek()); // 4
```

English: Lines 5 and 6 add elements to the queue. Line 7 asks the first element waiting the longest to come off the queue. Line 8 checks for the next entry in the queue while leaving it in place. **Türkçe:** Lines 5–6 queue'ya element ekler. Line 7, en uzun süredir bekleyen ilk element'in queue'dan çıkmasını ister. Line 8, sıradaki entry'yi yerinde bırakarak kontrol eder.

English: Next, we move on to the `Deque` interface. Since the `Deque` interface supports double-ended queues, it inherits all `Queue` methods and adds more so that it is clear if we are working with the front or back of the queue. Table 9.4 shows the methods when using it as a double-ended queue. **Türkçe:** Sonra `Deque` interface'ine geçiyoruz. `Deque` interface'i double-ended queue'ları desteklediğinden bütün `Queue` method'larını inherit eder; ayrıca queue'nun front'u mu back'i mi üzerinde çalıştığımızı açık kılacak method'lar ekler. Table 9.4, double-ended queue olarak kullanıldığında bu method'ları gösterir.

Table 9.4 · Deque methods / Deque method'ları

Functionality / İşlev	Throws an exception / Exception fırlatır	Uses return value / Return value kullanır
Add to front / Front'a ekle	<code>public void addFirst(E e)</code>	<code>public boolean offerFirst(E e)</code>
Add to back / Back'e ekle	<code>public void addLast(E e)</code>	<code>public boolean offerLast(E e)</code>
Read from front / Front'tan oku	<code>public E getFirst()</code>	<code>public E peekFirst()</code>
Read from back / Back'ten oku	<code>public E getLast()</code>	<code>public E peekLast()</code>
Get and remove from front / Front'tan getir ve kaldır	<code>public E removeFirst()</code>	<code>public E pollFirst()</code>
Get and remove from back / Back'ten getir ve kaldır	<code>public E removeLast()</code>	<code>public E pollLast()</code>

Türkçe tablo özeti: Front/back'e ekleme için `addFirst()`/`addLast()` ve `offerFirst()`/`offerLast()`; okumak için `getFirst()`/`getLast()` ve `peekFirst()`/`peekLast()`; getirip kaldırmak için `removeFirst()`/`removeLast()` ve `pollFirst()`/`pollLast()` çiftleri vardır. Kaynakta kalın yazılan ilk sütun empty/full probleminde exception fırlatan sürümlerdir.

English: Let's try an example that works with both ends of the queue: **Türkçe:** Queue'nun iki ucuyla da çalışan bir örnek deneyelim:

```
Deque<Integer> deque = new LinkedList<>();
```

English: This is more complicated, so we use Figure 9.6 to show what the queue looks like at each step of the code. **Türkçe:** Bu daha karmaşık olduğundan code'un her adımında queue'nun nasıl görüldüğünü göstermek için Figure 9.6'yı kullanıyoruz.

English: Lines 13 and 14 successfully add an element to the front and back of the queue, respectively. Some queues are limited in size, which would cause offering an element to the queue to fail. You won't encounter a scenario like that on the exam. Line 15 looks at the first element in the queue, but it does not remove it. Lines 16 and 17 remove the elements from the queue, one from each end. This results in an empty queue. Lines 18 and 19 try to look at the first element of the queue, which results in `null`. **Türkçe:** Lines 13–14 sırasıyla queue'nun front ve back tarafına başarıyla bir element ekler. Bazı queue'ların size'i sınırlıdır; bu durumda queue'ya element offer etmek başarısız olabilir. Sınavda böyle bir scenario ile karşılaşmayacaksınız. Line 15 queue'nun ilk element'ına bakar, fakat onu kaldırmaz. Lines 16–17 her uçtan birer element kaldırır ve queue boşalır. Lines 18–19 queue'nun ilk element'ına bakmaya çalışır; sonuç `null`dir.

Figure 9.6 · Working with a Deque / Deque ile çalışma

Code / Kod	Return / Dönüş	Deque after operation / İşlem sonrası Deque (front → back / ön → arka)
13: deque.offerFirst(10);	true	10
14: deque.offerLast(4);	true	10 4
15: deque.peekFirst();	10	10 4
16: deque.pollFirst();	10	4
17: deque.pollLast();	4	empty / boş
18: deque.pollFirst();	null	empty / boş
19: deque.peekFirst();	null	empty / boş

Türkçe şekil özeti: `offerFirst()` front'a, `offerLast()` back'e ekler. `peekFirst()` okumakla yetinir; `pollFirst()` ve `pollLast()` ilgili uçtan getirip kaldırır. Empty deque üzerinde `pollFirst()` ve `peekFirst()` null döndürür.

English: In addition to FIFO queues, there are LIFO (last-in, first-out) queues, which are commonly referred to as stacks. Picture a stack of plates. You always add to or remove from the top of the stack to avoid a mess. Luckily, we can use the same double-ended queue implementations. Different methods are used for clarity, as shown in Table 9.5. **Türkçe:** FIFO queue'lara ek olarak genellikle stack olarak adlandırılan LIFO (last-in, first-out) queue'lar vardır. Bir tabak yığını düşünün. Dağınıklığı önlemek için her zaman stack'in top kısmına ekler veya buradan kaldırırsınız. Neyse ki aynı double-ended queue implementation'larını kullanabiliriz. Table 9.5'te gösterildiği gibi açıklık için farklı method'lar kullanılır.

Table 9.5 · Using a Deque as a stack / Deque'u stack olarak kullanma

Functionality / İşlev	Method
Add to the front/top / Front/top tarafına ekle	<code>public void push(E e)</code>
Remove from the front/top / Front/top tarafından kaldır	<code>public E pop()</code>
Get first element / İlk element'i getir	<code>public E peek()</code>

Türkçe tablo özeti: Stack'in front/top tarafına `push()` ile eklenir, `pop()` ile buradan kaldırılır; ilk element kaldırılmadan `peek()` ile okunur.

English: Let's try another one using the `Deque` as a stack: **Türkçe:** `Deque`u stack olarak kullanan başka bir örnek deneyelim:

```
Deque<Integer> stack = new ArrayDeque<>();
```

English: This time, Figure 9.7 shows what the stack looks like at each step of the code. Lines 13 and 14 successfully put an element on the front/top of the stack. The remaining code looks at the front as well.

Türkçe: Bu kez Figure 9.7 code'un her adımında stack'in nasıl görüldüğünü gösterir. Lines 13–14 stack'in front/top kısmına başarıyla bir element koyar. Kalan code da front tarafına bakar.

English: When using a `Deque`, it is really important to determine if it is being used as a FIFO queue, a LIFO stack, or a double-ended queue. To review, a FIFO queue is like a line of people. You get on in the back and off in the front. A LIFO stack is like a stack of plates. You put the plate on the top and take it off the top. A double-ended queue uses both ends. **Türkçe:** `Deque` kullanırken onun FIFO queue, LIFO stack veya double-ended queue olarak mı kullanıldığını belirlemek gerçekten önemlidir. Tekrarlarsak FIFO queue insan sırası gibidir: back'ten girer, front'tan çıkarsınız. LIFO stack tabak yığını gibidir: tabağı top'a koyar ve top'tan alırsınız. Double-ended queue iki ucu da kullanır.

Figure 9.7 · Working with a stack / Stack ile çalışma

Code / Kod	Return / Dönüş	Stack after operation / İşlem sonrası stack (top → bottom / üst → alt)
13: <code>stack.push(10);</code>	—	10
14: <code>stack.push(4);</code>	—	4 10
15: <code>stack.peek();</code>	4	4 10
16: <code>stack.poll();</code>	4	10
17: <code>stack.poll();</code>	10	empty / boş
18: <code>stack.peek();</code>	null	empty / boş

Türkçe şekil özeti: İki `push()` çağrısından sonra 4 top'tadır. `peek()` 4'ü kaldırmadan okur. İki `poll()` sırasıyla 4 ve 10'u kaldırır; empty stack üzerinde `peek()` null döndürür.

Using the Map Interface

English: You use a `Map` when you want to identify values by a key. For example, when you use the contact list in your phone, you look up “George” rather than looking through each phone number in turn.

Türkçe: Value'ları bir key ile belirlemek istediğinizde `Map` kullanırsınız. Örneğin telefonunuzdaki kişi listesini kullanırken her telefon numarasına sırayla bakmak yerine “George” adını ararsınız.

English: You can envision a `Map` as shown in Figure 9.8. You don't need to know the names of the specific interfaces that the different maps implement, but you do need to know that `TreeMap` is sorted.

Türkçe: Bir `Map`i Figure 9.8'deki gibi canlandırabilirsiniz. Farklı map'lerin implement ettiği belirli interface'lerin adlarını bilmeniz gerekmez; ancak `TreeMap`in sorted olduğunu bilmeniz gerekir.

Figure 9.8 · Example of a Map / Map örneği

Key / Anahtar	Value / Değer
George	555-555-5555
May	777-777-7777

Türkçe şekil özeti: George key'i 555-555-5555, May key'i 777-777-7777 value'suna eşlenmiştir.

English: The main thing that all `Map` classes have in common is that they have keys and values. Beyond that, they each offer different functionality. We look at the implementations you need to know and the available methods. **Türkçe:** Bütün `Map` class'larının ortak temel özelliği key ve value'lara sahip olmalarıdır. Bunun ötesinde her biri farklı functionality sunar. Bilmeniz gereken implementation'lara ve kullanılabilir method'lara bakacağız.

`Map.of()` and `Map.copyOf()`

English: Just like `List` and `Set`, there is a factory method to create a `Map`. You pass any number of pairs of keys and values. **Türkçe:** `List` ve `Set`te olduğu gibi `Map` oluşturmak için bir factory method vardır. İstediğiniz sayıda key ve value çifti geçirirsiniz.

```
Map.of("key1", "value1", "key2", "value2");
```

English: Unlike `List` and `Set`, this is less than ideal. Passing keys and values is harder to read because you have to keep track of which parameter is which. Luckily, there is a better way. `Map` also provides a method that lets you supply key/value pairs. **Türkçe:** `List` ve `Set`ten farklı olarak bu kullanım ideal değildir. Hangi parameter'ın hangisi olduğunu takip etmek zorunda olduğunuz için key ve value'ları geçirmek daha zor okunur. Neyse ki daha iyi bir yol vardır. `Map`, key/value pair sağlamanıza olanak tanıyan başka bir method da sunar.

```
Map.ofEntries(
    Map.entry("key1", "value1"),
    Map.entry("key2", "value2"));
```

English: Now we can't forget to pass a value. If we leave out a parameter, the `entry()` method won't compile. Conveniently, `Map.copyOf(map)` works just like the `List` and `Set` interface `copyOf()` methods. **Türkçe:** Artık bir value geçirmeyi unutamayız. Bir parameter'ı atlarsak `entry()` method'u derlenmez. Kullanışlı biçimde `Map.copyOf(map)`, `List` ve `Set` interface'lerindeki `copyOf()` method'larıyla aynı şekilde çalışır.

Comparing Map Implementations

English: A `HashMap` stores the keys in a hash table. This means that it uses the `hashCode()` method of the keys to retrieve their values more efficiently. **Türkçe:** `HashMap` key'leri hash table'da saklar. Bu, value'ları daha efficient getirmek için key'lerin `hashCode()` method'unu kullandığı anlamına gelir.

English: The main benefit is that adding elements and retrieving the element by key both have constant time. The trade-off is that you lose the order in which you inserted the elements. Most of the time, you aren't concerned with this in a map anyway. If you were, you could use `LinkedHashMap`, but that's not in scope for the exam. **Türkçe:** Ana yararı, element eklemenin ve element'ı key ile getirmenin constant time almasıdır. Karşılığında element'ları eklediğiniz order'ı kaybedersiniz. Çoğu zaman map'te bu order'ı zaten önemsemezsiniz. Önemsemeniz durumunda `LinkedHashMap` kullanabilirdiniz; ancak bu sınav kapsamında değildir.

English: A `TreeMap` stores the keys in a sorted tree structure. The main benefit is that the keys are always in sorted order. Like a `TreeSet`, the trade-off is that adding and checking whether a key is present takes longer as the tree grows larger. **Türkçe:** `TreeMap` key'leri sorted tree structure içinde saklar. Ana yararı key'lerin her zaman sorted order'da olmasıdır. `TreeSet`'te olduğu gibi karşılığında tree büyüdükçe key eklemek ve bir key'in varlığını kontrol etmek daha uzun sürer.

Working with Map Methods

English: Given that `Map` doesn't extend `Collection`, more methods are specified on the `Map` interface. Since there are both keys and values, we need generic type parameters for both. The class uses `K` for key and `V` for value. The methods you need to know for the exam are in Table 9.6. Some of the method signatures are simplified to make them easier to understand. **Türkçe:** `Map`, `Collection`'ı extend etmediği için `Map` interface'inde daha fazla method tanımlanmıştır. Hem key hem value bulunduğu için de generic type parameter gerekir. Class, key için `K`, value için `V` kullanır. Sınav için bilmeniz gereken method'lar Table 9.6'dadır. Bazı method signature'ları anlaşılmasını kolaylaştırmak için sadeleştirilmiştir.

Table 9.6 · Map methods / Map method'ları · Part / Kısım 1

Method	Description / Açıklama
<code>public void clear()</code>	Removes all keys and values from map / Map'teki bütün key ve value'ları kaldırır
<code>public boolean containsKey(Object key)</code>	Returns whether key is in map / Key'in map'te olup olmadığını döndürür
<code>public boolean containsValue(Object value)</code>	Returns whether value is in map / Value'nun map'te olup olmadığını döndürür
<code>public Set<Map.Entry<K,V>> entrySet()</code>	Returns <code>Set</code> of key/value pairs / Key/value pair'lerinden oluşan <code>Set</code> döndürür

Türkçe tablo özeti: `clear()` bütün key/value'ları kaldırır. `containsKey()` key'in, `containsValue()` value'nun map'te olup olmadığını döndürür. `entrySet()` key/value pair'lerinden oluşan `Set` döndürür.

Table 9.6 · Map methods / Map method'ları · Part / Kısım 2

Method	Description / Açıklama
<code>public void forEach(BiConsumer<K key, V value>)</code>	Loops through each key/value pair / Her key/value pair'ini dolaşır
<code>public V get(Object key)</code>	Returns value mapped by key or <code>null</code> if none is mapped / Key'e eşlenen value'yu; eşleme yoksa <code>null</code> döndürür
<code>public V getOrDefault(Object key, V defaultValue)</code>	Returns value mapped by key or default value if none is mapped / Key'e eşlenen value'yu; eşleme yoksa default value'yu döndürür
<code>public boolean isEmpty()</code>	Returns whether map is empty / Map'in empty olup olmadığını döndürür
<code>public Set<K> keySet()</code>	Returns set of all keys / Bütün key'lerden oluşan set'i döndürür
<code>public V merge(K key, V value, Function<V, V> func)</code>	Sets value if key not set; runs function if key is set to determine new value; removes if value is <code>null</code> / Key ayarlı değilse value'yu ayarlar; ayarlıysa yeni value için function'ı çalıştırır; value <code>null</code> ise kaldırır
<code>public V put(K key, V value)</code>	Adds or replaces key/value pair; returns previous value or <code>null</code> / Key/value pair ekler veya değiştirir; önceki value'yu ya da <code>null</code> döndürür
<code>public V putIfAbsent(K key, V value)</code>	Adds value if key not present and returns <code>null</code> ; otherwise returns existing value / Key yoksa value ekleyip <code>null</code> , aksi hâlde mevcut value'yu döndürür
<code>public V remove(Object key)</code>	Removes and returns value mapped to key; returns <code>null</code> if none / Key'e eşlenen value'yu kaldırıp döndürür; yoksa <code>null</code> döndürür
<code>public V replace(K key, V value)</code>	Replaces value for given key if key is set; returns original value or <code>null</code> if none / Key ayarlıysa value'yu değiştirir; özgün value'yu, yoksa <code>null</code> döndürür
<code>public void replaceAll(BiFunction<K, V, V> func)</code>	Replaces each value with results of function / Her value'yu function sonuçlarıyla değiştirir
<code>public int size()</code>	Returns number of entries (key/value pairs) in map / Map'teki entry (key/value pair) sayısını döndürür
<code>public Collection<V> values()</code>	Returns <code>Collection</code> of all values / Bütün value'lardan oluşan <code>Collection</code> döndürür

Türkçe tablo özeti: `forEach()` pair'leri dolaşır; `get()` ve `getOrDefault()` value getirir; `keySet()` key'leri, `values()` value'ları verir. `put()`/`putIfAbsent()` ekler, `remove()` kaldırır, `replace()`/`replaceAll()` değiştirir, `merge()` var olan ve yeni value'yu birleştirir. `isEmpty()` ve `size()` entry sayısını sorgular.

Editor note: Table 9.6 explicitly says its signatures are simplified. The printed `forEach(BiConsumer<K key, V value>)` and `merge(...Function<V, V, V> func)` rows are not valid Java signatures. Java 17 actually declares default `void forEach(BiConsumer<? super K, ? super V> action)` and a `BiFunction<? super V, ? super V, ? extends V>` parameter for `merge()`; the printed rows are retained so no source line is silently changed. **Editör notu:** Table 9.6 signature'ların sadeleştirildiğini açıkça söyler. Basılı `forEach(BiConsumer<K key, V value>)` ve `merge(...Function<V, V, V> func)` satırları geçerli Java signature'ları değildir. Java 17 gerçekte default `void forEach(BiConsumer<? super K, ? super V> action)` signature'ını ve `merge()` için `BiFunction<? super V, ? super V, ? extends V>` parameter'ını bildirir; hiçbir kaynak satırı sessizce değiştirilmesin diye basılı satırlar korunmuştur.

English: While Table 9.6 is a pretty long list of methods, don't worry; many of the names are straightforward. Also, many exist as a convenience. For example, `containsKey()` can be replaced with a `get()` call that checks if the result is `null`. Which one you use is up to you. **Türkçe:** Table 9.6 oldukça uzun bir method listesi olsa da endişelenmeyin; adların çoğu açıktır. Birçoğu convenience olarak bulunur. Örneğin `containsKey()`, sonucun `null` olup olmadığını kontrol eden `get()` çağrısıyla değiştirilebilir. Hangisini kullanacağınız size bağlıdır.

Editor note — null distinction: `map.get(key) != null` is not always equivalent to `map.containsKey(key)`. A map may contain the key with an explicit `null` value; then `get()` returns `null` while `containsKey()` returns `true`. **Editör notu — null ayrımı:** `map.get(key) != null` her zaman `map.containsKey(key)` ile eşdeğer değildir. Map, key'i açıkça `null` value ile tutabilir; bu durumda `get()` `null`, `containsKey()` ise `true` döndürür.

Calling Basic Methods

English: Let's start out by comparing the same code with two `Map` types. First up is `HashMap`. **Türkçe:** Aynı kodu iki `Map` type'ıyla karşılaştırarak başlayalım. Önce `HashMap`:

```
Map<String, String> map = new HashMap<>();
map.put("koala", "bamboo");
map.put("lion", "meat");
map.put("giraffe", "leaf");
String food = map.get("koala"); // bamboo
for (String key : map.keySet())
    System.out.print(key + ","); // koala,giraffe,lion,
```

English: Here we use the `put()` method to add key/value pairs to the map and `get()` to get a value given a key. We also use the `keySet()` method to get all the keys. **Türkçe:** Burada map'e key/value pair eklemek için `put()`, bir key verildiğinde value getirmek için `get()` kullanıyoruz. Bütün key'leri almak için de `keySet()` method'unu kullanıyoruz.

English: Java uses the `hashCode()` of the key to determine the order. The order here happens not to be sorted order or the order in which we typed the values. Now let's look at `TreeMap`. **Türkçe:** Java order'ı belirlemek için key'in `hashCode()` unu kullanır. Buradaki order ne sorted order ne de value'ları yazdığımız order'dır. Şimdi `TreeMap`e bakalım:

```
Map<String, String> map = new TreeMap<>();
map.put("koala", "bamboo");
map.put("lion", "meat");
map.put("giraffe", "leaf");
String food = map.get("koala"); // bamboo
for (String key : map.keySet())
    System.out.print(key + ","); // giraffe,koala,lion,
```

English: `TreeMap` sorts the keys as we would expect. If we called `values()` instead of `keySet()`, the order of the values would correspond to the order of the keys. **Türkçe:** `TreeMap` key'leri beklediğimiz gibi sıralar. `keySet()` yerine `values()` çağırırsaydık value'ların order'ı key'lerin order'ına karşılık gelirdi.

English: With our same map, we can try some boolean checks: **Türkçe:** Aynı map ile bazı boolean kontroller deneyebiliriz:

```
System.out.println(map.contains("lion")); // DOES NOT COMPILE
System.out.println(map.containsKey("lion")); // true
System.out.println(map.containsValue("lion")); // false
System.out.println(map.size()); // 3
map.clear();
System.out.println(map.size()); // 0
System.out.println(map.isEmpty()); // true
```

English: The first line is a little tricky. The `contains()` method is on the `Collection` interface but not the `Map` interface. The next two lines show that keys and values are checked separately. We can see that there are three key/value pairs in our map. Then we clear out the contents of the map and see that there are zero elements and it is empty. **Türkçe:** İlk satır biraz tuzaklıdır. `contains()` method'u `Collection` interface'inde vardır, `Map` interface'inde yoktur. Sonraki iki satır key ve value'ların ayrı kontrol edildiğini gösterir. Map'imizde üç key/value pair olduğunu görürüz. Ardından map contents'ini temizler, sıfır element bulunduğunu ve map'in boş olduğunu görürüz.

English: In the following sections, we show `Map` methods you might not be as familiar with. **Türkçe:** Sonraki kısımlarda daha az aşina olabileceğiniz `Map` method'larını gösteriyoruz.

Iterating through a Map

English: You saw the `forEach()` method earlier in the chapter. Note that it works a little differently on a `Map`. This time, the lambda used by the `forEach()` method has two parameters: the key and the value. Let's look at an example, shown here: **Türkçe:** `forEach()` method'unu bölümün başlarında gördünüz. Bir `Map` üzerinde biraz farklı çalıştığına dikkat edin. Bu kez `forEach()` method'unun kullandığı lambda iki parameter'a sahiptir: key ve value. Şu örneğe bakalım:

```
Map<Integer, Character> map = new HashMap<>();
map.put(1, 'a');
map.put(2, 'b');
map.put(3, 'c');
map.forEach((k, v) -> System.out.println(v));
```

English: The lambda has both the key and value as the parameters. It happens to print out the value but could do anything with the key and/or value. Interestingly, since we don't care about the key, this particular code could have been written with the `values()` method and a method reference instead. **Türkçe:** Lambda hem key'i hem value'yu parameter olarak alır. Bu örnekte value'yu yazdırır; ancak key ve/veya value ile herhangi bir işlem yapabilirdi. İlginç biçimde key'i önemsemediğimiz için bu belirli code, `values()` method'u ve method reference ile de yazılabilirdi.

```
map.values().forEach(System.out::println);
```

English: Another way of going through all the data in a map is to get the key/value pairs in a `Set`. Java has a static interface inside `Map` called `Entry`. It provides methods to get the key and value of each pair. **Türkçe:** Map'teki bütün data üzerinde dolaşmanın başka bir yolu key/value pair'lerini bir `Set` içinde almaktır. Java'nın `Map` içinde `Entry` adlı static interface'i vardır. Bu interface, her pair'in key ve value'sunu getiren method'lar sağlar.

```
map.entrySet().forEach(e ->
    System.out.println(e.getKey() + " " + e.getValue()));
```

Getting Values Safely

English: The `get()` method returns `null` if the requested key is not in the map. Sometimes you prefer to have a different value returned. Luckily, the `getOrDefault()` method makes this easy. Let's compare the two methods: **Türkçe:** İstenen key map'te yoksa `get()` method'u `null` döndürür. Bazen farklı bir value'nun dönmesini tercih edersiniz. Neyse ki `getOrDefault()` bunu kolaylaştırır. İki method'u karşılaştıralım:

```
3: Map<Character, String> map = new HashMap<>();
4: map.put('x', "spot");
5: System.out.println("X marks the " + map.get('x'));
6: System.out.println("X marks the " + map.getOrDefault('x', ""));
7: System.out.println("Y marks the " + map.get('y'));
8: System.out.println("Y marks the " + map.getOrDefault('y', ""));
```

English: This code prints the following: **Türkçe:** Bu code aşağıdakileri yazdırır:

```
X marks the spot
X marks the spot
Y marks the null
Y marks the
```

English: As you can see, lines 5 and 6 have the same output because `get()` and `getOrDefault()` behave the same way when the key is present. They return the value mapped by that key. Lines 7 and 8 give different output, showing that `get()` returns `null` when the key is not present. By contrast, `getOrDefault()` returns the empty string we passed as a parameter. **Türkçe:** Görüldüğü gibi key var olduğunda `get()` ve `getOrDefault()` aynı davrandığı için lines 5–6 aynı output'a sahiptir; o key'e mapped value'yu döndürürler. Lines 7–8 farklı output verir ve key yokken `get()` in `null` döndürdüğünü gösterir. Buna karşılık `getOrDefault()` parameter olarak geçirdiğimiz empty string'i döndürür.

Replacing Values

English: These methods are similar to the `List` version, except a key is involved: **Türkçe:** Bir key'in söz konusu olması dışında bu method'lar `List` sürümüne benzer:

```
21: Map<Integer, Integer> map = new HashMap<>();
22: map.put(1, 2);
23: map.put(2, 4);
24: Integer original = map.replace(2, 10); // 4
25: System.out.println(map);              // {1=2, 2=10}
26: map.replaceAll((k, v) -> k + v);
27: System.out.println(map);              // {1=3, 2=12}
```

English: Line 24 replaces the value for key 2 and returns the original value. Line 26 calls a function and sets the value of each element of the map to the result of that function. In our case, we added the key and value together. **Türkçe:** Line 24, key 2'nin value'sunu değiştirip original value'yu döndürür. Line 26 bir function çağırır ve map'in her element'inin value'sunu bu function'ın sonucuna ayarlar. Bizim örneğimizde key ile value'yu topladık.

Putting if Absent

English: The `putIfAbsent()` method sets a value in the map but skips it if the value is already set to a non-null value. **Türkçe:** `putIfAbsent()` method'u map'te bir value ayarlar; ancak value zaten non-null bir value'ya ayarlıysa işlemi atlar.

```
Map<String, String> favorites = new HashMap<>();
favorites.put("Jenny", "Bus Tour");
favorites.put("Tom", null);
favorites.putIfAbsent("Jenny", "Tram");
favorites.putIfAbsent("Sam", "Tram");
favorites.putIfAbsent("Tom", "Tram");
System.out.println(favorites); // {Tom=Tram, Jenny=Bus Tour, Sam=Tram}
```

English: As you can see, Jenny's value is not updated because one was already present. Sam wasn't there at all, so he was added. Tom was present as a key but had a `null` value. Therefore, he was added as well. **Türkçe:** Görüldüğü gibi zaten bir value bulunduğu için Jenny'nin value'su güncellenmez. Sam hiç bulunmadığından eklenir. Tom key olarak vardır fakat `null` value'ya sahiptir; bu nedenle o da eklenir.

Merging Data

English: The `merge()` method adds logic of what to choose. Suppose we want to choose the ride with the longest name. We can write code to express this by passing a mapping function to the `merge()` method: **Türkçe:** `merge()` method'u neyin seçileceğine ilişkin logic ekler. En uzun ada sahip ride'i seçmek istediğimizi varsayalım. `merge()` method'una mapping function geçirerek bunu ifade eden code yazabiliriz:

```
11: BiFunction<String, String, String> mapper = (v1, v2)
12:     -> v1.length() > v2.length() ? v1 : v2;
```

```

13:
14: Map<String, String> favorites = new HashMap<>();
15: favorites.put("Jenny", "Bus Tour");
16: favorites.put("Tom", "Tram");
17:
18: String jenny = favorites.merge("Jenny", "Skyride", mapper);
19: String tom = favorites.merge("Tom", "Skyride", mapper);
20:
21: System.out.println(favorites); // {Tom=Skyride, Jenny=Bus Tour}
22: System.out.println(jenny);    // Bus Tour
23: System.out.println(tom);      // Skyride

```

English: The code on lines 11 and 12 takes two parameters and returns a value. Our implementation returns the one with the longest name. Line 18 calls this mapping function, and it sees that `Bus Tour` is longer than `Skyride`, so it leaves the value as `Bus Tour`. Line 19 calls this mapping function again. This time, `Tram` is shorter than `Skyride`, so the map is updated. Line 21 prints out the new map contents. Lines 22 and 23 show that the result is returned from `merge()`. **Türkçe:** Lines 11–12'deki code iki parameter alıp bir value döndürür. Implementation'ımız en uzun ada sahip olanı döndürür. Line 18 mapping function'ı çağırır; `Bus Tour`un `Skyride`den uzun olduğunu görüp value'yu `Bus Tour` olarak bırakır. Line 19 mapping function'ı yeniden çağırır. Bu kez `Tram`, `Skyride`den kısa olduğu için map güncellenir. Line 21 yeni map contents'ini yazdırır. Lines 22–23 sonucun `merge()` den döndürüldüğünü gösterir.

English: The `merge()` method also has logic for what happens if `null` values or missing keys are involved. In this case, it doesn't call the `BiFunction` at all, and it simply uses the new value. **Türkçe:** `merge()` method'u `null` value veya missing key söz konusu olduğunda ne olacağına ilişkin logic de içerir. Bu durumda `BiFunction`ı hiç çağırılmaz ve doğrudan yeni value'yu kullanır.

```

BiFunction<String, String, String> mapper =
    (v1, v2) -> v1.length() > v2.length() ? v1 : v2;
Map<String, String> favorites = new HashMap<>();
favorites.put("Sam", null);
favorites.merge("Tom", "Skyride", mapper);
favorites.merge("Sam", "Skyride", mapper);
System.out.println(favorites); // {Tom=Skyride, Sam=Skyride}

```

English: Notice that the mapping function isn't called. If it were, we'd have a `NullPointerException`. The mapping function is used only when there are two actual values to decide between. **Türkçe:** Mapping function'ın çağırılmadığına dikkat edin. Çağrılseydi `NullPointerException` oluşurdu. Mapping function yalnız aralarında seçim yapılacak iki gerçek value bulunduğu kullanılır.

English: The final thing to know about `merge()` is what happens when the mapping function is called and returns `null`. The key is removed from the map when this happens: **Türkçe:** `merge()` hakkında bilinmesi gereken son nokta, mapping function çağırılıp `null` döndürdüğünde ne olduğudur. Böyle olduğunda key map'ten kaldırılır:

```
BiFunction<String, String, String> mapper = (v1, v2) -> null;
Map<String, String> favorites = new HashMap<>();
favorites.put("Jenny", "Bus Tour");
favorites.put("Tom", "Bus Tour");
favorites.merge("Jenny", "Skyride", mapper);
favorites.merge("Sam", "Skyride", mapper);
System.out.println(favorites); // {Tom=Bus Tour, Sam=Skyride}
```

English: Tom was left alone since there was no `merge()` call for that key. Sam was added since that key was not in the original list. Jenny was removed because the mapping function returned `null`. **Türkçe:** O key için `merge()` çağırısı olmadığından Tom'a dokunulmadı. Sam'in key'i original list'te bulunmadığından eklendi. Mapping function `null` döndürdüğü için Jenny kaldırıldı.

Editor note: The object in this example is a `Map`, not a `List`. The source's phrase "original list" refers to the original map contents. **Editör notu:** Bu örnekteki object bir `List` değil, `Map` tir. Kaynaktaki "original list" ifadesi map'in başlangıç içeriğini anlatmaktadır.

English: Table 9.7 shows all of these scenarios as a reference. **Türkçe:** Table 9.7 bütün bu scenario'ları başvuru amacıyla gösterir.

Table 9.7 · Behavior of the `merge()` method / `merge()` method'unun davranışı

If the requested key... / İstenen key...	And mapping function returns... / Mapping function şunu döndürürse...	Then... / Sonuç...
Has a <code>null</code> value in map / Map'te <code>null</code> value'ya sahipse	N/A (mapping function not called) / Uygulanmaz (mapping function çağrılmaz)	Update key's value in map with <code>value</code> parameter / Map'teki key value'sunu <code>value</code> parameter'ıyla güncelle
Has a non-null value in map / Map'te non-null value'ya sahipse	<code>null</code>	Remove key from map / Key'i map'ten kaldır
Has a non-null value in map / Map'te non-null value'ya sahipse	A non-null value / Non-null bir value	Set key to mapping function result / Key'i mapping function sonucuna eşle
Is not in map / Map'te yoksa	N/A (mapping function not called) / Uygulanmaz (mapping function çağrılmaz)	Add key with <code>value</code> parameter to map directly without calling mapping function / Mapping function'ı çağırmadan key'i <code>value</code> parameter'ıyla doğrudan map'e ekle

Türkçe tablo özeti: Key `null` value'ya sahipse veya map'te yoksa mapping function çağrılmaz ve `value` parameter'ı doğrudan kullanılır. Key non-null value'ya sahipse function çağrılır; sonuç `null` ise key kaldırılır, non-null ise key bu sonuca eşlenir.

Comparing Collection Types

English: We conclude this section with a review of all the collection classes. Make sure that you can fill in Table 9.8 to compare the four collection types from memory. **Türkçe:** Bu kısmı bütün collection class'larını gözden geçirerek tamamlıyoruz. Dört collection type'ını hafızadan karşılaştırmak için Table 9.8'i doldurabildiğinizden emin olun.

Table 9.8 · Java Collections Framework types / Java Collections Framework type'ları

Type	Can contain duplicate elements? / Duplicate element içerebilir mi?	Elements always ordered? / Element'lar her zaman ordered mı?	Has keys and values? / Key ve value var mı?	Must add/remove in specific order? / Belirli sırada ekleme-kaldırma gerekir mi?
List	Yes / Evet	Yes (by index) / Evet (index'e göre)	No / Hayır	No / Hayır
Map	Yes (for values) / Evet (value'lar için)	No / Hayır	Yes / Evet	No / Hayır
Queue	Yes / Evet	Yes (retrieved in defined order) / Evet (tanımlı order'da getirilir)	No / Hayır	Yes / Evet
Set	No / Hayır	No / Hayır	No / Hayır	No / Hayır

Türkçe tablo özeti: **List** duplicate kabul eder ve index'e göre ordered'dır. **Map**te value'lar duplicate olabilir; key/value yapısı vardır. **Queue** duplicate kabul eder ve defined retrieval order'a göre ekleme/kaldırma ister. **Set** duplicate kabul etmez ve order garantisi vermez.

English: Additionally, make sure you can fill in Table 9.9 to describe the types on the exam. **Türkçe:** Ayrıca sınavdaki type'ları açıklamak için Table 9.9'u doldurabildiğinizden emin olun.

Table 9.9 · Collection attributes / Collection özellikleri

Type	Java Collections Framework interface'i	Sorted? / Sıralı mı?	Calls hashCode()? / hashCode() çağırır mı?	Calls compareTo()? / compareTo() çağırır mı?
ArrayDeque	Deque	No / Hayır	No / Hayır	No / Hayır
ArrayList	List	No / Hayır	No / Hayır	No / Hayır
HashMap	Map	No / Hayır	Yes / Evet	No / Hayır
HashSet	Set	No / Hayır	Yes / Evet	No / Hayır
LinkedList	List, Deque	No / Hayır	No / Hayır	No / Hayır
TreeMap	Map	Yes / Evet	No / Hayır	Yes / Evet
TreeSet	Set	Yes / Evet	No / Hayır	Yes / Evet

Türkçe tablo özeti: `HashMap` ve `HashSet`, `hashCode()` çağırır. `TreeMap` ve `TreeSet` sorted'dır ve `compareTo()` çağırır. `ArrayDeque`, `ArrayList` ve `LinkedList` bu iki karşılaştırma mekanizmasını kullanmaz.

English: Next, the exam expects you to know which data structures allow `null` values. The data structures that involve sorting do not allow `null` values. **Türkçe:** Sınav ayrıca hangi data structure'ların `null` value'lara izin verdiğini bilmenizi bekler. Sorting içeren data structure'lar `null` value'lara izin vermez.

Editor note — precise Java 17 rule: Natural-order `TreeSet` elements and natural-order `TreeMap` keys cannot be `null`. A `TreeMap` can nevertheless store `null values`. A custom comparator can also define an order for `null`, so the source sentence is a useful exam shortcut rather than a universal API rule.

Editör notu — kesin Java 17 kuralı: Natural order kullanan `TreeSet` element'ları ve `TreeMap` key'leri `null` olamaz. Buna karşılık `TreeMap`, `null value` saklayabilir. Custom comparator `null` için de order tanımlayabildiğinden kaynak cümlesi evrensel API kuralı değil, kullanışlı bir sınav kısayoludur.

English: Finally, the exam expects you to be able to choose the right collection type given a description of a problem. We recommend first identifying which type of collection the question is asking about. Figure out whether you are looking for a list, map, queue, or set. This lets you eliminate a number of answers. Then you can figure out which of the remaining choices is the best answer. **Türkçe:** Son olarak sınav, bir problem description'ı verildiğinde doğru collection type'ını seçebilmenizi bekler. Önce sorunun hangi collection type'ını istediğini belirlemenizi öneririz. List, map, queue veya set mi aradığınızı bulun. Böylece birçok answer'ı eleyebilirsiniz. Ardından kalan choice'lar içinde en iyi answer'ı belirleyebilirsiniz.

Older Collections

English: There are a few collections that are no longer on the exam but that you might come across in older code. All three were early Java data structures you could use with threads: **Türkçe:** Artık sınavda bulunmayan fakat eski code'da karşılaşılabileceğiniz birkaç collection vardır. Üçü de thread'lerle kullanabildiğiniz erken dönem Java data structure'larıydı:

English: `Vector`: Implements `List`. **Türkçe:** `Vector`: `List`i implement eder.

English: `Hashtable`: Implements `Map`. **Türkçe:** `Hashtable`: `Map`i implement eder.

English: `Stack`: Implements `Queue`. **Türkçe:** `Stack`: `Queue`yu implement eder.

Editor note — Java 17 type hierarchy: `java.util.Stack` extends `Vector`; it does not implement `Queue`. For new LIFO code, use a `Deque` implementation such as `ArrayDeque`. **Editör notu — Java 17 type hierarchy:** `java.util.Stack`, `Vector`i extends eder; `Queue`yu implement etmez. Yeni LIFO code için `ArrayDeque` gibi bir `Deque` implementation'ı kullanın.

English: These classes are rarely used anymore, as there are much better concurrent alternatives that we cover in Chapter 13. **Türkçe:** Chapter 13'te ele aldığımız çok daha iyi concurrent alternatifler bulunduğu için bu class'lar artık nadiren kullanılır.

Sorting Data

English: We discussed “order” for the `TreeSet` and `TreeMap` classes. For numbers, order is obvious—it is numerical order. For `String` objects, order is defined according to the Unicode character mapping.

Türkçe: `TreeSet` ve `TreeMap` class'ları için “order”ı tartıştık. Number'larda order açıktır: numerical order. `String` object'lerinde order, Unicode character mapping'e göre tanımlanır.

English — memory note: When working with a `String`, remember that numbers sort before letters, and uppercase letters sort before lowercase letters. **Türkçe — hafıza notu:** `String` ile çalışırken number character'ların letter'lardan, uppercase letter'ların lowercase letter'lardan önce sıralandığını unutmayın.

English: We use `Collections.sort()` in many of these examples. It returns `void` because the method parameter is what gets sorted. **Türkçe:** Bu örneklerin çoğunda `Collections.sort()` kullanıyoruz. Sıralanan şey method parameter'ı olduğundan `void` döndürür.

English: You can also sort objects that you create yourself. Java provides an interface called `Comparable`. If your class implements `Comparable`, it can be used in data structures that require comparison. There is also a class called `Comparator`, which is used to specify that you want to use a different order than the object itself provides. **Türkçe:** Kendiniz oluşturduğunuz object'leri de sıralayabilirsiniz. Java, `Comparable` adlı bir interface sağlar. Class'ınız `Comparable` implement ederse comparison gerektiren data structure'larda kullanılabilir. Ayrıca object'in kendisinin sağladığından farklı bir order kullanmak istediğinizi belirtmek için kullanılan `Comparator` adlı bir class da vardır.

Editor note: In Java 17, `Comparator` is an interface, not a class. The preceding English sentence preserves the source wording; all examples and explanations below use the technically correct interface classification. **Editör notu:** Java 17'de `Comparator` class değil, interface'tir. Önceki English cümle kaynak ifadesini korur; aşağıdaki bütün örnek ve açıklamalar teknik olarak doğru interface sınıflandırmasını kullanır.

English: `Comparable` and `Comparator` are similar enough to be tricky. The exam likes to see if it can trick you into mixing up the two. Don't be confused! In this section, we discuss `Comparable` first. Then, as we go through `Comparator`, we point out all of the differences. **Türkçe:** `Comparable` ve `Comparator` tuzak oluşturacak kadar benzerdir. Sınav ikisini karıştırıp karıştırmadığınızı ölçmeyi sever. Kafanız karışmasın! Bu kısımda önce `Comparable` ele alıyor, sonra `Comparator` boyunca bütün farkları gösteriyoruz.

Creating a Comparable Class

English: The `Comparable` interface has only one method. In fact, this is the entire interface: **Türkçe:** `Comparable` interface'inin yalnız bir method'u vardır. Aslında interface'in tamamı şöyledir:

```
public interface Comparable<T> {  
    int compareTo(T o);  
}
```

English: The generic `T` lets you implement this method and specify the type of your object. This lets you avoid a cast when implementing `compareTo()`. Any object can be `Comparable`. **Türkçe:** Generic `T`, method'u implement ederken object'inizin type'ını belirtmenizi sağlar. Böylece `compareTo()` implementation'ında cast gereksinimi ortadan kalkar. Her object `Comparable` olabilir.

English: For example, we have a bunch of ducks and want to sort them by name. First, we update the class declaration to inherit `Comparable<Duck>`, and then we implement the `compareTo()` method: **Türkçe:** Örneğin birçok ördeğimiz olsun ve onları ada göre sıralamak isteyelim. Önce class declaration'ı `Comparable<Duck>` inherit edecek şekilde günceller, ardından `compareTo()` method'unu implement ederiz:

```
import java.util.*;  
  
public class Duck implements Comparable<Duck> {  
    private String name;  
  
    public Duck(String name) {  
        this.name = name;  
    }  
  
    public String toString() {  
        return name; // use readable output  
    }  
}
```

```
public int compareTo(Duck d) {  
    return name.compareTo(d.name); // sorts ascendingly by name  
}  
  
public static void main(String[] args) {  
    var ducks = new ArrayList<Duck>();  
    ducks.add(new Duck("Quack"));  
    ducks.add(new Duck("Puddles"));  
    Collections.sort(ducks); // sort by name  
    System.out.println(ducks); // [Puddles, Quack]  
}
```

English: Without implementing that interface, all we have is a method named `compareTo()`, but it wouldn't be a `Comparable` object. We could also implement `Comparable<Object>` or some other class for `T`, but this wouldn't be as useful for sorting a group of `Duck` objects. **Türkçe:** Bu interface'i implement etmeden yalnız `compareTo()` adlı bir method'umuz olur; object ise `Comparable` olmazdı. `T` için `Comparable<Object>` veya başka bir class da implement edebilirdik, ancak bu bir `Duck` object grubunu sıralamak için o kadar yararlı olmazdı.

English — output note: The `Duck` class overrides the `toString()` method from `Object`, which we described in Chapter 8. This override provides useful output when printing out ducks. Without this override, the output would be something like `[Duck@70dea4e, Duck@5c647e05]`—hardly useful in seeing which duck's name comes first. **Türkçe — output notu:** `Duck` class'ı Chapter 8'de açıkladığımız `Object.toString()` method'unu override eder. Bu override örnekleri yazdırırken anlamlı output sağlar. Override olmasaydı output `[Duck@70dea4e, Duck@5c647e05]` benzeri olurdu; bu da hangi ördeğin adının önce geldiğini görmek için hiç yararlı değildir.

English: Finally, the `Duck` class implements `compareTo()`. Since `Duck` is comparing objects of type `String` and the `String` class already has a `compareTo()` method, it can just delegate. We still need to know what the `compareTo()` method returns so that we can write our own. There are three rules to know: **Türkçe:** Son olarak `Duck` class'ı `compareTo()` yu implement eder. `Duck`, `String` type'indeki object'leri karşılaştırdığından ve `String` class'ında zaten `compareTo()` bulunduğundan çağırısı ona delegate edebilir. Kendi implementation'ımızı yazabilmek için yine de `compareTo()` nun ne döndürdüğünü bilmeliyiz. Bilinecek üç kural vardır:

English: The number 0 is returned when the current object is equivalent to the argument to `compareTo()`. **Türkçe:** Current object, `compareTo()` argument'ına equivalent olduğunda 0 döndürülür.

English: A negative number (less than 0) is returned when the current object is smaller than the argument to `compareTo()`. **Türkçe:** Current object, `compareTo()` argument'ından küçük olduğunda negative number, yani 0'dan küçük bir değer döndürülür.

English: A positive number (greater than 0) is returned when the current object is larger than the argument to `compareTo()`. **Türkçe:** Current object, `compareTo()` argument'ından büyük olduğunda positive number, yani 0'dan büyük bir değer döndürülür.

English: Let's look at an implementation of `compareTo()` that compares numbers instead of `String` objects: **Türkçe:** `String` object'leri yerine number'ları karşılaştıran bir `compareTo()` implementation'ına bakalım:

```
1: public class Animal implements Comparable<Animal> {
2:     private int id;
3:     public int compareTo(Animal a) {
4:         return id - a.id; // sorts ascending by id
5:     }
6:     public static void main(String[] args) {
7:         var a1 = new Animal();
```

```
8:         var a2 = new Animal();
9:         a1.id = 5;
10:        a2.id = 7;
11:        System.out.println(a1.compareTo(a2)); // -2
12:        System.out.println(a1.compareTo(a1)); // 0
13:        System.out.println(a2.compareTo(a1)); // 2
14:    } }
```

English: Lines 7 and 8 create two `Animal` objects. Lines 9 and 10 set their `id` values. This is not a good way to set instance variables. It would be better to use a constructor or setter method. Since the exam shows nontraditional code to make sure that you understand the rules, we throw in some nontraditional code as well. **Türkçe:** Lines 7–8 iki `Animal` object oluşturur. Lines 9–10 bunların `id` value'larını ayarlar. Bu, instance variable ayarlamamanın iyi bir yolu değildir; constructor veya setter method kullanmak daha iyi olurdu. Sınav kuralları anlayıp anlamadığınızı görmek için alışılmadık code gösterdiğinden biz de alışılmadık code ekliyoruz.

English: Lines 3–5 show one way to compare two `int` values. We could have used `Integer.compare(id, a.id)` instead. Be sure you can recognize both approaches. **Türkçe:** Lines 3–5 iki `int` value'yu karşılaştırmanın bir yolunu gösterir. Bunun yerine `Integer.compare(id, a.id)` kullanabilirdik. İki yaklaşımı da tanıyabildiğinizden emin olun.

English — memory note: Remember that `id - a.id` sorts in ascending order, and `a.id - id` sorts in descending order. **Türkçe — hafıza notu:** `id - a.id` ifadesinin ascending order, `a.id - id` ifadesinin descending order ürettiğini unutmayın.

Java 17 / OCP technical note: Subtraction is easy to recognize in exam code, but extreme `int` values can overflow and reverse the sign, breaking the comparator contract. Prefer `Integer.compare(id, a.id)` for ascending order and `Integer.compare(a.id, id)` for descending order. **Java 17 / OCP teknik notu:** Çıkarma biçimini sınav code'unda tanımak yararlıdır; ancak uç `int` value'larında overflow işareti tersine çevirip comparator contract'ını bozabilir. Ascending order için `Integer.compare(id, a.id)`, descending order için `Integer.compare(a.id, id)` güvenli tercihtir.

English: Lines 11–13 confirm that we've implemented `compareTo()` correctly. Line 11 compares a smaller `id` to a larger one, and therefore it prints a negative number. Line 12 compares animals with the same `id`, and therefore it prints 0. Line 13 compares a larger `id` to a smaller one, and therefore it returns a positive number. **Türkçe:** Lines 11–13 `compareTo()` yu doğru implement ettiğimizi doğrular. Line 11 küçük `id`'yi büyük `id` ile karşılaştırıp negative number yazdırır. Line 12 aynı `id`'ye sahip hayvanları karşılaştırıp 0 yazdırır. Line 13 büyük `id`'yi küçük `id` ile karşılaştırıp positive number döndürür.

Casting the `compareTo()` Argument

English: When dealing with legacy code or code that does not use generics, the `compareTo()` method requires a cast since it is passed an `Object`. **Türkçe:** Legacy code veya generics kullanmayan code ile çalışırken `compareTo()` ya `Object` geçirildiğinden method bir cast gerektirir.

```
public class LegacyDuck implements Comparable {
    private String name;
    public int compareTo(Object obj) {
        LegacyDuck d = (LegacyDuck) obj; // cast because no generics
        return name.compareTo(d.name);
    }
}
```

English: Since we don't specify a generic type for `Comparable`, Java assumes that we want an `Object`, which means that we have to cast to `LegacyDuck` before accessing instance variables on it. **Türkçe:** `Comparable` için generic type belirtmediğimizden Java `Object` istediğimizi varsayar; bu da onun instance variable'larına erişmeden önce `LegacyDuck`a cast etmemiz gerektiği anlamına gelir.

Checking for `null`

English: When working with `Comparable` and `Comparator` in this chapter, we tend to assume the data has values, but this is not always the case. When writing your own compare methods, you should check the data before comparing it if it is not validated ahead of time. **Türkçe:** Bu bölümde `Comparable` ve `Comparator` ile çalışırken data'nın value taşıdığını varsayma eğilimindeyiz; ancak durum her zaman böyle değildir. Kendi compare method'larınızı yazarken data önceden validate edilmediyse karşılaştırmadan önce kontrol etmelisiniz.

```
public class MissingDuck implements Comparable<MissingDuck> {
    private String name;
    public int compareTo(MissingDuck quack) {
        if (quack == null)
            throw new IllegalArgumentException("Poorly formed duck!");
        if (this.name == null && quack.name == null)
            return 0;
        else if (this.name == null) return -1;
        else if (quack.name == null) return 1;
        else return name.compareTo(quack.name);
    }
}
```

English: This method throws an exception if it is passed a `null` `MissingDuck` object. What about the ordering? If the name of a duck is `null`, it's sorted first. **Türkçe:** Bu method'a `null` bir `MissingDuck` object geçirilirse exception fırlatır. Peki ordering nasıl olur? Bir ördeğin adı `null` ise önce sıralanır.

Keeping `compareTo()` and `equals()` Consistent

English: If you write a class that implements `Comparable`, you introduce new business logic for determining equality. The `compareTo()` method returns 0 if two objects are equal, while your `equals()` method returns `true` if two objects are equal. A natural ordering that uses `compareTo()` is said to be consistent with `equals` if, and only if, `x.equals(y)` is `true` whenever `x.compareTo(y)` equals 0. Similarly, `x.equals(y)` must be `false` whenever `x.compareTo(y)` is not 0. You are strongly encouraged to make your `Comparable` classes consistent with `equals` because not all collection classes behave predictably if the `compareTo()` and `equals()` methods are not consistent. **Türkçe:** `Comparable` implement eden class yazarsanız equality'yi belirlemek için yeni business logic getirirsiniz. İki object equal olduğunda `compareTo()` 0, `equals()` ise `true` döndürür. `compareTo()` kullanan natural ordering; yalnız ve ancak `x.compareTo(y)` 0 olduğunda `x.equals(y)` de `true` ise `equals` ile consistent kabul edilir. Benzer biçimde `x.compareTo(y)` 0 değilken `x.equals(y)` `false` olmalıdır. `compareTo()` ve `equals()` consistent olmadığında bütün collection class'ları predictable davranmadığından `Comparable` class'larınızı `equals` ile consistent yapmanız kuvvetle önerilir.

English: For example, the following `Product` class defines a `compareTo()` method that is not consistent with `equals`. **Türkçe:** Örneğin aşağıdaki `Product` class'ı `equals` ile consistent olmayan bir `compareTo()` method'u tanımlar:

```
public class Product implements Comparable<Product> {
    private int id;
    private String name;

    public int hashCode() { return id; }
    public boolean equals(Object obj) {
        if (!(obj instanceof Product)) return false;
        var other = (Product) obj;
        return this.id == other.id;
    }
    public int compareTo(Product obj) {
        return this.name.compareTo(obj.name);
    }
}
```

English: You might be sorting `Product` objects by name, but names are not unique. The `compareTo()` method does not have to be consistent with `equals`. One way to fix that is to use a `Comparator` to define the sort elsewhere. **Türkçe:** `Product` object'lerini ada göre sıralıyor olabilirsiniz; fakat adlar unique değildir. `compareTo()` method'unun `equals` ile consistent olması zorunlu değildir. Bunu düzeltmenin bir yolu sort'u başka yerde tanımlamak için `Comparator` kullanmaktır.

English: Now that you know how to implement `Comparable` objects, you get to look at a `Comparator` and focus on the differences. **Türkçe:** `Comparable` object'lerin nasıl implement edildiğini bildiğinize göre artık `Comparators` bakıp farklara odaklanabilirsiniz.

Comparing Data with a Comparator

English: Sometimes you want to sort an object that did not implement `Comparable`, or you want to sort objects in different ways at different times. Suppose that we add weight to our `Duck` class. We now have the following: **Türkçe:** Bazen `Comparable` implement etmeyen bir object'i sıralamak veya object'leri farklı zamanlarda farklı yollarla sıralamak istersiniz. `Duck` class'ımıza weight eklediğimizi varsayalım. Artık şunlara sahibiz:

```
1: import java.util.ArrayList;
2: import java.util.Collections;
3: import java.util.Comparator;
4:
5: public class Duck implements Comparable<Duck> {
6:     private String name;
7:     private int weight;
8:
9:     // Assume getters/setters/constructors provided
10:
11:     public String toString() { return name; }
12:
13:     public int compareTo(Duck d) {
14:         return name.compareTo(d.name);
15:     }
16:
17:     public static void main(String[] args) {
18:         Comparator<Duck> byWeight = new Comparator<Duck>() {
19:             public int compare(Duck d1, Duck d2) {
20:                 return d1.getWeight() - d2.getWeight();
21:             }
22:         };
23:         var ducks = new ArrayList<Duck>();
24:         ducks.add(new Duck("Quack", 7));
25:         ducks.add(new Duck("Puddles", 10));
26:         Collections.sort(ducks);
27:         System.out.println(ducks); // [Puddles, Quack]
28:         Collections.sort(ducks, byWeight);
29:
30:         System.out.println(ducks); // [Quack, Puddles]
31:     }
32: }
```

English: First, notice that this program imports `java.util.Comparator` on line 3. We don't always show imports since you can assume they are present if not shown. Here, we do show the import to call attention to the fact that `Comparable` and `Comparator` are in different packages: `java.lang` and `java.util`, respectively. That means `Comparable` can be used without an import statement, while `Comparator` cannot. **Türkçe:** Önce programın line 3'te `java.util.Comparator` import ettiğine dikkat edin. Gösterilmediklerinde var olduklarını varsayabileceğiniz için import'ları her zaman göstermeyiz. Burada `Comparable` ve `Comparator`'ın sırasıyla `java.lang` ve `java.util` olmak üzere farklı package'larda bulunduğuna dikkat çekmek için import'u gösteriyoruz. Dolayısıyla `Comparable` import statement olmadan kullanılabilirken `Comparator` kullanılamaz.

English: The `Duck` class itself can define only one `compareTo()` method. In this case, name was chosen. If we want to sort by something else, we have to define that sort order outside the `compareTo()` method using a separate class or lambda expression. **Türkçe:** `Duck` class'ının kendisi yalnız bir `compareTo()` method'u tanımlayabilir. Bu örnekte name seçilmiştir. Başka bir şeye göre sıralamak istersek sort order'ı `compareTo()` dışında ayrı bir class veya lambda expression kullanarak tanımlamalıyız.

English: Lines 18–22 of the `main()` method show how to define a `Comparator` using an inner class. On lines 26–29, we sort without the `Comparator` and then with the `Comparator` to see the difference in output. **Türkçe:** `main()` method'unun lines 18–22'si inner class kullanarak `Comparator` tanımlamayı gösterir. Lines 26–29'da output farkını görmek için önce `Comparator` olmadan, ardından `Comparator` ile sıralarız.

English: `Comparator` is a functional interface since there is only one abstract method to implement. This means that we can rewrite the `Comparator` on lines 18–22 using a lambda expression, as shown here: **Türkçe:** Implement edilecek yalnız bir abstract method bulunduğundan `Comparator` functional interface'tir. Bu nedenle lines 18–22'deki `Comparator`ı aşağıdaki lambda expression ile yeniden yazabiliriz:

```
Comparator<Duck> byWeight =  
    (d1, d2) -> d1.getWeight() - d2.getWeight();
```

English: Alternatively, we can use a method reference and a helper method to specify that we want to sort by weight. **Türkçe:** Alternatif olarak weight'e göre sıralamak istediğimizi belirtmek için method reference ve helper method kullanabiliriz.

```
Comparator<Duck> byWeight = Comparator.comparing(Duck::getWeight);
```

English: In this example, `Comparator.comparing()` is a static interface method that creates a `Comparator` given a lambda expression or method reference. Convenient, isn't it? **Türkçe:** Bu örnekte `Comparator.comparing()`, lambda expression veya method reference verildiğinde `Comparator` oluşturan static interface method'udur. Kullanışlı, değil mi?

Is Comparable a Functional Interface?

English: We said that `Comparator` is a functional interface because it has a single abstract method. `Comparable` is also a functional interface since it also has a single abstract method. However, using a lambda for `Comparable` would be silly. The point of `Comparable` is to implement it inside the object being compared. **Türkçe:** Tek bir abstract method'a sahip olduğu için `Comparator`ın functional interface olduğunu söyledik. `Comparable` da tek bir abstract method'a sahip olduğundan functional interface'tir. Ancak `Comparable` için lambda kullanmak anlamsız olurdu. `Comparable`ın amacı onu karşılaştırılan object içinde implement etmektir.

Comparing Comparable and Comparator

English: There are several differences between `Comparable` and `Comparator`. We've listed them for you in Table 9.10. **Türkçe:** `Comparable` ve `Comparator` arasında çeşitli farklar vardır. Bunları Table 9.10'da listeledik.

Table 9.10 · Comparison of Comparable and Comparator / Comparable ve Comparator karşılaştırması

Difference / Fark	Comparable	Comparator
Package name / Package adı	<code>java.lang</code>	<code>java.util</code>
Interface must be implemented by class comparing? / Karşılaştırılan class interface'i implement etmeli mi?	Yes / Evet	No / Hayır
Method name in interface / Interface'teki method adı	<code>compareTo()</code>	<code>compare()</code>
Number of parameters / Parameter sayısı	1	2
Common to declare using a lambda / Lambda ile bildirilmesi yaygın mı?	No / Hayır	Yes / Evet

Türkçe tablo özeti: `Comparable`, `java.lang` içindedir; karşılaştırılan class tarafından implement edilir, tek parameter'lı `compareTo()` yu kullanır ve genellikle lambda ile bildirilmez. `Comparator`, `java.util` içindedir; class'ın onu implement etmesi gerekmez, iki parameter'lı `compare()` yu kullanır ve sıklıkla lambda ile bildirilir.

English — exam note: Memorize this table—really. The exam will try to trick you by mixing up the two and seeing if you can catch it. Do you see why this doesn't compile? **Türkçe — sınav notu:** Bu tabloyu gerçekten ezberleyin. Sınav ikisini karıştırarak tuzağı yakalayıp yakalayamadığınızı ölçmeye çalışacaktır. Aşağıdaki kodun neden derlenmediğini görüyor musunuz?

```
var byWeight = new Comparator<Duck>() { // DOES NOT COMPILE
    public int compareTo(Duck d1, Duck d2) {
        return d1.getWeight() - d2.getWeight();
    }
};
```

English: The method name is wrong. A `Comparator` must implement a method named `compare()`. Pay special attention to method names and the number of parameters when you see `Comparator` and `Comparable` in questions. **Türkçe:** Method adı yanlıştır. `Comparator`, `compare()` adlı method'u implement etmelidir. Sorularda `Comparator` ve `Comparable` gördüğünüzde method adlarına ve parameter sayılarına özellikle dikkat edin.

Comparing Multiple Fields

English: When writing a `Comparator` that compares multiple instance variables, the code gets a little messy. Suppose that we have a `Squirrel` class, as shown here: **Türkçe:** Birden fazla instance variable'ı karşılaştıran `Comparator` yazarken code biraz karışık olur. Aşağıdaki gibi bir `Squirrel` class'ımız olduğunu varsayalım:

```
public class Squirrel {
    private int weight;
    private String species;
    // Assume getters/setters/constructors provided
}
```

English: We want to write a `Comparator` to sort by species name. If two squirrels are from the same species, we want to sort the one that weighs the least first. We could do this with code that looks like this:

Türkçe: Species adına göre sıralamak için `Comparator` yazmak istiyoruz. İki sincap aynı species'tense daha hafif olanı önce sıralamak istiyoruz. Bunu aşağıdaki gibi code ile yapabiliriz:

```
public class MultiFieldComparator implements Comparator<Squirrel> {
    public int compare(Squirrel s1, Squirrel s2) {
        int result = s1.getSpecies().compareTo(s2.getSpecies());
        if (result != 0) return result;
        return s1.getWeight() - s2.getWeight();
    }
}
```

English: This works assuming no species' names are `null`. It checks one field. If they don't match, we are finished sorting. If they do match, it looks at the next field. This isn't easy to read, though. It is also easy to get wrong. Changing `!=` to `==` breaks the sort completely. **Türkçe:** Hiçbir species adının `null` olmadığını varsayarsak bu çalışır. Bir field'ı kontrol eder. Eşleşmezlerse sorting biter; eşleşirlerse sonraki field'a bakar. Bununla birlikte okunması kolay değildir ve yanlış yazılması da kolaydır. `!=` yerine `==` yazmak sort'u tamamen bozar.

English: Alternatively, we can use method references and build the `Comparator`. This code represents logic for the same comparison: **Türkçe:** Alternatif olarak method reference'lar kullanıp `Comparator` oluşturabiliriz. Aşağıdaki code aynı comparison logic'ini temsil eder:

```
Comparator<Squirrel> c =
    Comparator.comparing(Squirrel::getSpecies)
                .thenComparingInt(Squirrel::getWeight);
```

English: This time, we chain the methods. First, we create a `Comparator` on species ascending. Then, if there is a tie, we sort by weight. We can also sort in descending order. Some methods on `Comparator`, like `thenComparingInt()`, are default methods. **Türkçe:** Bu kez method'ları chain ederiz. Önce species'e göre ascending bir `Comparator` oluştururuz. Ardından tie varsa weight'e göre sıralarız. Descending order'da da sıralayabiliriz. `thenComparingInt()` gibi bazı `Comparator` method'ları default method'dur.

English: Suppose we want to sort in descending order by species. **Türkçe:** Species'e göre descending order'da sıralamak istediğimizi varsayalım.

```
var c = Comparator.comparing(Squirrel::getSpecies).reversed();
```

English: Table 9.11 shows the helper methods you should know for building a `Comparator`. We've omitted the parameter types to keep you focused on the methods. They use many of the functional interfaces you learned about in the previous chapter. **Türkçe:** Table 9.11, `Comparator` oluşturmak için bilmeniz gereken helper method'ları gösterir. Method'lara odaklanmanız için parameter type'larını çıkardık. Önceki bölümde öğrendiğiniz birçok functional interface'i kullanırlar.

Table 9.11 · Helper static methods for building a `Comparator` / `Comparator` oluşturan helper static method'lar

Method	Description / Açıklama
<code>comparing(function)</code>	Compare by results of function that returns any <code>Object</code> (or primitive autoboxed into <code>Object</code>) / Herhangi bir <code>Object</code> (veya <code>Object</code> te autobox edilen primitive) döndüren function sonucuna göre karşılaştırır
<code>comparingDouble(function)</code>	Compare by results of function that returns <code>double</code> / <code>double</code> döndüren function sonucuna göre karşılaştırır
<code>comparingInt(function)</code>	Compare by results of function that returns <code>int</code> / <code>int</code> döndüren function sonucuna göre karşılaştırır
<code>comparingLong(function)</code>	Compare by results of function that returns <code>long</code> / <code>long</code> döndüren function sonucuna göre karşılaştırır
<code>naturalOrder()</code>	Sort using order specified by the <code>Comparable</code> implementation on object itself / <code>Object</code> 'in kendi <code>Comparable</code> implementation'ının belirlediği order ile sıralar
<code>reverseOrder()</code>	Sort using reverse of order specified by <code>Comparable</code> implementation on object itself / <code>Object</code> 'in kendi <code>Comparable</code> implementation'ının belirlediği order'ın tersiyle sıralar

Türkçe tablo özeti: `comparing()` object result'ına; `comparingDouble()` / `comparingInt()` / `comparingLong()` ilgili primitive result'a göre karşılaştırır. `naturalOrder()` object'in `Comparable` ordering'ini, `reverseOrder()` bunun tersini kullanır.

English: Table 9.12 shows the methods that you can chain to a `Comparator` to further specify its behavior. **Türkçe:** Table 9.12, davranışı daha ayrıntılı belirlemek için bir `Comparator`a chain edebileceğiniz method'ları gösterir.

Table 9.12 · Helper default methods for building a Comparator / Comparator oluşturan helper default method'lar

Method	Description / Açıklama
<code>reversed()</code>	Reverse order of chained <code>Comparator</code> / Chained <code>Comparator</code> order'ını tersine çevirir
<code>thenComparing(function)</code>	If previous <code>Comparator</code> returns 0, use this comparator that returns <code>Object</code> or can be autoboxed into one / Önceki <code>Comparator</code> 0 döndürürse <code>Object</code> döndüren veya <code>Object</code> te autobox edilebilen bu comparator'ı kullanır
<code>thenComparingDouble(function)</code>	If previous <code>Comparator</code> returns 0, use this comparator that returns <code>double</code> ; otherwise return previous result / Önceki <code>Comparator</code> 0 döndürürse <code>double</code> döndüren bu comparator'ı, aksi hâlde önceki sonucu kullanır
<code>thenComparingInt(function)</code>	If previous <code>Comparator</code> returns 0, use this comparator that returns <code>int</code> ; otherwise return previous result / Önceki <code>Comparator</code> 0 döndürürse <code>int</code> döndüren bu comparator'ı, aksi hâlde önceki sonucu kullanır
<code>thenComparingLong(function)</code>	If previous <code>Comparator</code> returns 0, use this comparator that returns <code>long</code> ; otherwise return previous result / Önceki <code>Comparator</code> 0 döndürürse <code>long</code> döndüren bu comparator'ı, aksi hâlde önceki sonucu kullanır

Türkçe tablo özeti: `reversed()` chained `Comparator` order'ını tersine çevirir. `thenComparing*()` method'ları önceki comparator 0 döndürürse sonraki function'ı kullanır; aksi hâlde önceki comparator'ın result'ı korunur.

English — real-world note: You've probably noticed by now that we often ignore `null` values in checking equality and comparing objects. This works fine for the exam. In the real world, though, things aren't so neat. You will have to decide how to handle `null` values or prevent them from being in your object. **Türkçe — gerçek dünya notu:** Equality kontrolünde ve object'leri karşılaştırırken `null` value'ları sıklıkla yok saydığımızı muhtemelen fark ettiniz. Bu sınav için yeterlidir; ancak gerçek dünyada durum bu kadar düzenli değildir. `null` value'ların nasıl ele alınacağına karar vermeli veya object'inizde bulunmalarını önlemelisiniz.

Sorting and Searching

English: Now that you've learned all about `Comparable` and `Comparator`, we can finally do something useful with them, like sorting. The `Collections.sort()` method uses the `compareTo()` method to sort. It expects the objects to be sorted to be `Comparable`. **Türkçe:** `Comparable` ve `Comparator` hakkında her şeyi öğrendiğimize göre artık bunlarla sorting gibi yararlı bir şey yapabiliriz. `Collections.sort()` sorting için `compareTo()` method'unu kullanır. Sıralanacak object'lerin `Comparable` olmasını bekler.

```

2: public class SortRabbits {
3:     static record Rabbit(int id) {}
4:     public static void main(String[] args) {
5:         List<Rabbit> rabbits = new ArrayList<>();
6:         rabbits.add(new Rabbit(3));
7:         rabbits.add(new Rabbit(1));
8:         Collections.sort(rabbits); // DOES NOT COMPILE
9:     } }

```

English: Java knows that the `Rabbit` record is not `Comparable`. It knows sorting will fail, so it doesn't even let the code compile. You can fix this by passing a `Comparator` to `sort()`. Remember that a `Comparator` is useful when you want to specify sort order without using a `compareTo()` method.

Türkçe: Java, `Rabbit` record'unun `Comparable` olmadığını bilir. Sorting'in başarısız olacağını bildiği için code'un derlenmesine bile izin vermez. `sort()` a `Comparator` geçirerek bunu düzeltebilirsiniz. `Comparator`ın, `compareTo()` kullanmadan sort order belirtmek istediğinizde yararlı olduğunu unutmayın.

```

8: Comparator<Rabbit> c = (r1, r2) -> r1.id - r2.id;
9: Collections.sort(rabbits, c);
10: System.out.println(rabbits); // [Rabbit[id=1], Rabbit[id=3]]

```

English: Suppose you want to sort the rabbits in descending order. You could change the `Comparator` to `r2.id - r1.id`. Alternatively, you could reverse the contents of the list afterward: **Türkçe:** Tavşanları descending order'da sıralamak istediğinizi varsayalım. `Comparator`ı `r2.id - r1.id` olarak değiştirebilirsiniz. Alternatif olarak daha sonra list contents'ini tersine çevirebilirsiniz:

```

8: Comparator<Rabbit> c = (r1, r2) -> r1.id - r2.id;
9: Collections.sort(rabbits, c);
10: Collections.reverse(rabbits);
11: System.out.println(rabbits); // [Rabbit[id=3], Rabbit[id=1]]

```

English: The `sort()` and `binarySearch()` methods allow you to pass in a `Comparator` object when you don't want to use the natural order. **Türkçe:** Natural order'ı kullanmak istemediğinizde `sort()` ve `binarySearch()` method'ları bir `Comparator` object geçirmenize izin verir.

Reviewing `binarySearch()`

English: The `binarySearch()` method requires a sorted `List`. **Türkçe:** `binarySearch()` method'u sorted bir `List` gerektirir.

```

11: List<Integer> list = Arrays.asList(6, 9, 1, 8);
12: Collections.sort(list); // [1, 6, 8, 9]
13: System.out.println(Collections.binarySearch(list, 6)); // 1
14: System.out.println(Collections.binarySearch(list, 3)); // -2

```

English: Line 12 sorts the `List` so we can call binary search properly. Line 13 prints the index at which a match is found. Line 14 prints one less than the negated index of where the requested value would need to be inserted. The number 3 would need to be inserted at index 1 (after the number 1 but before the number 6). Negating that gives us -1, and subtracting 1 gives us -2. **Türkçe:** Line 12, binary search'ü doğru çağırabilmemiz için `List`i sıralar. Line 13 eşleşmenin bulunduğu index'i yazdırır. Line 14, istenen value'nun eklenmesi gereken index'in negatifinin bir eksiğini yazdırır. 3 sayısı index 1'e, yani 1'den sonra fakat 6'dan önce eklenmelidir. Bunun negatifi -1, bundan 1 çıkarılınca sonuç -2 olur.

English: There is a trick in working with `binarySearch()`. What do you think the following outputs?

Türkçe: `binarySearch()` ile çalışırken bir tuzak vardır. Aşağıdaki kod sizce ne yazdırır?

```
3: var names = Arrays.asList("Fluffy", "Hoppy");
4: Comparator<String> c = Comparator.reverseOrder();
5: var index = Collections.binarySearch(names, "Hoppy", c);
6: System.out.println(index);
```

English: The answer happens to be -1. Before you panic, you don't need to know that the answer is -1. You do need to know that the answer is not defined. Line 3 creates a list, **Türkçe:** Sonuç tesadüfen -1'dir. Telaşa kapılmadan önce şunu bilin: Cevabın -1 olduğunu bilmeniz gerekmez; cevabın defined olmadığını bilmeniz gerekir. Line 3 bir list oluşturur:

English: `[Fluffy, Hoppy]`. This list happens to be sorted in ascending order. Line 4 creates a `Comparator` that reverses the natural order. Line 5 requests a binary search in descending order. Since the list is not in that order, we don't meet the precondition for doing a search. **Türkçe:** `[Fluffy, Hoppy]`. Bu list ascending order'da sıralanmıştır. Line 4 natural order'ı tersine çeviren bir `Comparator` oluşturur. Line 5 descending order'da binary search ister. List bu order'da olmadığından search'ün precondition'ını karşılamayız.

English: While the result of calling `binarySearch()` on an improperly sorted list is undefined, sometimes you can get lucky. For example, search starts in the middle of an odd-numbered list. If you happen to ask for the middle element, the index returned will be what you expect. **Türkçe:** Yanlış sıralanmış list üzerinde `binarySearch()` çağrısının sonucu undefined olsa da bazen şanslı olabilirsiniz. Örneğin search, odd-numbered list'in ortasından başlar. Tesadüfen middle element'ı isterseniz dönen index beklediğiniz değer olur.

English: Earlier in the chapter, we talked about collections that require classes to implement `Comparable`. Unlike sorting, they don't check that you have implemented `Comparable` at compile time. **Türkçe:** Bölümün önceki kısmında class'ların `Comparable` implement etmesini gerektiren collection'ları tartıştık. Sorting'den farklı olarak bunlar `Comparable` implement edip etmediğinizi compile time'da kontrol etmez.

English: Going back to our `Rabbit` that does not implement `Comparable`, we try to add it to a `TreeSet`:

Türkçe: `Comparable` implement etmeyen `Rabbit`ımıza dönüp onu `TreeSet`e eklemeyi deneyelim:

```
2: public class UseTreeSet {
3:     static class Rabbit { int id; }
4:     public static void main(String[] args) {
5:         Set<Duck> ducks = new TreeSet<>();
6:         ducks.add(new Duck("Puddles"));
7:
8:         Set<Rabbit> rabbits = new TreeSet<>();
9:         rabbits.add(new Rabbit()); // ClassCastException
10:    } }
```

English: Line 6 is fine. `Duck` does implement `Comparable`. `TreeSet` is able to sort it into the proper position in the set. Line 9 is a problem. When `TreeSet` tries to sort it, Java discovers the fact that `Rabbit` does not implement `Comparable`. Java throws an exception that looks like this: **Türkçe:** Line 6 geçerlidir; `Duck`, `Comparable` implement eder ve `TreeSet` onu set'teki doğru konuma sıralayabilir. Line 9 sorundur. `TreeSet` onu sıralamaya çalıştığında Java, `Rabbit`'in `Comparable` implement etmediğini keşfeder ve şöyle bir exception fırlatır:

```
Exception in thread "main" java.lang.ClassCastException:
class Rabbit cannot be cast to class java.lang.Comparable
```

English: It may seem weird for this exception to be thrown when the first object is added to the set. After all, there is nothing to compare yet. Java works this way for consistency. **Türkçe:** İlk object set'e eklenirken bu exception'ın fırlatılması tuhaf görünebilir; sonuçta henüz karşılaştırılacak başka bir şey yoktur. Java consistency için bu şekilde çalışır.

English: Just like searching and sorting, you can tell collections that require sorting that you want to use a specific `Comparator`. For example: **Türkçe:** Searching ve sorting'de olduğu gibi sorting gerektiren collection'lara belirli bir `Comparator` kullanmak istediğinizi söyleyebilirsiniz. Örneğin:

```
8: Set<Rabbit> rabbits =
    new TreeSet<>((r1, r2) -> r1.id - r2.id);
9: rabbits.add(new Rabbit());
```

English: Now Java knows that you want to sort by `id`, and all is well. A `Comparator` is a helpful object. It lets you separate sort order from the object to be sorted. Notice that line 9 in both of the previous examples is the same. It's the declaration of the `TreeSet` that has changed. **Türkçe:** Artık Java `id`'ye göre sıralamak istediğinizi bilir ve sorun kalmaz. `Comparator` yararlı bir object'tir; sort order'ı sıralanacak object'ten ayırmanızı sağlar. Önceki iki örnekte line 9'un aynı olduğuna dikkat edin. Değişen, `TreeSet` declaration'ıdır.

Sorting a List

English: While you can call `Collections.sort(list)`, you can also sort directly on the list object.

Türkçe: `Collections.sort(list)` çağırabildiğiniz gibi doğrudan list object üzerinde de sort yapabilirsiniz.

```

3: List<String> bunnies = new ArrayList<>();
4: bunnies.add("long ear");
5: bunnies.add("floppy");
6: bunnies.add("hoppy");
7: System.out.println(bunnies); // [long ear, floppy, hoppy]
8: bunnies.sort((b1, b2) -> b1.compareTo(b2));
9: System.out.println(bunnies); // [floppy, hoppy, long ear]

```

English: On line 8, we sort the list alphabetically. The `sort()` method takes a `Comparator` that provides the sort order. Remember that `Comparator` takes two parameters and returns an `int`. If you need a review of what the return value of a `compare()` operation means, check the `Comparator` section in this chapter or the “Comparing” section in Chapter 4, “Core APIs.” This is really important to memorize!

Türkçe: Line 8'de list'i alfabetik olarak sıralarız. `sort()` method'u sort order sağlayan bir `Comparator` alır. `Comparator`'ın iki parameter alıp `int` döndürdüğünü unutmayın. `compare()` operation'ının return value'sunun anlamını tekrar etmek isterseniz bu bölümdeki `Comparator` kısmına veya Chapter 4, “Core APIs” içindeki “Comparing” kısmına bakın. Bunu ezberlemek gerçekten önemlidir.

English: There is not a sort method on `Set` or `Map`. Both of those types are unordered, so it wouldn't make sense to sort them. **Türkçe:** `Set` veya `Map` üzerinde sort method'u yoktur. Bu type'ların ikisi de unordered olduğundan onları sort etmek anlamlı olmazdı.

Working with Generics

English: We conclude this chapter with one of the most useful, and at times most confusing, features in the Java language: generics. In fact, we've been using them extensively in the last two chapters—the type between the `<>`. Why do we need generics? Imagine if we weren't specifying the type of our lists and merely hoped the caller didn't put in something that we didn't expect. The following does just that: **Türkçe:** Bu bölümü Java dilinin en yararlı, zaman zaman da en kafa karıştırıcı özelliklerinden biriyle tamamlıyoruz: generics. Aslında son iki bölümde onları, yani `<>` arasındaki type'ı yoğun biçimde kullandık. Generics'e neden gereksinim duyarız? List'lerimizin type'ını belirtmediğimizi ve caller'ın beklemediğimiz bir şeyi koymasını umduğumuzu düşünün. Aşağıdaki code tam olarak bunu yapar:

```

14: static void printNames(List list) {
15:     for (int i = 0; i < list.size(); i++) {
16:         String name = (String)list.get(i); // ClassCastException
17:         System.out.println(name);
18:     }
19: }
20: public static void main(String[] args) {
21:     List names = new ArrayList();
22:     names.add(new StringBuilder("Webby"));
23:     printNames(names);
24: }

```

English: This code throws a `ClassCastException`. Line 22 adds a `StringBuilder` to `list`. This is legal because a non-generic list can contain anything. However, line 16 is written to expect a specific class to be in there. It casts to a `String`, reflecting this assumption. **Türkçe:** Bu code `ClassCastException` fırlatır. Line 22 `liste` `StringBuilder` ekler. Non-generic list her şeyi içerebildiğinden bu yasalır. Ancak line 16 orada belirli bir class bulunmasını bekleyecek şekilde yazılmıştır ve bu assumption'ı yansıtarak `String` cast eder.

English: Since the assumption is incorrect, the code throws a `ClassCastException` that `java.lang.StringBuilder` cannot be cast to `java.lang.String`. **Türkçe:** Assumption yanlış olduğundan code, `java.lang.StringBuilder`ın `java.lang.String`e cast edilemeyeceğini bildiren bir `ClassCastException` fırlatır.

English: Generics fix this by allowing you to write and use parameterized types. Since we specify that we want an `ArrayList` of `String` objects, the compiler has enough information to prevent this problem in the first place. **Türkçe:** Generics, parameterized type yazıp kullanmanıza olanak tanıyarak bunu düzeltir. `String` object'lerinden oluşan bir `ArrayList` istediğimizi belirttiğimiz için compiler daha baştan bu sorunu önleyecek yeterli bilgiye sahiptir.

```
List<String> names = new ArrayList<String>();
names.add(new StringBuilder("Webby")); // DOES NOT COMPILE
```

English: Getting a compiler error is good. You'll know right away that something is wrong rather than hoping to discover it later. **Türkçe:** Compiler error almak iyidir. Sorunu daha sonra keşfetmeyi ummak yerine bir şeylerin yanlış olduğunu hemen bilirsiniz.

Creating Generic Classes

English: You can introduce generics into your own classes. The syntax for introducing a generic is to declare a formal type parameter in angle brackets. For example, the following class named `Crate` has a generic type variable declared after the name of the class: **Türkçe:** Kendi class'larınıza generics ekleyebilirsiniz. Generic ekleme syntax'ı angle bracket içinde formal type parameter bildirmektir. Örneğin aşağıdaki `Crate` adlı class'ın class adından sonra bildirilen generic type variable'ı vardır:

```
public class Crate<T> {
    private T contents;
    public T lookInCrate() {
        return contents;
    }
    public void packCrate(T contents) {
        this.contents = contents;
    }
}
```

English: The generic type `T` is available anywhere within the `Crate` class. When you instantiate the class, you tell the compiler what `T` should be for that particular instance. **Türkçe:** Generic type `T`, `Crate` class'ının her yerinde kullanılabilir. Class'ı instantiate ederken o particular instance için `T`nin ne olması gerektiğini compiler'a söylersiniz.

Naming Conventions for Generics

English: A type parameter can be named anything you want. The convention is to use single uppercase letters to make it obvious that they aren't real class names. The following are common letters to use: **Türkçe:** Type parameter istediğiniz şekilde adlandırılabilir. Gerçek class adı olmadıklarını açık kılmak için tek uppercase letter kullanmak convention'dır. Aşağıdakiler yaygın harflerdir:

English: `E` for an element. **Türkçe:** Element için `E`.

English: `K` for a map key. **Türkçe:** Map key için `K`.

English: `V` for a map value. **Türkçe:** Map value için `V`.

English: `N` for a number. **Türkçe:** Number için `N`.

English: `T` for a generic data type. **Türkçe:** Generic data type için `T`.

English: `S`, `U`, `V`, and so forth for multiple generic types. **Türkçe:** Birden fazla generic type için `S`, `U`, `V` ve devamı.

English: For example, suppose an `Elephant` class exists, and we are moving our elephant to a new and larger enclosure in our zoo. (The San Diego Zoo did this in 2009. It was interesting seeing the large metal crate.) **Türkçe:** Örneğin bir `Elephant` class'ı bulunduğunu ve filimizi hayvanat bahçesindeki yeni, daha büyük bir enclosure'a taşıdığımızı varsayalım. (San Diego Zoo bunu 2009'da yaptı. Büyük metal crate'i görmek ilginçti.)

```
Elephant elephant = new Elephant();
Crate<Elephant> crateForElephant = new Crate<>();
crateForElephant.packCrate(elephant);
Elephant inNewHome = crateForElephant.lookInCrate();
```

English: To be fair, we didn't pack the crate so much as the elephant walked into it. However, you can see that the `Crate` class is able to deal with an `Elephant` without knowing anything about it. **Türkçe:** Doğrusu, crate'i bizim paketlememizden çok fil kendi içine girdi. Bununla birlikte `Crate` class'ının `Elephant` hakkında hiçbir şey bilmeden onunla çalışabildiğini görebilirsiniz.

English: This probably doesn't seem particularly impressive. We could have just typed in `Elephant` instead of `T` when coding `Crate`. What if we wanted to create a `Crate` for another animal? **Türkçe:** Bu muhtemelen pek etkileyici görünmez. `Crate`i yazarken `T` yerine doğrudan `Elephant` yazabilirdik. Başka bir hayvan için `Crate` oluşturmak isteseydik ne olurdu?

```
Crate<Zebra> crateForZebra = new Crate<>();
```

English: Now we couldn't have simply hard-coded `Elephant` in the `Crate` class since a `Zebra` is not an `Elephant`. However, we could have created an `Animal` superclass or interface and used that in `Crate`. **Türkçe:** `Zebra`, `Elephant` olmadığından artık `Crate` class'ına yalnız `Elephant`ı hard-code edemedik. Yine de bir `Animal` superclass veya interface oluşturup `Crate`'te onu kullanabilirdik.

English: Generic classes become useful when the classes used as the type parameter can have absolutely nothing to do with each other. For example, we need to ship our 120-pound robot to another city: **Türkçe:** Type parameter olarak kullanılan class'ların birbiriyle hiçbir ilişkisi bulunmadığında generic class'lar yararlı hâle gelir. Örneğin 120 pound ağırlığındaki robotumuzu başka bir şehre göndermemiz gerekiyor:

```
Robot joeBot = new Robot();
Crate<Robot> robotCrate = new Crate<>();
robotCrate.packCrate(joeBot);
// ship to Houston
Robot atDestination = robotCrate.lookInCrate();
```

English: Now it is starting to get interesting. The `Crate` class works with any type of class. Before generics, we would have needed `Crate` to use the `Object` class for its instance variable, which would have put the burden on the caller to cast the object it receives on emptying the crate. **Türkçe:** Şimdi konu ilginçleşmeye başlar. `Crate` class'ı her class type'ı ile çalışır. Generics'ten önce `Crate`'in instance variable'ı için `Object` kullanması gerekirdi; bu da crate boşaltılırken alınan object'i cast etme yükünü caller'a bırakırdı.

English: In addition to `Crate` not needing to know about the objects that go into it, those objects don't need to know about `Crate`. We aren't requiring the objects to implement an interface named `Crateable` or the like. A class can be put in the `Crate` without any changes at all. **Türkçe:** `Crate`'in içine giren object'leri bilmesi gerekmediği gibi bu object'lerin de `Crate`'i bilmesi gerekmez. Object'lerin `Crateable` benzeri bir interface implement etmesini istemiyoruz. Bir class hiçbir değişiklik yapılmadan `Crate` içine konabilir.

English — practical note: Don't worry if you can't think of a use for generic classes of your own. Unless you are writing a library for others to reuse, generics hardly show up in the class definitions you write. You've already seen them frequently in the code you call, such as functional interfaces and collections. **Türkçe — pratik not:** Kendi generic class'larınız için kullanım alanı düşünemiyorsanız endişelenmeyin. Başkalarının reuse edeceği library yazmıyorsanız generics, yazdığınız class definition'larında pek görünmez. Functional interface ve collection gibi çağırdığınız code içinde onları zaten sıkça gördünüz.

English: Generic classes aren't limited to having a single type parameter. This class shows two generic parameters: **Türkçe:** Generic class'lar tek bir type parameter ile sınırlı değildir. Aşağıdaki class iki generic parameter gösterir:

```
public class SizeLimitedCrate<T, U> {
    private T contents;
    private U sizeLimit;
    public SizeLimitedCrate(T contents, U sizeLimit) {
        this.contents = contents;
        this.sizeLimit = sizeLimit;
    }
}
```

English: `T` represents the type that we are putting in the crate. `U` represents the unit that we are using to measure the maximum size for the crate. To use this generic class, we can write the following: **Türkçe:** `T` crate içine koyduğumuz type'i temsil eder. `U`, crate'in maximum size'ını ölçmekte kullandığımız unit'i temsil eder. Bu generic class'ı kullanmak için şunu yazabiliriz:

```
Elephant elephant = new Elephant();
Integer numPounds = 15_000;
SizeLimitedCrate<Elephant, Integer> c1 =
    new SizeLimitedCrate<>(elephant, numPounds);
```

English: Here we specify that the type is `Elephant`, and the unit is `Integer`. We also throw in a reminder that numeric literals can contain underscores. **Türkçe:** Burada type'ın `Elephant`, unit'in `Integer` olduğunu belirtiriz. Ayrıca numeric literal'ların underscore içerebildiğini hatırlatırız.

Understanding Type Erasure

English: Specifying a generic type allows the compiler to enforce proper use of the generic type. For example, specifying the generic type of `Crate` as `Robot` is like replacing the `T` in the `Crate` class with `Robot`. However, this is just for compile time. **Türkçe:** Generic type belirtmek, compiler'ın bu type'ın doğru kullanımını enforce etmesini sağlar. Örneğin `Crate`in generic type'ını `Robot` olarak belirtmek, `Crate` class'ındaki `T`yi `Robot` ile değiştirmek gibidir. Ancak bu yalnız compile time içindir.

English: Behind the scenes, the compiler replaces all references to `T` in `Crate` with `Object`. In other words, after the code compiles, your generics are just `Object` types. The `Crate` class looks like the following at runtime: **Türkçe:** Arka planda compiler, `Crate` içindeki bütün `T` reference'larını `Object` ile değiştirir. Başka bir deyişle code derlendikten sonra generic'leriniz yalnız `Object` type'larıdır. Runtime'da `Crate` class'ı şöyle görünür:

```
public class Crate {
    private Object contents;
    public Object lookInCrate() {
        return contents;
    }
    public void packCrate(Object contents) {
        this.contents = contents;
    }
}
```

English: This means there is only one class file. There aren't different copies for different parameterized types. (Some other languages work that way.) This process of removing the generics syntax from your code is referred to as type erasure. Type erasure allows your code to be compatible with older versions of Java that do not contain generics. **Türkçe:** Bu, yalnız bir class file bulunduğu anlamına gelir. Farklı parameterized type'lar için farklı kopyalar yoktur. (Bazı diğer diller böyle çalışır.) Generics syntax'ını code'dan kaldırma sürecine type erasure denir. Type erasure, code'unuzun generics içermeyen eski Java sürümleriyle compatible olmasını sağlar.

English: The compiler adds the relevant casts for your code to work with this type of erased class. For example, you type the following: **Türkçe:** Compiler, code'unuzun bu erased class type'ıyla çalışması için gerekli cast'leri ekler. Örneğin siz şunu yazarsınız:

```
Robot r = crate.lookInCrate();
```

English: The compiler turns it into the following: **Türkçe:** Compiler bunu aşağıdakine dönüştürür:

```
Robot r = (Robot) crate.lookInCrate();
```

English: In the following sections, we look at the implications of generics for method declarations.

Türkçe: Sonraki kısımlarda generics'in method declaration'ları üzerindeki etkilerine bakıyoruz.

Overloading a Generic Method

English: Only one of these two methods is allowed in a class because type erasure will reduce both sets of arguments to `(List input)`: **Türkçe:** Type erasure iki argument kümesini de `(List input)` biçimine indireceğinden bu iki method'dan yalnız birine class içinde izin verilir:

```
public class LongTailAnimal {
    protected void chew(List<Object> input) {}
    protected void chew(List<Double> input) {} // DOES NOT COMPILE
}
```

English: For the same reason, you also can't overload a generic method from a parent class. **Türkçe:** Aynı nedenle parent class'tan gelen generic method'u da overload edemezsiniz.

```
public class LongTailAnimal {
    protected void chew(List<Object> input) {}
}
public class Anteater extends LongTailAnimal {
    protected void chew(List<Double> input) {} // DOES NOT COMPILE
}
```

English: Both of these examples fail to compile because of type erasure. In the compiled form, the generic type is dropped, and it appears as an invalid overloaded method. Now, let's look at a subclass:

Türkçe: Bu iki örnek type erasure nedeniyle derlenmez. Compiled form'da generic type düşer ve geçersiz overloaded method gibi görünür. Şimdi bir subclass'a bakalım:

```
public class Anteater extends LongTailAnimal {
    protected void chew(List<Object> input) {}
    protected void chew(ArrayList<Double> input) {}
}
```

English: The first `chew()` method compiles because it uses the same generic type in the overridden method as the one defined in the parent class. The second `chew()` method compiles as well. **Türkçe:** İlk `chew()` method'u overridden method'da parent class'ta tanımlananla aynı generic type'ı kullandığı için derlenir. İkinci `chew()` method'u da derlenir.

English: However, it is an overloaded method because one of the method arguments is a `List` and the other is an `ArrayList`. When working with generic methods, it's important to consider the underlying type. **Türkçe:** Ancak method argument'larından biri `List`, diğeri `ArrayList` olduğundan bu overloaded method'dur. Generic method'larla çalışırken underlying type'ı dikkate almak önemlidir.

Returning Generic Types

English: When you're working with overridden methods that return generics, the return values must be covariant. In terms of generics, this means that the return type of the class or interface declared in the overriding method must be a subtype of the class defined in the parent class. The generic parameter type must match its parent's type exactly. **Türkçe:** Generic döndüren overridden method'larla çalışırken return value'lar covariant olmalıdır. Generics açısından bu, overriding method'da bildirilen class veya interface'in return type'ının parent class'ta tanımlanan class'ın subtype'ı olması demektir. Generic parameter type, parent'in type'ıyla exact eşleşmelidir.

English: Given the following declaration for the `Mammal` class, which of the two subclasses, `Monkey` and `Goat`, compile? **Türkçe:** Aşağıdaki `Mammal` declaration'ı verildiğinde iki subclass'tan hangileri, `Monkey` ve `Goat`, derlenir?

```
public class Mammal {
    public List<CharSequence> play() { ... }
    public CharSequence sleep() { ... }
}
public class Monkey extends Mammal {
    public ArrayList<CharSequence> play() { ... }
}
public class Goat extends Mammal {
    public List<String> play() { ... } // DOES NOT COMPILE
    public String sleep() { ... }
}
```

English: The `Monkey` class compiles because `ArrayList` is a subtype of `List`. The `play()` method in the `Goat` class does not compile, though. For the return types to be covariant, the generic type parameter must match. Even though `String` is a subtype of `CharSequence`, it does not exactly match the generic type defined in the `Mammal` class. Therefore, this is considered an invalid override. **Türkçe:** `ArrayList`, `List`'in subtype'ı olduğu için `Monkey` class'ı derlenir. Ancak `Goat` class'ındaki `play()` method'u derlenmez. Return type'ların covariant olması için generic type parameter eşleşmelidir. `String`, `CharSequence`'in subtype'ı olsa da `Mammal` class'ında tanımlanan generic type ile exact eşleşmez. Bu nedenle invalid override kabul edilir.

English: Notice that the `sleep()` method in the `Goat` class does compile since `String` is a subtype of `CharSequence`. This example shows that covariance applies to the return type, just not the generic parameter type. **Türkçe:** `String`, `CharSequence`ın subtype'ı olduğundan `Goat` class'ındaki `sleep()` method'unun derlendiğine dikkat edin. Bu örnek covariance'ın return type'a uygulandığını, yalnız generic parameter type'a uygulanmadığını gösterir.

English: For the exam, it might be helpful for you to apply type erasure to questions involving generics to ensure that they compile properly. Once you've determined which methods are overridden and which are being overloaded, work backward, making sure the generic types match for overridden methods. And remember, generic methods cannot be overloaded by changing the generic parameter type only. **Türkçe:** Sınavda generics içeren sorulara type erasure uygulayarak doğru derlenip derlenmediklerini kontrol etmek yararlı olabilir. Hangi method'ların overridden, hangilerinin overloaded olduğunu belirledikten sonra geriye doğru çalışıp overridden method'larda generic type'ların eşleştiğinden emin olun. Generic method'ların yalnız generic parameter type değiştirilerek overload edilemeyeceğini unutmayın.

Implementing Generic Interfaces

English: Just like a class, an interface can declare a formal type parameter. For example, the following `Shippable` interface uses a generic type as the argument to its `ship()` method: **Türkçe:** Class gibi interface de formal type parameter bildirebilir. Örneğin aşağıdaki `Shippable` interface'i `ship()` method'unun argument'i olarak generic type kullanır:

```
public interface Shippable<T> {  
    void ship(T t);  
}
```

English: There are three ways a class can approach implementing this interface. The first is to specify the generic type in the class. The following concrete class says that it deals only with robots. This lets it declare the `ship()` method with a `Robot` parameter: **Türkçe:** Bir class bu interface'i üç yolla implement edebilir. İlki generic type'ı class'ta belirtmektir. Aşağıdaki concrete class yalnız robotlarla çalıştığını söyler. Böylece `ship()` method'unu `Robot` parameter ile bildirebilir:

```
class ShippableRobotCrate implements Shippable<Robot> {  
    public void ship(Robot t) { }  
}
```

English: The next way is to create a generic class. The following concrete class allows the caller to specify the type of the generic: **Türkçe:** Sonraki yol generic class oluşturmaktır. Aşağıdaki concrete class, generic type'ını caller'ın belirtmesine izin verir:

```
class ShippableAbstractCrate<U> implements Shippable<U> {  
    public void ship(U t) { }  
}
```

English: In this example, the type parameter could have been named anything, including `T`. We used `U` in the example to avoid confusion about what `T` refers to. The exam won't mind trying to confuse you by using the same type parameter name. **Türkçe:** Bu örnekte type parameter `T` dahil herhangi bir şekilde adlandırılabilirdi. `T`'nin neye gönderme yaptığı konusunda karışıklığı önlemek için `U` kullandık. Sınav aynı type parameter adını kullanarak kafanızı karıştırmayı deneyebilir.

English: The final way is to not use generics at all. This is the old way of writing code. It generates a compiler warning about `Shippable` being a raw type, but it does compile. Here the `ship()` method has an `Object` parameter since the generic type is not defined. **Türkçe:** Son yol hiç generics kullanmamaktır. Bu eski code yazma yoludur. `Shippable`'in raw type olması hakkında compiler warning üretir, fakat derlenir. Generic type tanımlanmadığından burada `ship()` method'u `Object` parameter'a sahiptir:

```
class ShippableCrate implements Shippable {  
    public void ship(Object t) { }  
}
```

What You Can't Do with Generic Types

English: There are some limitations on what you can do with a generic type. These aren't on the exam, but it will be helpful to refer to this scenario when you are writing practice programs and run into one of these situations. **Türkçe:** Generic type ile yapabileceklerinize ilişkin bazı limitations vardır. Bunlar sınavda değildir; ancak practice program yazarken bu durumlardan biriyle karşılaştığınızda bu scenario'ya bakmak yararlı olur.

English: Most of the limitations are due to type erasure. Oracle refers to types whose information is fully available at runtime as reifiable. Reifiable types can do anything that Java allows. Non-reifiable types have some limitations. **Türkçe:** Limitations'ın çoğu type erasure'dan kaynaklanır. Oracle, bilgileri runtime'da tamamen kullanılabilir olan type'lara reifiable der. Reifiable type'lar Java'nın izin verdiği her şeyi yapabilir. Non-reifiable type'ların bazı limitations'ı vardır.

English: Here are the things that you can't do with generics (and by "can't," we mean without resorting to contortions like passing in a class object): **Türkçe:** Generics ile yapamayacağınız şeyler şunlardır ("yapamazsınız" derken class object geçirmek gibi dolambaçlı yollara başvurmadan demek istiyoruz):

English: Call a constructor: Writing `new T()` is not allowed because at runtime, it would be `new Object()`. **Türkçe:** Constructor çağırmak: Runtime'da `new Object()` olacağından `new T()` yazmaya izin verilmez.

English: Create an array of that generic type: This one is the most annoying, but it makes sense because you'd be creating an array of `Object` values. **Türkçe:** O generic type'ta array oluşturmak: En can sıkıcı limitation budur; ancak `Object` value'lardan oluşan array oluşturacak olmanız nedeniyle mantıklıdır.

English: Call `instanceof`: This is not allowed because at runtime `List<Integer>` and `List<String>` look the same to Java, thanks to type erasure. **Türkçe:** `instanceof` çağırmak: Type erasure nedeniyle runtime'da `List<Integer>` ile `List<String>` Java'ya aynı görüldüğünden buna izin verilmez.

English: Use a primitive type as a generic type parameter: This isn't a big deal because you can use the wrapper class instead. If you want a type of `int`, just use `Integer`. **Türkçe:** Generic type parameter olarak primitive type kullanmak: Bunun yerine wrapper class kullanabildiğiniz için büyük bir sorun değildir. `int` type istiyorsanız `Integer` kullanın.

English: Create a static variable as a generic type parameter: This is not allowed because the type is linked to the instance of the class. **Türkçe:** Generic type parameter type'ında static variable oluşturmak: Type, class instance'ına bağlı olduğundan buna izin verilmez.

Writing Generic Methods

English: Up until this point, you've seen formal type parameters declared on the class or interface level. It is also possible to declare them on the method level. This is often useful for static methods since they aren't part of an instance that can declare the type. However, it is also allowed on non-static methods. **Türkçe:** Buraya kadar class veya interface level'da bildirilen formal type parameter'ları gördünüz. Bunları method level'da bildirmek de mümkündür. Type bildirebilen bir instance'ın parçası olmadıkları için bu, static method'larda sıklıkla yararlıdır. Bununla birlikte non-static method'larda da izin verilir.

English: In this example, both methods use a generic parameter: **Türkçe:** Bu örnekte iki method da generic parameter kullanır:

```
public class Handler {
    public static <T> void prepare(T t) {
        System.out.println("Preparing " + t);
    }
    public static <T> Crate<T> ship(T t) {
        System.out.println("Shipping " + t);
        return new Crate<T>();
    }
}
```

English: The method parameter is the generic type `T`. Before the return type, we declare the formal type parameter of `<T>`. In the `ship()` method, we show how you can use the generic parameter in the return type, `Crate<T>`, for the method. **Türkçe:** Method parameter generic type `T`dir. Return type'tan önce formal type parameter `<T>`yi bildiririz. `ship()` method'unda generic parameter'ın method return type'ı olan `Crate<T>` içinde nasıl kullanılacağını gösteririz.

English: Unless a method is obtaining the generic formal type parameter from the class/interface, it is specified immediately before the return type of the method. This can lead to some interesting-looking code! **Türkçe:** Method generic formal type parameter'ı class/interface'ten almıyorsa bu parameter method'un return type'ından hemen önce belirtilir. Bu, ilginç görünen code'a yol açabilir!

```
2: public class More {
3:     public static <T> void sink(T t) { }
```

```

4:     public static <T> T identity(T t) { return t; }
5:     public static T noGood(T t) { return t; } // DOES NOT COMPILE
6: }

```

English: Line 3 shows the formal parameter type immediately before the return type of `void`. Line 4 shows the return type being the formal parameter type. It looks weird, but it is correct. Line 5 omits the formal parameter type and therefore does not compile. **Türkçe:** Line 3 formal parameter type'ı `void` return type'ından hemen önce gösterir. Line 4 return type'in formal parameter type olduğunu gösterir. Tuhaf görünür, fakat doğrudur. Line 5 formal parameter type'ı atladığı için derlenmez.

Optional Syntax for Invoking a Generic Method

English: You can call a generic method normally, and the compiler will try to figure out which one you want. Alternatively, you can specify the type explicitly to make it obvious what the type is. **Türkçe:** Generic method'u normal biçimde çağırabilirsiniz ve compiler hangi type'ı istediğinizi bulmaya çalışır. Alternatif olarak type'ın ne olduğunu açık kılmak için onu explicitly belirtebilirsiniz.

```

Box.<String>ship("package");
Box.<String[]>ship(args);

```

English: It is up to you whether this makes things clearer. You should at least be aware that this syntax exists. **Türkçe:** Bunun anlatımı daha açık yapıp yapmadığı size bağlıdır. En azından bu syntax'ın var olduğunu bilmelisiniz.

English: When you have a method declare a generic parameter type, it is independent of the class generics. Take a look at this class that declares a generic `T` at both levels: **Türkçe:** Bir method generic parameter type bildirdiğinde bu type class generic'lerinden bağımsızdır. İki level'da da generic `T` bildiren şu class'a bakın:

```

1: public class TrickyCrate<T> {
2:     public <T> T tricky(T t) {
3:         return t;
4:     }
5: }

```

English: See if you can figure out the type of `T` on lines 1 and 2 when we call the code as follows: **Türkçe:** Code'u aşağıdaki gibi çağırdığımızda lines 1–2'deki `T`'nin type'ını bulup bulamayacağınıza bakın:

```

10: public static String crateName() {
11:     TrickyCrate<Robot> crate = new TrickyCrate<>();
12:     return crate.tricky("bot");
13: }

```

English: Clearly, “T is for tricky.” Let’s see what is happening. On line 1, T is Robot because that is what gets referenced when constructing a Crate. On line 2, T is String because that is what is passed to the method. When you see code like this, take a deep breath and write down what is happening so you don’t get confused. **Türkçe:** Belli ki “T, tricky içindir.” Neler olduğuna bakalım. Crate oluşturulurken reference verilen type bu olduğu için line 1’de T, Robot’tır. Method’a geçirilen type bu olduğu için line 2’de T, String’dir. Böyle code gördüğünüzde derin bir nefes alın ve kafanız karışmasın diye neler olduğunu yazın.

Creating a Generic Record

English: Generics can also be used with records. This record takes a single generic type parameter:

Türkçe: Generics record’larla da kullanılabilir. Bu record tek bir generic type parameter alır:

```
public record CrateRecord<T>(T contents) {
    @Override
    public T contents() {
        if (contents == null)
            throw new IllegalStateException("missing contents");
        return contents;
    }
}
```

English: This works the same way as classes. You can create a record of the robot! **Türkçe:** Bu, class’larla aynı şekilde çalışır. Robot için bir record oluşturabilirsiniz!

```
Robot robot = new Robot();
CrateRecord<Robot> record = new CrateRecord<>(robot);
```

English: This is convenient. Now we have an immutable, generic record! **Türkçe:** Bu kullanışlıdır. Artık immutable, generic bir record’umuz var!

Bounding Generic Types

English: By now, you might have noticed that generics don’t seem particularly useful since they are treated as Objects and, therefore, don’t have many methods available. Bounded wildcards solve this by restricting what types can be used in a wildcard. A bounded parameter type is a generic type that specifies a bound for the generic. Be warned that this is the hardest section in the chapter, so don’t feel bad if you have to read it more than once. **Türkçe:** Generics’in Object olarak ele alındıkları ve bu nedenle çok sayıda kullanılabilir method’ları olmadığı için pek yararlı görünmediğini fark etmiş olabilirsiniz. Bounded wildcard’lar bir wildcard’da hangi type’ların kullanılabileceğini kısıtlayarak bunu çözer. Bounded parameter type, generic için bound belirten generic type’tir. Bunun bölümün en zor kısmı olduğu konusunda uyaralım; birden fazla okumanız gerekirse kendinizi kötü hissetmeyin.

English: A wildcard generic type is an unknown generic type represented with a question mark (?). You can use generic wildcards in three ways, as shown in Table 9.13. This section looks at each of these three wildcard types. **Türkçe:** Wildcard generic type, question mark (?) ile temsil edilen unknown generic type’tir. Table 9.13’te gösterildiği gibi generic wildcard’ları üç şekilde kullanabilirsiniz. Bu kısım üç wildcard type’ının her birine bakar.

Table 9.13 · Types of bounds / Bound type'ları

Type of bound / Bound type'i	Syntax	Example / Örnek
Unbounded wildcard / Sınırsız wildcard	<code>?</code>	<code>List<?> a = new ArrayList<String>();</code>
Wildcard with upper bound / Upper-bound wildcard	<code>? extends type</code>	<code>List<? extends Exception> a = new ArrayList<RuntimeException>();</code>
Wildcard with lower bound / Lower-bound wildcard	<code>? super type</code>	<code>List<? super Exception> a = new ArrayList<Object>();</code>

Türkçe tablo özeti: Unbounded wildcard yalnız `?` kullanır. Upper bound `? extends type`, lower bound `? super type` syntax'ına sahiptir.

Creating Unbounded Wildcards

English: An unbounded wildcard represents any data type. You use `?` when you want to specify that any type is okay with you. Let's suppose that we want to write a method that looks through a list of any type.

Türkçe: Unbounded wildcard herhangi bir data type'ı temsil eder. Herhangi bir type'ın uygun olduğunu belirtmek istediğinizde `?` kullanırsınız. Herhangi bir type'taki list üzerinde dolaşan method yazmak istediğimizi varsayalım.

```
public static void printList(List<Object> list) {
    for (Object x : list)
        System.out.println(x);
}
public static void main(String[] args) {
    List<String> keywords = new ArrayList<>();
    keywords.add("java");
    printList(keywords); // DOES NOT COMPILE
}
```

English: Wait. What's wrong? A `String` is a subclass of an `Object`. This is true. However, `List<String>` cannot be assigned to `List<Object>`. We know, it doesn't sound logical. Java is trying to protect us from ourselves with this one. Imagine if we could write code like this: **Türkçe:** Durun. Sorun nedir? `String`, `Object`'in subclass'ıdır; bu doğrudur. Ancak `List<String>`, `List<Object>`'e atanamaz. Kulağa mantıklı gelmediğini biliyoruz. Java burada bizi kendimizden korumaya çalışır. Şöyle code yazabildiğimizi düşünün:

```
4: List<Integer> numbers = new ArrayList<>();
5: numbers.add(Integer.valueOf(42));
6: List<Object> objects = numbers; // DOES NOT COMPILE
7: objects.add("forty two");
8: System.out.println(numbers.get(1));
```

English: On line 4, the compiler promises us that only `Integer` objects will appear in `numbers`. If line 6 compiled, line 7 would break that promise by putting a `String` in there since `numbers` and `objects` are references to the same object. Good thing the compiler prevents this. **Türkçe:** Line 4'te compiler `numbers` içinde yalnız `Integer` object'leri bulunacağını garanti eder. Line 6 derlenseydi `numbers` ve `objects` aynı object'e reference verdiği için line 7 oraya `String` koyarak bu garantiyi bozardı. Compiler'ın bunu önlemesi iyi bir şeydir.

English: Going back to printing a list, we cannot assign a `List<String>` to a `List<Object>`. That's fine; we don't need a `List<Object>`. What we really need is a `List` of "whatever." That's what `List<?>` is. The following code does what we expect: **Türkçe:** List yazdırma örneğine dönersek `List<String>`'i `List<Object>`'e atayamayız. Sorun değil; `List<Object>`'e gereksinimimiz yoktur. Asıl gereksinimimiz "herhangi bir şey" list'idir ve `List<?>` bunu ifade eder. Aşağıdaki code beklediğimizi yapar:

```
public static void printList(List<?> list) {
    for (Object x : list)
        System.out.println(x);
}
public static void main(String[] args) {
    List<String> keywords = new ArrayList<>();
    keywords.add("java");
    printList(keywords);
}
```

English: The `printList()` method takes any type of list as a parameter. The `keywords` variable is of type `List<String>`. We have a match! `List<String>` is a list of anything. "Anything" just happens to be a `String` here. **Türkçe:** `printList()` method'u parameter olarak herhangi bir type'ta list alır. `keywords` variable'nın type'ı `List<String>`dir. Eşleşme vardır! `List<String>` herhangi bir şeyin list'idir; buradaki "herhangi bir şey" tesadüfen `String`dir.

English: Finally, let's look at the impact of `var`. Do you think these two statements are equivalent?

Türkçe: Son olarak `var`'ın etkisine bakalım. Sizce bu iki statement equivalent mıdır?

```
List<?> x1 = new ArrayList<>();
var x2 = new ArrayList<>();
```

English: They are not. There are two key differences. First, `x1` is of type `List`, while `x2` is of type `ArrayList`. Additionally, we can only assign `x2` to a `List<Object>`. These two variables do have one thing in common. Both return type `Object` when calling the `get()` method. **Türkçe:** Değildir. İki temel fark vardır. İlki, `x1`in type'ı `List` iken `x2`nin type'ı `ArrayList`dir. Ayrıca `x2`yi yalnız `List<Object>`'e atayabiliriz. İki variable'ın ortak bir yönü de vardır: `get()` çağrıldığında ikisi de `Object` type döndürür.

Creating Upper-Bounded Wildcards

English: Let's try to write a method that adds up the total of a list of numbers. We've established that a generic type can't just use a subclass. **Türkçe:** Number list'inin toplamını hesaplayan method yazmayı deneyelim. Generic type'ın doğrudan subclass kullanamayacağını belirledik.

```
ArrayList<Number> list = new ArrayList<Integer>(); // DOES NOT COMPILE
```

English: Instead, we need to use a wildcard: **Türkçe:** Bunun yerine wildcard kullanmalıyız:

```
List<? extends Number> list = new ArrayList<Integer>();
```

English: The upper-bounded wildcard says that any class that extends `Number` or `Number` itself can be used as the formal parameter type: **Türkçe:** Upper-bounded wildcard, `Number`ı extend eden herhangi bir class'ın veya `Number`ın kendisinin formal parameter type olarak kullanılabileceğini söyler:

```
public static long total(List<? extends Number> list) {
    long count = 0;
    for (Number number : list)
        count += number.longValue();
    return count;
}
```

English: Remember how we kept saying that type erasure makes Java think that a generic type is an `Object`? That is still happening here. Java converts the previous code to something equivalent to the following: **Türkçe:** Type erasure'ın Java'nın generic type'ı `Object` olarak görmesine yol açtığını sürekli söylediğimizi hatırlıyor musunuz? Burada da aynı şey olur. Java önceki code'u aşağıdakine equivalent bir biçime dönüştürür:

```
public static long total(List list) {
    long count = 0;
    for (Object obj : list) {
        Number number = (Number) obj;
        count += number.longValue();
    }
    return count;
}
```

English: Something interesting happens when we work with upper bounds or unbounded wildcards. The list becomes logically immutable and therefore cannot be modified. Technically, you can remove elements from the list, but the exam won't ask about this. **Türkçe:** Upper bound veya unbounded wildcard ile çalışırken ilginç bir şey olur. List logically immutable hâle gelir ve bu nedenle değiştirilemez. Teknik olarak list'ten element kaldırabilirsiniz; ancak sınav bunu sormaz.

```
2: static class Sparrow extends Bird { }
3: static class Bird { }
4:
5: public static void main(String[] args) {
6:     List<? extends Bird> birds = new ArrayList<Bird>();
7:     birds.add(new Sparrow()); // DOES NOT COMPILE
8:     birds.add(new Bird());    // DOES NOT COMPILE
9: }
```

English: The problem stems from the fact that Java doesn't know what type `List<? extends Bird>` really is. It could be `List<Bird>` or `List<Sparrow>` or some other generic type that hasn't even been written yet. Line 7 doesn't compile because we can't add a `Sparrow` to `List<? extends Bird>`, and line 8 doesn't compile because we can't add a `Bird` to `List<Sparrow>`. From Java's point of view, both scenarios are equally possible, so neither is allowed. **Türkçe:** Sorun, Java'nın `List<? extends Bird>`'in gerçekte hangi type olduğunu bilmemesinden kaynaklanır. `List<Bird>`, `List<Sparrow>` veya henüz yazılmamış başka bir generic type olabilir. `List<? extends Bird>`'e `Sparrow` ekleyemediğimiz için line 7; `List<Sparrow>`'a `Bird` ekleyemediğimiz için line 8 derlenmez. Java açısından iki scenario da eşit ölçüde mümkündür; bu nedenle ikisine de izin verilmez.

English: Now let's try an example with an interface. We have an interface and two classes that implement it. **Türkçe:** Şimdi interface içeren bir örnek deneyelim. Bir interface ve onu implement eden iki class'ımız vardır.

```
interface Flyer { void fly(); }
class HangGlider implements Flyer { public void fly() {} }
class Goose implements Flyer { public void fly() {} }
```

English: We also have two methods that use it. One just lists the interface, and the other uses an upper bound. **Türkçe:** Onu kullanan iki method'umuz da vardır. Biri yalnız interface'i listeler, diğeri upper bound kullanır.

```
private void anyFlyer(List<Flyer> flyer) {}
private void groupOfFlyers(List<? extends Flyer> flyer) {}
```

English: Note that we used the keyword `extends` rather than `implements`. Upper bounds are like anonymous classes in that they use `extends` regardless of whether we are working with a class or an interface. **Türkçe:** `implements` yerine `extends` keyword'ünü kullandığımıza dikkat edin. Upper bound'lar, class veya interface ile çalışmamızdan bağımsız olarak `extends` kullanmaları bakımından anonymous class'lara benzer.

English: You already learned that a variable of type `List<Flyer>` can be passed to either method. A variable of type `List<Goose>` can be passed only to the one with the upper bound. This shows a benefit of generics. Random flyers don't fly together. We want our `groupOfFlyers()` method to be called only with the same type. Geese fly together but don't fly with hang gliders. **Türkçe:** `List<Flyer>` type'indeki variable'ın iki method'a da geçirilebildiğini öğrendiniz. `List<Goose>` type'indeki variable yalnız upper bound içeren method'a geçirilebilir. Bu, generics'in bir yararını gösterir. Rastgele uçanlar birlikte uçmaz. `groupOfFlyers()` method'umuzun yalnız aynı type ile çağrılmasını isteriz. Kazlar birlikte uçar, hang glider'larla uçmaz.

Creating Lower-Bounded Wildcards

English: Let's try to write a method that adds a string `"quack"` to two lists: **Türkçe:** İki list'e `"quack"` string'ini ekleyen method yazmayı deneyelim:

```
List<String> strings = new ArrayList<String>();
strings.add("tweet");

List<Object> objects = new ArrayList<Object>(strings);
addSound(strings);
addSound(objects);
```

English: The problem is that we want to pass a `List<String>` and a `List<Object>` to the same method. First, make sure you understand why the first three examples in Table 9.14 do not solve this problem. **Türkçe:** Sorun, aynı method'a hem `List<String>` hem `List<Object>` geçirmek istememizdir. Önce Table 9.14'teki ilk üç örneğin bu sorunu neden çözmediğini anladığınızdan emin olun.

Table 9.14 · Why we need a lower bound / Neden lower bound gerekir?

static void addSound(list) { list.add("quack"); }	Method compiles / Method derlenir	Can pass a List<String> / List<String> geçirilebilir	Can pass a List<Object> / List<Object> geçirilebilir
<code>List<?></code>	No (unbounded generics are immutable) / Hayır (unbounded generic'ler ekleme açısından immutable'dır)	Yes / Evet	Yes / Evet
<code>List<? extends Object></code>	No (upper-bounded generics are immutable) / Hayır (upper-bound generic'ler ekleme açısından immutable'dır)	Yes / Evet	Yes / Evet
<code>List<Object></code>	Yes / Evet	No (with generics, must pass exact match) / Hayır (generic'lerde exact match gerekir)	Yes / Evet
<code>List<? super String></code>	Yes / Evet	Yes / Evet	Yes / Evet

Türkçe tablo özeti: Unbounded ve upper-bounded list'e "quack" eklenemez. `List<Object>`e eklenebilir fakat ona `List<String>` geçirilemez. `List<? super String>` hem method'un derlenmesini hem iki list type'ının geçirilmesini sağlar.

English: To solve this problem, we need to use a lower bound. **Türkçe:** Bu sorunu çözmek için lower bound kullanmalıyız.

```
public static void addSound(List<? super String> list) {
    list.add("quack");
}
```

English: With a lower bound, we are telling Java that the list will be a list of `String` objects or a list of some objects that are a superclass of `String`. Either way, it is safe to add a `String` to that list. **Türkçe:** Lower bound ile Java'ya list'in `String` object'lerinden veya `String`in superclass'ı olan bazı object'lerden oluşacağını söyleriz. Her iki durumda da bu list'e `String` eklemek güvenlidir.

English: Just like generic classes, you probably won't use this in your code unless you are writing code for others to reuse. Even then, it would be rare. But it's on the exam, so now is the time to learn it! **Türkçe:** Generic class'larda olduğu gibi başkalarının reuse edeceği code yazmıyorsanız muhtemelen bunu kendi code'unuzda kullanmayacaksınız. O durumda bile nadir olurdu. Ancak sınavda yer aldığı için öğrenmenin zamanı şimdi!

Understanding Generic Supertypes

English: When you have subclasses and superclasses, lower bounds can get tricky. **Türkçe:** Subclass ve superclass'larınız olduğunda lower bound'lar zorlaşabilir.

```
3: List<? super IOException> exceptions = new ArrayList<Exception>();
4: exceptions.add(new Exception());           // DOES NOT COMPILE
5: exceptions.add(new IOException());
6: exceptions.add(new FileNotFoundException());
```

English: Line 3 references a `List` that could be `List<IOException>` or `List<Exception>` or `List<Object>`. Line 4 does not compile because we could have a `List<IOException>`, and an `Exception` object wouldn't fit in there. **Türkçe:** Line 3, `List<IOException>`, `List<Exception>` veya `List<Object>` olabilecek bir `Liste` reference verir. `List<IOException>` söz konusu olabileceği ve `Exception` object oraya sığmayacağı için line 4 derlenmez.

English: Line 5 is fine. `IOException` can be added to any of those types. Line 6 is also fine. `FileNotFoundException` can also be added to any of those three types. This is tricky because `FileNotFoundException` is a subclass of `IOException`, and the keyword says `super`. Java says, "Well, `FileNotFoundException` also happens to be an `IOException`, so everything is fine." **Türkçe:** Line 5 geçerlidir. `IOException` bu type'ların herhangi birine eklenebilir. Line 6 da geçerlidir; `FileNotFoundException` üç type'ın herhangi birine eklenebilir. `FileNotFoundException`, `IOException`ın subclass'ı olduğu hâlde keyword `super` dediği için bu tuzaklıdır. Java, "`FileNotFoundException` aynı zamanda bir `IOException`;" dolayısıyla sorun yok" der.

Putting It All Together

English: At this point, you know everything that you need to know to ace the exam questions on generics. It is possible to put these concepts together to write some really confusing code, which the exam likes to do. **Türkçe:** Bu noktada generics hakkındaki sınav sorularını başarıyla çözmek için bilmeniz gereken her şeyi biliyorsunuz. Bu kavramları birleştirerek gerçekten kafa karıştırıcı code yazmak mümkündür; sınav da bunu yapmayı sever.

English: This section is going to be difficult to read. It contains the hardest questions that you could possibly be asked about generics. The exam questions will probably be easier to read than these. We want you to encounter the really tough ones here so that you are ready for the exam. In other words, don't panic. Take it slow, and reread the code a few times. You'll get it. **Türkçe:** Bu kısmın okunması zor olacaktır. Generics hakkında sorulabilecek en zor soruları içerir. Sınav sorularının okunması muhtemelen bunlardan daha kolay olacaktır. Sınava hazır olmanız için gerçekten zor olanlarla burada karşılaşmanızı istiyoruz. Başka bir deyişle paniğe kapılmayın; yavaş ilerleyin ve code'u birkaç kez yeniden okuyun. Anlayacaksınız.

Combining Generic Declarations

English: Let's try an example. First, we declare three classes that the example will use: **Türkçe:** Bir örnek deneyelim. Önce örneğin kullanacağı üç class'ı bildiriyoruz:

```
class A {}  
class B extends A {}  
class C extends B {}
```

English: Ready? Can you figure out why these do or don't compile? Also, try to figure out what they do.

Türkçe: Hazır mısınız? Bunların neden derlenip derlenmediğini bulabilir misiniz? Ayrıca ne yaptıklarını belirlemeyi deneyin.

```
6: List<?> list1 = new ArrayList<A>();  
7: List<? extends A> list2 = new ArrayList<A>();  
8: List<? super A> list3 = new ArrayList<A>();
```

English: Line 6 creates an `ArrayList` that can hold instances of class `A`. It is stored in a variable with an unbounded wildcard. Any generic type can be referenced from an unbounded wildcard, making this okay.

Türkçe: Line 6, class `A` instance'larını tutabilen `ArrayList` oluşturur. Unbounded wildcard'lı bir variable'da saklanır. Her generic type unbounded wildcard'dan reference edilebildiği için bu geçerlidir.

English: Line 7 tries to store a list in a variable declaration with an upper-bounded wildcard. This is okay. You can have `ArrayList<A>`, `ArrayList<B>`, or `ArrayList<C>` stored in that reference. Line 8 is also okay. This time, you have a lower-bounded wildcard. The lowest type you can reference is `A`. Since that is what you have, it compiles.

Türkçe: Line 7 bir list'i upper-bounded wildcard'lı variable declaration'da saklar; bu geçerlidir. O reference içinde `ArrayList<A>`, `ArrayList<B>` veya `ArrayList<C>` bulunabilir. Line 8 de geçerlidir. Bu kez lower-bounded wildcard vardır. Reference edebileceğiniz en alt type `A`dır ve elinizdeki de bu olduğu için derlenir.

English: Did you get those right? Let's try a few more. **Türkçe:** Bunları doğru buldunuz mu? Birkaç tane daha deneyelim.

```
9: List<? extends B> list4 = new ArrayList<A>(); // DOES NOT COMPILE  
10: List<? super B> list5 = new ArrayList<A>();  
11: List<?> list6 = new ArrayList<? extends A>(); // DOES NOT COMPILE
```

English: Line 9 has an upper-bounded wildcard that allows `ArrayList<B>` or `ArrayList<C>` to be referenced. Since you have `ArrayList<A>` that is trying to be referenced, the code does not compile.

Line 10 has a lower-bounded wildcard, which allows a reference to `ArrayList<A>`, `ArrayList<B>`, or `ArrayList<Object>`. **Türkçe:** Line 9, `ArrayList<B>` veya `ArrayList<C>` reference'ına izin veren upper-bounded wildcard'a sahiptir. Reference edilmeye çalışılan `ArrayList<A>` olduğu için code derlenmez. Line 10; `ArrayList<A>`, `ArrayList<B>` veya `ArrayList<Object>` reference'ına izin veren lower-bounded wildcard'a sahiptir.

English: Finally, line 11 allows a reference to any generic type since it is an unbounded wildcard. The problem is that you need to know what that type will be when instantiating the `ArrayList`. It wouldn't be useful anyway, because you can't add any elements to that `ArrayList`. **Türkçe:** Son olarak line 11 unbounded wildcard olduğu için herhangi bir generic type reference'ına izin verir. Sorun, `ArrayList` instantiate edilirken bu type'ın ne olacağını bilmeniz gerekmesidir. Zaten bu `ArrayList` hiçbir element ekleyemeyeceğiniz için yararlı da olmazdı.

Passing Generic Arguments

English: Now on to the methods. Same question: try to figure out why they don't compile or what they do. We will present the methods one at a time because there is more to think about. **Türkçe:** Şimdi method'lara geçelim. Soru aynı: neden derlenmediklerini veya ne yaptıklarını bulmaya çalışın. Düşünülecek daha çok şey olduğundan method'ları birer birer sunacağız.

```
<T> T first(List<? extends T> list) {  
    return list.get(0);  
}
```

English: The first method, `first()`, is a perfectly normal use of generics. It uses a method-specific type parameter, `T`. It takes a parameter of `List<T>`, or some subclass of `T`, and it returns a single object of that `T` type. For example, you could call it with a `List<String>` parameter and have it return a `String`. Or you could call it with a `List<Number>` parameter and have it return a `Number`. Or—well, you get the idea. **Türkçe:** İlk method `first()`, generics'in tamamen normal bir kullanımıdır. Method-specific type parameter `T`'yi kullanır. `List<T>` veya `T`nin bir subclass'ından parameter alır ve bu `T` type'ında tek bir object döndürür. Örneğin `List<String>` parameter ile çağırıp `String`, `List<Number>` parameter ile çağırıp `Number` döndürmesini sağlayabilirsiniz. Fikri anladınız.

English: Given that, you should be able to see what is wrong with this one: **Türkçe:** Buna göre aşağıdaki method'da neyin yanlış olduğunu görebilmeniz gerekir:

```
<T> <? extends T> second(List<? extends T> list) { // DOES NOT COMPILE  
    return list.get(0);  
}
```

English: The next method, `second()`, does not compile because the return type isn't actually a type. You are writing the method. You know what type it is supposed to return. You don't get to specify this as a wildcard. **Türkçe:** Sonraki method `second()`, return type gerçekte bir type olmadığı için derlenmez. Method'u siz yazıyorsunuz ve hangi type'ı döndürmesi gerektiğini biliyorsunuz. Bunu wildcard olarak belirtemezsiniz.

English: Now be careful—this one is extra tricky: **Türkçe:** Şimdi dikkatli olun; bu özellikle tuzaklıdır:

```
<B extends A> B third(List<B> list) {  
    return new B(); // DOES NOT COMPILE  
}
```

English: This method, `third()`, does not compile. `<B extends A>` says that you want to use `B` as a type parameter just for this method and that it needs to extend the `A` class. Coincidentally, `B` is also the name of a class. Well, it isn't a coincidence. It's an evil trick. Within the scope of the method, `B` can represent class `A`, `B`, or `C`, because all extend the `A` class. Since `B` no longer refers to the `B` class in the method, you can't instantiate it. **Türkçe:** `third()` method'u derlenmez. `<B extends A>`, yalnız bu method için `B`yi type parameter olarak kullanmak istediğinizi ve onun `A` class'ını extend etmesi gerektiğini söyler. Tesadüfen `B` aynı zamanda bir class adıdır; aslında bu tesadüf değil, kötü bir tuzaktır. Method scope'u içinde `B`, `A` class'ını extend ettikleri için class `A`, `B` veya `C`yi temsil edebilir. Method içinde `B` artık `B` class'ına referans vermediğinden onu instantiate edemezsiniz.

English: After that, it would be nice to get something straightforward. **Türkçe:** Bundan sonra açık bir örnek görmek iyi olacaktır.

```
void fourth(List<? super B> list) {}
```

English: We finally get a method, `fourth()`, that is a normal use of generics. You can pass the type `List<B>`, `List<A>`, or `List<Object>`. **Türkçe:** Sonunda generics'in normal bir kullanımı olan `fourth()` method'unu görürüz. `List<B>`, `List<A>` veya `List<Object>` type'ını geçirebilirsiniz.

English: Finally, can you figure out why this example does not compile? **Türkçe:** Son olarak bu örneğin neden derlenmediğini bulabilir misiniz?

```
<X> void fifth(List<X super B> list) { // DOES NOT COMPILE
}
```

English: This last method, `fifth()`, does not compile because it tries to mix a method-specific type parameter with a wildcard. A wildcard must have a `?` in it. **Türkçe:** Son method `fifth()`, method-specific type parameter ile wildcard'ı karıştırmaya çalıştığı için derlenmez. Bir wildcard içinde `?` bulunmalıdır.

English: Phew. You made it through generics. It's the hardest topic in this chapter (and why we covered it last!). Remember that it's okay if you need to go over this material a few times to get your head around it. **Türkçe:** Oh! Generics kısmını tamamladınız. Bölümün en zor konusudur ve bu yüzden en son ele aldık. Anlamak için material'ı birkaç kez gözden geçirmeniz gerekmesinin normal olduğunu unutmayın.

Summary / Özet

English: The Java Collections Framework includes four main types of data structures: lists, sets, queues, and maps. The `Collection` interface is the parent interface of `List`, `Set`, and `Queue`. Additionally, `Deque` extends `Queue`. The `Map` interface does not extend `Collection`. You need to recognize the following: **Türkçe:** Java Collections Framework dört ana data structure type'ı içerir: list, set, queue ve map. `Collection` interface'i `List`, `Set` ve `Queue`un parent interface'idir. Ayrıca `Deque`, `Queue`yu extend eder. `Map` interface'i `Collection`ı extend etmez. Aşağıdakileri tanımanız gerekir:

English: `List`: An ordered collection of elements that allows duplicate entries. **Türkçe:** `List`: Duplicate entry'lere izin veren ordered element collection'ı.

English: `ArrayList`: Standard resizable list. **Türkçe:** `ArrayList`: Standard resizable list.

English: `LinkedList`: Can easily add/remove from beginning or end. **Türkçe:** `LinkedList`: Başlangıçtan veya sondan kolayca ekleme/kaldırma yapabilir.

English: `Set`: Does not allow duplicates. **Türkçe:** `Set`: Duplicate'lere izin vermez.

English: `HashSet`: Uses `hashCode()` to find unordered elements. **Türkçe:** `HashSet`: Unordered element'ları bulmak için `hashCode()` kullanır.

English: `TreeSet`: Sorted. Does not allow `null` values. **Türkçe:** `TreeSet`: Sorted'dır. `null` value'lara izin vermez.

English: `Queue/Deque`: Orders elements for processing. **Türkçe:** `Queue/Deque`: Element'ları işlenmek üzere order'a koyar.

English: `ArrayDeque`: Double-ended queue. **Türkçe:** `ArrayDeque`: Double-ended queue.

English: `LinkedList`: Double-ended queue and list. **Türkçe:** `LinkedList`: Double-ended queue ve list.

English: `Map`: Maps unique keys to values. **Türkçe:** `Map`: Unique key'leri value'lara map eder.

English: `HashMap`: Uses `hashCode()` to find keys. **Türkçe:** `HashMap`: Key'leri bulmak için `hashCode()` kullanır.

English: `TreeMap`: Sorted map. Does not allow `null` keys. **Türkçe:** `TreeMap`: Sorted map'tir. `null` key'lere izin vermez.

English: The `Comparable` interface declares the `compareTo()` method. This method returns a negative number if the object is smaller than its argument, 0 if the two objects are equal, and a positive number otherwise. The `compareTo()` method is declared on the object that is being compared, and it takes one parameter. The `Comparator` interface defines the `compare()` method. A negative number is returned if the first argument is smaller, zero if they are equal, and a positive number otherwise. The `compare()` method can be declared in any code, and it takes two parameters. A `Comparator` is often implemented using a lambda. **Türkçe:** `Comparable` interface'i `compareTo()` method'unu bildirir. Bu method object argument'ından küçükse negative number, ikisi equal ise 0, aksi hâlde positive number döndürür. `compareTo()` karşılaştırılan object üzerinde bildirilir ve tek parameter alır. `Comparator` interface'i `compare()` method'unu tanımlar. İlk argument küçükse negative number, equal ise zero, aksi hâlde positive number döndürülür. `compare()` herhangi bir code içinde bildirilebilir ve iki parameter alır. `Comparator` sıklıkla lambda ile implement edilir.

English: Generics are type parameters for code. To create a class with a generic parameter, add `<T>` after the class name. You can use any name you want for the type parameter. Single uppercase letters are common choices. Generics allow you to specify wildcards. `<?>` is an unbounded wildcard that means any type. `<? extends Object>` is an upper bound that means any type that is `Object` or extends it. `<? extends MyInterface>` means any type that implements `MyInterface`. `<? super Number>` is a lower bound that means any type that is `Number` or a superclass. A compiler error results from code that attempts to add an item in a list with an unbounded or upper-bounded wildcard. **Türkçe:** Generics code için type parameter'lardır. Generic parameter'lı class oluşturmak için class adından sonra `<T>` ekleyin. Type parameter için istediğiniz adı kullanabilirsiniz; tek uppercase letter'lar yaygın seçimdir. Generics wildcard belirtmenize izin verir. `<?>` herhangi bir type anlamına gelen unbounded wildcard'dır. `<? extends Object>`, `Object` veya onu extend eden herhangi bir type anlamındaki upper bound'dur. `<? extends MyInterface>`, `MyInterface`i implement eden herhangi bir type demektir. `<? super Number>`, `Number` veya superclass'ı olan herhangi bir type anlamındaki lower bound'dur. Unbounded veya upper-bounded wildcard'lı list'e öge eklemeye çalışan code compiler error üretir.

Exam Essentials / Sınav İçin Temel Noktalar

English: Pick the correct type collection from a description. A `List` allows duplicates and orders the elements. A `Set` does not allow duplicates. A `Deque` orders its elements to facilitate retrievals from the front or back. A `Map` maps keys to values. Be familiar with the differences in implementations of these interfaces. **Türkçe:** Description'dan doğru collection type'ını seçin. `List` duplicate'lere izin verir ve element'ları order'a koyar. `Set` duplicate'lere izin vermez. `Deque`, front veya back'ten retrieval'i kolaylaştırmak için element'larını order'a koyar. `Map` key'leri value'lara map eder. Bu interface'lerin implementation'larındaki farklara aşina olun.

English: Work with convenience methods. The Collections Framework contains many methods such as `contains()`, `forEach()`, and `removeIf()` that you need to know for the exam. There are too many to list in this paragraph for review, so please do review the tables in this chapter. **Türkçe:** Convenience method'larla çalışın. Collections Framework sınav için bilmeniz gereken `contains()`, `forEach()` ve `removeIf()` gibi birçok method içerir. Bu paragraph'ta review için listelenemeyecek kadar çok olduklarından bu bölümdeki tabloları mutlaka gözden geçirin.

English: Differentiate between `Comparable` and `Comparator`. Classes that implement `Comparable` are said to have a natural ordering and implement the `compareTo()` method. A class is allowed to have only one natural ordering. A `Comparator` takes two objects in the `compare()` method. Different ones can have different sort orders. A `Comparator` is often implemented using a lambda such as `(a, b) -> a.num - b.num`. **Türkçe:** `Comparable` ile `Comparator`ı ayırt edin. `Comparable` implement eden class'ların natural ordering'e sahip olduğu söylenir ve bunlar `compareTo()` method'unu implement eder. Bir class'ın yalnız bir natural ordering'i olabilir. `Comparator`, `compare()` method'unda iki object alır. Farklı comparator'lar farklı sort order'lara sahip olabilir. `Comparator` sıklıkla `(a, b) -> a.num - b.num` gibi lambda ile implement edilir.

English: Identify valid and invalid uses of generics and wildcards. `<T>` represents a type parameter. Any name can be used, but a single uppercase letter is the convention. `<?>` is an unbounded wildcard. `<? extends X>` is an upper-bounded wildcard. `<? super X>` is a lower-bounded wildcard. **Türkçe:** Generics ve wildcard'ların valid ve invalid kullanımlarını belirleyin. `<T>` type parameter'ı temsil eder. Her ad kullanılabilir; ancak tek uppercase letter convention'dır. `<?>` unbounded wildcard, `<? extends X>` upper-bounded wildcard, `<? super X>` lower-bounded wildcard'dır.

Review Questions / Gözden Geçirme Soruları

English: The answers to the chapter review questions can be found in the Appendix. **Türkçe:** Bölüm review question'larının cevapları Appendix'te bulunabilir.

Question 1

English: Suppose you need to display a collection of products for sale, which may contain duplicates. Additionally, you have a collection of sales that you need to track, sorted by the natural order of the sale ID, and you need to retrieve the text of each. Which two of the following from the `java.util` package best suit your needs for this scenario? (Choose two.) **Türkçe:** Satıştaki ürünlerden oluşan ve duplicate içerebilen bir collection'ı göstermeniz gerektiğini varsayın. Ayrıca izlemeniz gereken, sale ID'nin natural order'ına göre sorted bir satış collection'ınız var ve her birinin text'ini getirmeniz gerekiyor. `java.util` package'ındaki aşağıdakilerden hangi ikisi bu scenario'daki gereksinimlerinize en uygundur? (İki tane seçin.)

English — A: `ArrayList` **Türkçe — A:** `ArrayList`

English — B: `HashMap` **Türkçe — B:** `HashMap`

English — C: `HashSet` **Türkçe — C:** `HashSet`

English — D: `LinkedList` **Türkçe — D:** `LinkedList`

English — E: `TreeMap` **Türkçe — E:** `TreeMap`

English — F: `TreeSet` **Türkçe — F:** `TreeSet`

Question 2

English: Which of the following are true? (Choose all that apply.) **Türkçe:** Aşağıdakilerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

```
12: List<?> q = List.of("mouse", "parrot");
13: var v = List.of("mouse", "parrot");
14:
15: q.removeIf(String::isEmpty);
16: q.removeIf(s -> s.length() == 4);
17: v.removeIf(String::isEmpty);
18: v.removeIf(s -> s.length() == 4);
```

English — A: This code compiles and runs without error. **Türkçe — A:** Bu code derlenir ve hata olmadan çalışır.

English — B: Exactly one of these lines contains a compiler error. **Türkçe — B:** Bu satırların tam olarak biri compiler error içerir.

English — C: Exactly two of these lines contain a compiler error. **Türkçe — C:** Bu satırların tam olarak ikisi compiler error içerir.

English — D: Exactly three of these lines contain a compiler error. **Türkçe — D:** Bu satırların tam olarak üçü compiler error içerir.

English — E: Exactly four of these lines contain a compiler error. **Türkçe — E:** Bu satırların tam olarak dördü compiler error içerir.

English — F: If any lines with compiler errors are removed, this code runs without throwing an exception. **Türkçe — F:** Compiler error içeren satırlar kaldırılırsa bu code exception fırlatmadan çalışır.

English — G: If any lines with compiler errors are removed, this code throws an exception. **Türkçe — G:** Compiler error içeren satırlar kaldırılırsa bu code exception fırlatır.

Question 3

English: What is the result of the following statements? **Türkçe:** Aşağıdaki statement'ların sonucu nedir?

```
3: var greetings = new ArrayDeque<String>();
4: greetings.offerLast("hello");
5: greetings.offerLast("hi");
6: greetings.offerFirst("ola");
7: greetings.pop();
8: greetings.peek();
```

```
9: while (greetings.peek() != null)
10:     System.out.print(greetings.pop());
```

English — A: `hello` **Türkçe — A:** `hello`

English — B: `hellohi` **Türkçe — B:** `hellohi`

English — C: `hellohiola` **Türkçe — C:** `hellohiola`

English — D: `hiola` **Türkçe — D:** `hiola`

English — E: The code does not compile. **Türkçe — E:** Code derlenmez.

English — F: An exception is thrown. **Türkçe — F:** Exception fırlatılır.

Question 4

English: Which of these statements compile? (Choose all that apply.) **Türkçe:** Bu statement'lardan hangileri derlenir? (Uygun olanların tümünü seçin.)

English — A: `HashSet<Number> hs = new HashSet<Integer>();` **Türkçe — A:**
`HashSet<Number> hs = new HashSet<Integer>();`

English — B: ``HashSet<? super ClassCastException> set = new HashSet<Exception>();`` **Türkçe — B:**
``HashSet<? super ClassCastException> set = new HashSet<Exception>();``

English — C: `List<> list = new ArrayList<String>();` **Türkçe — C:** `List<> list = new ArrayList<String>();`

English — D: `List<Object> values = new HashSet<Object>();` **Türkçe — D:** `List<Object> values = new HashSet<Object>();`

English — E: `List<Object> objects = new ArrayList<? extends Object>();` **Türkçe — E:**
`List<Object> objects = new ArrayList<? extends Object>();`

English — F: ``Map<String, ? extends Number> hm = new HashMap<String, Integer>();`` **Türkçe — F:**
``Map<String, ? extends Number> hm = new HashMap<String, Integer>();``

Question 5

English: What is the result of the following code? **Türkçe:** Aşağıdaki code'un sonucu nedir?

```

1: public record Hello<T>(T t) {
2:     public Hello(T t) { this.t = t; }
3:     private <T> void println(T message) {
4:         System.out.print(t + "-" + message);
5:     }
6:     public static void main(String[] args) {
7:         new Hello<String>("hi").println(1);
8:         new Hello("hola").println(true);
9:     } }

```

English — A: `hi` followed by a runtime exception **Türkçe — A:** `hi` ve ardından runtime exception

English — B: `hi-1hola-true` **Türkçe — B:** `hi-1hola-true`

English — C: The first compiler error is on line 1. **Türkçe — C:** İlk compiler error line 1'dedir.

English — D: The first compiler error is on line 3. **Türkçe — D:** İlk compiler error line 3'tedir.

English — E: The first compiler error is on line 8. **Türkçe — E:** İlk compiler error line 8'dedir.

English — F: The first compiler error is on another line. **Türkçe — F:** İlk compiler error başka bir satırdadır.

Question 6

English: Which of the following can fill in the blank to print `[7, 5, 3]`? (Choose all that apply.) **Türkçe:** `[7, 5, 3]` yazdırmak için aşağıdakilerden hangileri boşluğu doldurabilir? (Uygun olanların tümünü seçin.)

```

8: public record Platypus(String name, int beakLength) {
9:     @Override public String toString() { return "" + beakLength; }
10:
11:     public static void main(String[] args) {
12:         Platypus p1 = new Platypus("Paula", 3);
13:         Platypus p2 = new Platypus("Peter", 5);
14:         Platypus p3 = new Platypus("Peter", 7);
15:
16:         List<Platypus> list = Arrays.asList(p1, p2, p3);
17:
18:         Collections.sort(list, Comparator.comparing____);
19:
20:         System.out.println(list);
21:     }
22: }

```

English — A: `(Platypus::beakLength)` **Türkçe — A:** `(Platypus::beakLength)`

English — B: `(Platypus::beakLength).reversed()` **Türkçe — B:**
`(Platypus::beakLength).reversed()`

English — C: `(Platypus::name).thenComparing(Platypus::beakLength)` **Türkçe — C:**
`(Platypus::name).thenComparing(Platypus::beakLength)`

English — D: ``(Platypus::name).thenComparing(Comparator.comparing(Platypus::beakLength).reversed())`` **Türkçe — D:**
``(Platypus::name).thenComparing(Comparator.comparing(Platypus::beakLength).reversed())``

English — E: ``(Platypus::name).thenComparingNumber(Platypus::beakLength).reversed())`` **Türkçe — E:**
``(Platypus::name).thenComparingNumber(Platypus::beakLength).reversed())``

English — F: ``(Platypus::name).thenComparingInt(Platypus::beakLength).reversed())`` **Türkçe — F:**
``(Platypus::name).thenComparingInt(Platypus::beakLength).reversed())``

English — G: None of the above **Türkçe — G:** Yukarıdakilerin hiçbirisi

Question 7

English: Which of the following method signatures are valid overrides of the `hairy()` method in the `Alpaca` class? (Choose all that apply.) **Türkçe:** Aşağıdaki method signature'larından hangileri `Alpaca` class'ındaki `hairy()` method'unun valid override'larıdır? (Uygun olanların tümünü seçin.)

```
import java.util.*;

public class Alpaca {
    public List<String> hairy(List<String> list) { return null; }
}
```

English — A: ``public List<String> hairy(List<CharSequence> list) { return null; }`` **Türkçe — A:** ``public List<String> hairy(List<CharSequence> list) { return null; }``

English — B: ``public List<String> hairy(ArrayList<String> list) { return null; }`` **Türkçe — B:** ``public List<String> hairy(ArrayList<String> list) { return null; }``

English — C: ``public List<String> hairy(List<Integer> list) { return null; }`` **Türkçe — C:** ``public List<String> hairy(List<Integer> list) { return null; }``

English — D: ``public List<CharSequence> hairy(List<String> list) { return null; }`` **Türkçe — D:** ``public List<CharSequence> hairy(List<String> list) { return null; }``

English — E: `public Object hairy(List<String> list) { return null; }` **Türkçe — E:**
`public Object hairy(List<String> list) { return null; }`

English — F: ``public ArrayList<String> hairy(List<String> list) { return null; }`` **Türkçe — F:** ``public ArrayList<String> hairy(List<String> list) { return null; }``

Question 8

English: What is the result of the following program? **Türkçe:** Aşağıdaki programın sonucu nedir?

```
3: public class MyComparator implements Comparator<String> {
4:     public int compare(String a, String b) {
5:         return b.toLowerCase().compareTo(a.toLowerCase());
6:     }
7:     public static void main(String[] args) {
8:         String[] values = { "123", "Abb", "aab" };
9:         Arrays.sort(values, new MyComparator());
10:        for (var s : values)
11:            System.out.print(s + " ");
12:    }
13: }
```

English — A: Abb aab 123 **Türkçe — A:** Abb aab 123

English — B: aab Abb 123 **Türkçe — B:** aab Abb 123

English — C: 123 Abb aab **Türkçe — C:** 123 Abb aab

English — D: 123 aab Abb **Türkçe — D:** 123 aab Abb

English — E: The code does not compile. **Türkçe — E:** Code derlenmez.

English — F: A runtime exception is thrown. **Türkçe — F:** Runtime exception fırlatılır.

Question 9

English: Which of these statements can fill in the blank so that the `Helper` class compiles successfully? (Choose all that apply.) **Türkçe:** Bu statement'lardan hangileri `Helper` class'ının başarıyla derlenmesi için boşluğu doldurabilir? (Uygun olanların tümünü seçin.)

```
2: public class Helper {
3:     public static <U extends Exception>
4:         void printException(U u) {
5:
6:         System.out.println(u.getMessage());
7:     }
8:     public static void main(String[] args) {
9:         Helper._____;
10:    } }
```

English — A: `printException(new FileNotFoundException("A"))` **Türkçe — A:**
`printException(new FileNotFoundException("A"))`

English — B: `printException(new Exception("B"))` **Türkçe — B:** `printException(new Exception("B"))`

English — C: `<Throwable>printException(new Exception("C"))` **Türkçe — C:**
`<Throwable>printException(new Exception("C"))`

English — D: ``<NullPointerException>printException(new NullPointerException("D"))`` **Türkçe — D:**
``<NullPointerException>printException(new NullPointerException("D"))``

English — E: `printException(new Throwable("E"))` **Türkçe — E:** `printException(new Throwable("E"))`

Question 10

English: Which of the following will compile when filling in the blank? (Choose all that apply.) **Türkçe:**
 Boşluk doldurulduğunda aşağıdakilerden hangileri derlenir? (Uygun olanların tümünü seçin.)

```
var list = List.of(1, 2, 3);
var set = Set.of(1, 2, 3);
var map = Map.of(1, 2, 3, 4);
_____.forEach(System.out::println);
```

English — A: `list` **Türkçe — A:** `list`

English — B: `set` **Türkçe — B:** `set`

English — C: `map` **Türkçe — C:** `map`

English — D: `map.keys()` **Türkçe — D:** `map.keys()`

English — E: `map.keySet()` **Türkçe — E:** `map.keySet()`

English — F: `map.values()` **Türkçe — F:** `map.values()`

English — G: `map.valueSet()` **Türkçe — G:** `map.valueSet()`

Question 11

English: Which of these statements can fill in the blank so that the `Wildcard` class compiles successfully? (Choose all that apply.) **Türkçe:** Bu statement'lerden hangileri `Wildcard` class'ının başarıyla derlenmesi için boşluğu doldurabilir? (Uygun olanların tümünü seçin.)

```
3: public class Wildcard {
4:     public void showSize(List<?> list) {
5:         System.out.println(list.size());
6:     }
7:     public static void main(String[] args) {
8:         Wildcard card = new Wildcard();
9:         _____;
10:        card.showSize(list);
11:    } }
```

English — A: `List<?> list = new HashSet<String>()` **Türkçe — A:** `List<?> list = new HashSet<String>()`

English — B: `ArrayList<? super Date> list = new ArrayList<Date>()` **Türkçe — B:** `ArrayList<? super Date> list = new ArrayList<Date>()`

English — C: `List<?> list = new ArrayList<?>()` **Türkçe — C:** `List<?> list = new ArrayList<?>()`

English — D: `List<Exception> list = new LinkedList<java.io.IOException>()` **Türkçe — D:** `List<Exception> list = new LinkedList<java.io.IOException>()`

English — E: `ArrayList<? extends Number> list = new ArrayList<Integer>()` **Türkçe — E:** `ArrayList<? extends Number> list = new ArrayList<Integer>()`

English — F: None of the above. **Türkçe — F:** Yukarıdakilerin hiçbiri.

Question 12

English: What is the result of the following program? **Türkçe:** Aşağıdaki programın sonucu nedir?

```

3: public record Sorted(int num, String text)
4:     implements Comparable<Sorted>, Comparator<Sorted> {
5:
6:     public String toString() { return "" + num; }
7:     public int compareTo(Sorted s) {
8:         return text.compareTo(s.text);
9:     }
10:    public int compare(Sorted s1, Sorted s2) {
11:        return s1.num - s2.num;
12:    }
13:    public static void main(String[] args) {
14:        var s1 = new Sorted(88, "a");
15:        var s2 = new Sorted(55, "b");
16:        var t1 = new TreeSet<Sorted>();
17:        t1.add(s1); t1.add(s2);
18:        var t2 = new TreeSet<Sorted>(s1);
19:        t2.add(s1); t2.add(s2);
20:        System.out.println(t1 + " " + t2);
21:    } }

```

English — A: [55, 88] [55, 88] **Türkçe — A:** [55, 88] [55, 88]

English — B: [55, 88] [88, 55] **Türkçe — B:** [55, 88] [88, 55]

English — C: [88, 55] [55, 88] **Türkçe — C:** [88, 55] [55, 88]

English — D: [88, 55] [88, 55] **Türkçe — D:** [88, 55] [88, 55]

English — E: The code does not compile. **Türkçe — E:** Code derlenmez.

English — F: A runtime exception is thrown. **Türkçe — F:** Runtime exception fırlatılır.

Question 13

English: What is the result of the following code? (Choose all that apply.) **Türkçe:** Aşağıdaki code'un sonucu nedir? (Uygun olanların tümünü seçin.)

```

Comparator<Integer> c1 = (o1, o2) -> o2 - o1;
Comparator<Integer> c2 = Comparator.naturalOrder();
Comparator<Integer> c3 = Comparator.reverseOrder();

```

```

var list = Arrays.asList(5, 4, 7, 2);
Collections.sort(list, _____);
Collections.reverse(list);
Collections.reverse(list);
System.out.println(Collections.binarySearch(list, 2));

```

English — A: One or more of the comparators can fill in the blank so that the code prints 0. **Türkçe — A:** Comparator'lerden biri veya daha fazlası boşluğu doldurabilir ve code 0 yazdırır.

English — B: One or more of the comparators can fill in the blank so that the code prints `1`. **Türkçe — B:** Comparator'lardan biri veya daha fazlası boşluğu doldurabilir ve code `1` yazdırır.

English — C: One or more of the comparators can fill in the blank so that the code prints `2`. **Türkçe — C:** Comparator'lardan biri veya daha fazlası boşluğu doldurabilir ve code `2` yazdırır.

English — D: The result is undefined regardless of which comparator is used. **Türkçe — D:** Hangi comparator kullanılırsa kullanılsın sonuç tanımsızdır.

English — E: A runtime exception is thrown regardless of which comparator is used. **Türkçe — E:** Hangi comparator kullanılırsa kullanılsın runtime exception fırlatılır.

English — F: The code does not compile. **Türkçe — F:** Code derlenmez.

Question 14

English: Which of the following lines can be inserted to make the code compile? (Choose all that apply.)

Türkçe: Code'un derlenmesini sağlamak için aşağıdaki satırlardan hangileri eklenebilir? (Uygun olanların tümünü seçin.)

```
class W {}
class X extends W {}
class Y extends X {}
class Z<Y> {
    // INSERT CODE HERE
}
```

English — A: `W w1 = new W();` **Türkçe — A:** `W w1 = new W();`

English — B: `W w2 = new X();` **Türkçe — B:** `W w2 = new X();`

English — C: `W w3 = new Y();` **Türkçe — C:** `W w3 = new Y();`

English — D: `Y y1 = new W();` **Türkçe — D:** `Y y1 = new W();`

English — E: `Y y2 = new X();` **Türkçe — E:** `Y y2 = new X();`

English — F: `Y y1 = new Y();` **Türkçe — F:** `Y y1 = new Y();`

Question 15

English: Which options are true of the following code? (Choose all that apply.) **Türkçe:** Aşağıdaki code hakkında hangi seçenekler doğrudur? (Uygun olanların tümünü seçin.)

```
3: _____ q = new LinkedList<>();
4: q.add(10);
5: q.add(12);
6: q.remove(1);
7: System.out.print(q);
```

English — A: If we fill in the blank with `List<Integer>`, the output is `[10]`. **Türkçe — A:** Boşluğu `List<Integer>` ile doldurursak çıktı `[10]` olur.

English — B: If we fill in the blank with `Queue<Integer>`, the output is `[10]`. **Türkçe — B:** Boşluğu `Queue<Integer>` ile doldurursak çıktı `[10]` olur.

English — C: If we fill in the blank with `var`, the output is `[10]`. **Türkçe — C:** Boşluğu `var` ile doldurursak çıktı `[10]` olur.

English — D: One or more of the scenarios does not compile. **Türkçe — D:** Senaryolardan biri veya daha fazlası derlenmez.

English — E: One or more of the scenarios throws a runtime exception. **Türkçe — E:** Senaryolardan biri veya daha fazlası runtime exception fırlatır.

Question 16

English: What is the result of the following code? **Türkçe:** Aşağıdaki code'un sonucu nedir?

```
4: Map m = new HashMap();
5: m.put(123, "456");
6: m.put("abc", "def");
7: System.out.println(m.contains("123"));
```

English — A: `false` **Türkçe — A:** `false`

English — B: `true` **Türkçe — B:** `true`

English — C: Compiler error on line 4. **Türkçe — C:** 4. satırda compiler error oluşur.

English — D: Compiler error on line 5. **Türkçe — D:** 5. satırda compiler error oluşur.

English — E: Compiler error on line 7. **Türkçe — E:** 7. satırda compiler error oluşur.

English — F: A runtime exception is thrown. **Türkçe — F:** Runtime exception fırlatılır.

Question 17

English: What is the result of the following code? (Choose all that apply.) **Türkçe:** Aşağıdaki code'un sonucu nedir? (Uygun olanların tümünü seçin.)

```
48: var map = Map.of(1, 2, 3, 6);
49: var list = List.copyOf(map.entrySet());
50:
51: List<Integer> one = List.of(8, 16, 2);
52: var copy = List.copyOf(one);
53: var copyOfCopy = List.copyOf(copy);
54: var thirdCopy = new ArrayList<>(copyOfCopy);
55:
56: list.replaceAll(x -> x * 2);
57: one.replaceAll(x -> x * 2);
58: thirdCopy.replaceAll(x -> x * 2);
59:
60: System.out.println(thirdCopy);
```

English — A: One line fails to compile. **Türkçe — A:** Bir satır derlenmez.

English — B: Two lines fail to compile. **Türkçe — B:** İki satır derlenmez.

English — C: Three lines fail to compile. **Türkçe — C:** Üç satır derlenmez.

English — D: The code compiles but throws an exception at runtime. **Türkçe — D:** Code derlenir ancak runtime'da exception fırlatır.

English — E: If any lines with compiler errors are removed, the code throws an exception at runtime. **Türkçe — E:** Compiler error içeren satırlar kaldırılırsa code runtime'da exception fırlatır.

English — F: If any lines with compiler errors are removed, the code prints `[16, 32, 4]`. **Türkçe — F:** Compiler error içeren satırlar kaldırılırsa code `[16, 32, 4]` yazdırır.

English — G: The code compiles and prints `[16, 32, 4]` without any changes. **Türkçe — G:** Code hiçbir değişiklik yapılmadan derlenir ve `[16, 32, 4]` yazdırır.

Question 18

English: What code change is needed to make the method compile, assuming there is no class named `T`? **Türkçe:** `T` adlı bir class bulunmadığı varsayıldığında method'un derlenmesi için hangi code değişikliği gerekir?

```
public static T identity(T t) {
    return t;
}
```

English — A: Add `<T>` after the `public` keyword. **Türkçe — A:** `public` keyword'ünden sonra `<T>` ekleyin.

English — B: Add `<T>` after the `static` keyword. **Türkçe — B:** `static` keyword'ünden sonra `<T>` ekleyin.

English — C: Add `<T>` after `T`. **Türkçe — C:** `T`'den sonra `<T>` ekleyin.

English — D: Add `<?>` after the `public` keyword. **Türkçe — D:** `public` keyword'ünden sonra `<?>` ekleyin.

English — E: Add `<?>` after the `static` keyword. **Türkçe — E:** `static` keyword'ünden sonra `<?>` ekleyin.

English — F: No change is required. The code already compiles. **Türkçe — F:** Değişiklik gerekmez. Code zaten derlenir.

Question 19

English: What is the result of the following? **Türkçe:** Aşağıdakinin sonucu nedir?

```
var map = new HashMap<Integer, Integer>();
map.put(1, 10);
map.put(2, 20);
map.put(3, null);
map.merge(1, 3, (a, b) -> a + b);
map.merge(3, 3, (a, b) -> a + b);
System.out.println(map);
```

English — A: `{1=10, 2=20}` **Türkçe — A:** `{1=10, 2=20}`

English — B: `{1=10, 2=20, 3=null}` **Türkçe — B:** `{1=10, 2=20, 3=null}`

English — C: `{1=10, 2=20, 3=3}` **Türkçe — C:** `{1=10, 2=20, 3=3}`

English — D: `{1=13, 2=20}` **Türkçe — D:** `{1=13, 2=20}`

English — E: `{1=13, 2=20, 3=null}` **Türkçe — E:** `{1=13, 2=20, 3=null}`

English — F: `{1=13, 2=20, 3=3}` **Türkçe — F:** `{1=13, 2=20, 3=3}`

English — G: The code does not compile. **Türkçe — G:** Code derlenmez.

English — H: An exception is thrown. **Türkçe — H:** Exception fırlatılır.

Question 20

English: Which of the following statements are true? (Choose all that apply.) **Türkçe:** Aşağıdaki ifadelerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

English — A: `Comparable` is in the `java.util` package. **Türkçe — A:** `Comparable`, `java.util` package'ındadır.

English — B: `Comparator` is in the `java.util` package. **Türkçe — B:** `Comparator`, `java.util` package'ındadır.

English — C: `compare()` is in the `Comparable` interface. **Türkçe — C:** `compare()`, `Comparable` interface'indedir.

English — D: `compare()` is in the `Comparator` interface. **Türkçe — D:** `compare()`, `Comparator` interface'indedir.

English — E: `compare()` takes one method parameter. **Türkçe — E:** `compare()` bir method parametresi alır.

English — F: `compare()` takes two method parameters. **Türkçe — F:** `compare()` iki method parametresi alır.

Kaynak kapsam notu: PDF physical page 530 boş bir ayraç sayfasıdır. Chapter 9'un 463–530 physical page aralığı burada eksiksiz tamamlanır.

Appendix · Official Review Question Answers / Resmî Cevaplar

Bu bölüm, kitabın Appendix kısmındaki Chapter 9 cevaplarını PDF physical pages 939–942'den aktarır. Önceki ve sonraki chapter'ların cevapları kapsam dışıdır.

Official Answer 1

English: A, E. For the first scenario, the answer needs to implement `List` because the scenario allows duplicates, narrowing it down to options A and D. Option A is a better answer than option D because `LinkedList` is both a `List` and a `Queue`, and you just need a regular `List`. **Türkçe: A, E.** İlk scenario için cevap `List` implement etmelidir; çünkü scenario duplicate'lere izin verir. Böylece seçenekler A ve D'ye daralır. Yalnızca normal bir `List` gerektiğinden ve `LinkedList` hem `List` hem `Queue` olduğundan A, D'den daha iyi cevaptır.

English: For the second scenario, the answer needs to implement `Map` because you are dealing with key/value pairs per the unique ID field. This narrows it down to options B and E. Since the question talks about ordering, you need the `TreeMap`. Therefore, the answer is option E. **Türkçe:** İkinci scenario'da unique ID field başına key/value pair'lerle çalışıldığı için cevap `Map` implement etmelidir. Bu durum seçenekleri B ve E'ye indirir. Soru ordering'den söz ettiğine göre `TreeMap` gerekir. Dolayısıyla cevap E seçeneğidir.

Official Answer 2

English: C, G. Line 12 creates a `List<?>`, which means it is treated as if all the elements are of type `Object` rather than `String`. Lines 15 and 16 do not compile since they call the `String` methods `isEmpty()` and `length()`, which are not defined on `Object`. Line 13 creates a `List<String>` because `var` uses the type that it deduces from the context. Lines 17 and 18 do compile. However, `List.of()` creates an immutable list, so both of those lines would throw an `UnsupportedOperationException` if run. Therefore, options C and G are correct. **Türkçe: C, G.** Line 12 bir `List<?>` oluşturur; bu nedenle bütün element'lar `String` yerine `Object` type'ındaymış gibi ele alınır. Lines 15 ve 16, `Object` üzerinde tanımlı olmayan `String.isEmpty()` ve `String.length()` method'larını çağırdığı için derlenmez. Line 13'te `var`, context'ten çıkardığı type'ı kullandığı için `List<String>` oluşturur. Lines 17 ve 18 derlenir. Bununla birlikte `List.of()` immutable list oluşturur; bu iki satır çalıştırılırsa `UnsupportedOperationException` fırlatır. Bu nedenle C ve G doğrudur.

Official Answer 3

English: B. This is a double-ended queue. On lines 4 and 5, we add to the back, giving us `[hello, hi]`. On line 6, we add to the front and have `[ola, hello, hi]`. On line 7, we remove the first element, which is `"ola"`. On line 8, we look at the new first element (`"hello"`) but don't remove it. On lines 9 and 10, we remove each element in turn until no elements are left, printing `hello` and `hi` together, which makes option B the answer. **Türkçe: B.** Bu bir double-ended queue'dur. Lines 4 ve 5'te back'e ekleme yaparak `[hello, hi]` elde ederiz. Line 6'da front'a ekleme yaptığımızda `[ola, hello, hi]` olur. Line 7, ilk element olan `"ola"`yı kaldırır. Line 8 yeni ilk element'a (`"hello"`) bakar, fakat onu kaldırmaz. Lines 9 ve 10, hiç element kalmayana kadar her element'ı sırayla kaldırır; `hello` ve `hi` yan yana yazdırıldığı için cevap B'dir.

Official Answer 4

English: B, F. Option A does not compile because the generic types are not compatible. We could say `HashSet<? extends Number> hs2 = new HashSet<Integer>();`. Option B uses a lower bound, so it allows superclass generic types. Option C does not compile because the diamond operator is allowed only on the right side. Option D does not compile because a `Set` is not a `List`. Option E does not compile because upper bounds are not allowed when instantiating the type. Finally, option F does compile because the upper bound is on the correct side of the `=`. **Türkçe: B, F.** Generic type'lar uyumlu olmadığı için A derlenmez. `HashSet<? extends Number> hs2 = new HashSet<Integer>();` yazılabilirdi. B lower bound kullanır; dolayısıyla superclass generic type'lara izin verir. Diamond operator yalnız sağ tarafta kullanılabildiğinden C derlenmez. `Set`, `List` olmadığı için D derlenmez. Type instantiate edilirken upper bound kullanılamadığından E derlenmez. Son olarak upper bound `=` işaretinin doğru tarafında olduğundan F derlenir.

Official Answer 5

English: B. The record compiles and runs without issue. Line 8 gives a compiler warning for not using generics but not a compiler error. Line 7 creates the `Hello` class with the generic type `String`. It also passes an `int` to the `println()` method, which gets autoboxed into an `Integer`. While the `println()` method takes a generic parameter of type `T`, it is not the same `<T>` defined for the class on line 1. Instead, it is a different `T` defined as part of the method declaration on line 3. Therefore, the `String` argument on line 7 applies only to the class. The method can take any object as a parameter, including autoboxed primitives. Line 8 creates the `Hello` class with the generic type `Object` since no type is specified for that instance. It passes a `boolean` to `println()`, which gets autoboxed into a `Boolean`. The result is that `hi-lhola-true` is printed, making option B correct. **Türkçe: B.** Record sorunsuz derlenir ve çalışır. Line 8'de generics kullanılmadığı için compiler warning vardır; compiler error yoktur. Line 7, `Hello` class'ını `String` generic type'ıyla oluşturur. Ayrıca `println()` method'una bir `int` geçirir ve bu value `Integer`a autobox edilir. `println()` method'u `T` type'ında generic parameter olsa da bu `T`, line 1'de class için tanımlanan `<T>` ile aynı değildir; line 3'te method declaration'ının parçası olarak tanımlanan ayrı bir `T`dir. Bu nedenle line 7'deki `String` argument yalnız class'a uygulanır. Method, autobox edilmiş primitive'ler dâhil herhangi bir object alabilir. Line 8, instance için type belirtilmediğinden `Hello` class'ını `Object` generic type'ıyla oluşturur. `println()` a geçirilen `boolean`, `Boolean`a autobox edilir. Sonuçta `hi-lhola-true` yazdırılır ve B doğrudur.

Official Answer 6

English: B, F. We're looking for a `Comparator` definition that sorts in descending order by `beakLength`. Option A is incorrect because it sorts in ascending order by `beakLength`. Similarly, option C is incorrect because it sorts by `beakLength` in ascending order within those matches that have the same name. Option E is incorrect because there is no `thenComparingNumber()` method. **Türkçe: B, F.** `beakLength`e göre descending order uygulayan bir `Comparator` definition arıyoruz. A, `beakLength`e göre ascending sıraladığı için yanlıştır. Benzer biçimde C, aynı name'e sahip eşleşmeler içinde `beakLength`hi ascending sıraladığı için yanlıştır. `thenComparingNumber()` adlı bir method bulunmadığından E de yanlıştır.

English: Option B is a correct answer, as it sorts by `beakLength` in descending order. Options D and F are trickier. First, notice that we can call either `thenComparing()` or `thenComparingInt()` because the former will simply autobox the `int` into an `Integer`. Then observe what `reversed()` applies to. Option D is incorrect because it sorts by name in ascending order and only reverses the beak length of those with the same name. Option F creates a comparator that sorts by name in ascending order and then by beak size in ascending order. Finally, it reverses the result. This is just what we want, so option F is correct. **Türkçe:** B, `beakLength`hi descending sıraladığı için doğrudur. D ve F daha yanıltıcıdır. Önce hem `thenComparing()` hem `thenComparingInt()` çağrılabilceğine dikkat edin; ilki `int`i yalnızca `Integer`a autobox eder. Ardından `reversed()` in neye uygulandığını belirleyin. D, name'i ascending sıralayıp yalnız aynı name'e sahip olanların beak length sırasını ters çevirdiği için yanlıştır. F; önce name'e, ardından beak size'a göre ascending sıralayan bir comparator oluşturur ve sonunda bütün sonucu ters çevirir. İstenen tam olarak budur; bu yüzden F doğrudur.

Official Answer 7

English: B, F. A valid override of a method with generic arguments must have a return type that is covariant, with matching generic type parameters. Options D and E are incorrect because the return type is too broad. Additionally, the generic arguments must have the same signature with the same generic types. This eliminates options A and C. The remaining options are correct, making the answer options B and F. **Türkçe: B, F.** Generic argument'lara sahip bir method'un valid override'ı, matching generic type parameter'larıyla birlikte covariant return type'a sahip olmalıdır. D ve E'nin return type'ı fazla geniş olduğundan bu seçenekler yanlıştır. Ayrıca generic argument'lar aynı generic type'larla aynı signature'a sahip olmalıdır. Bu kural A ve C'yi eler. Kaynağa göre kalan B ve F seçenekleri doğrudur.

Editor note — Java 17 conflict: The official Appendix key is reproduced exactly above, but its treatment of B as an override conflicts with Java 17. B changes the parameter from `List<String>` to `ArrayList<String>`, so it declares an overload, not an override. Adding `@Override` to B produces a compiler error. Under the question's exact word "overrides," only F is technically correct. If the intended question was "which declarations compile in a subclass," B and F compile. **Editör notu — Java 17 çelişkisi:** Resmî Appendix anahtarı yukarıda aynen aktarılmıştır; ancak B'yi override kabul etmesi Java 17 ile çelişir. B, parameter'ı `List<String>`den `ArrayList<String>`e değiştirdiği için override değil overload bildirir. B'ye `@Override` eklemek compiler error üretir. Sorudaki tam "overrides" sözcüğüne göre teknik olarak yalnız F doğrudur. Amaç "subclass içinde hangi declaration'lar derlenir?" ise B ve F derlenir.

Official Answer 8

English: A. The array is sorted using `MyComparator`, which sorts the elements in reverse alphabetical order in a case-insensitive fashion. Normally, numbers sort before letters. This code reverses that by calling the `compareTo()` method on `b` instead of `a`. Therefore, option A is correct. **Türkçe: A.** Array, element'ları case-insensitive biçimde reverse alphabetical order'a koyan `MyComparator` ile sıralanır. Normalde number'lar letter'lardan önce sıralanır. Bu code, `compareTo()` method'unu `a` yerine `b` üzerinde çağırarak sırayı tersine çevirir. Bu nedenle A doğrudur.

Official Answer 9

English: A, B, D. The generic type must be `Exception` or a subclass of `Exception` since this is an upper bound, making options A and B correct. Options C and E are wrong because `Throwable` is a superclass of `Exception`. Additionally, option D is correct despite the odd syntax by explicitly listing the type. You should still be able to recognize it as acceptable. **Türkçe: A, B, D.** Bu bir upper bound olduğundan generic type, `Exception` veya `Exception`ın subclass'ı olmalıdır; dolayısıyla A ve B doğrudur. `Throwable`, `Exception`ın superclass'ı olduğundan C ve E yanlıştır. Ayrıca garip görünen syntax'a rağmen type açıkça yazıldığı için D doğrudur; bu biçimin kabul edilebilir olduğunu tanıyabilmelisiniz.

Official Answer 10

English: A, B, E, F. The `forEach()` method works with a `List` or a `Set`. Therefore, options A and B are correct. Additionally, options E and F return a `Set` and can be used as well. Options D and G refer to methods that do not exist. Option C is tricky because a `Map` does have a `forEach()` method. However, it uses two lambda parameters rather than one. Since there is no matching `System.out.println` method, it does not compile. **Türkçe: A, B, E, F.** `forEach()` method'u `List` veya `Set` ile çalışır; dolayısıyla A ve B doğrudur. E ve F de bir `Set` döndürür ve kullanılabilir. D ve G, var olmayan method'lara başvurur. C yanıltıcıdır; çünkü `Map` gerçekten bir `forEach()` method'una sahiptir. Ancak bu method tek değil iki lambda parameter'ı kullanır. Eşleşen bir `System.out.println` method'u bulunmadığından C derlenmez.

Editor note — Java 17 return type: The printed explanation says both E and F return a `Set`. The answer key **A, B, E, F** is still correct, but the return-type statement is only true for E: `Map.keySet()` returns a `Set<K>`, whereas F, `Map.values()`, returns a `Collection<V>`. **Editör notu — Java 17 dönüş type'i:** Basılı açıklama E ve F'nin ikisinin de `Set` döndürdüğünü söyler. **A, B, E, F** cevap anahtarı yine doğrudur; ancak dönüş type'i ifadesi yalnız E için geçerlidir: `Map.keySet()` bir `Set<K>`, F'deki `Map.values()` ise bir `Collection<V>` döndürür.

Official Answer 11

English: B, E. The `showSize()` method can take any type of `List` since it uses an unbounded wildcard. Option A is incorrect because it is a `Set` and not a `List`. Option C is incorrect because the wildcard is not allowed to be on the right side of an assignment. Option D is incorrect because the generic types are not compatible. **Türkçe: B, E.** `showSize()` method'u unbounded wildcard kullandığından her type'ta `List` alabilir. A bir `List` değil `Set` olduğu için yanlıştır. Wildcard assignment'ın sağ tarafında kullanılmadığından C yanlıştır. Generic type'lar uyumlu olmadığından D de yanlıştır.

English: Option B is correct because a lower-bounded wildcard allows that same type to be the generic. Option E is correct because `Integer` is a subclass of `Number`. **Türkçe:** Lower-bounded wildcard aynı type'ın generic olmasına izin verdiği için B doğrudur. `Integer`, `Number`'ın subclass'ı olduğundan E de doğrudur.

Official Answer 12

English: C. This question is difficult because it defines both `Comparable` and `Comparator` on the same object. The `t1` object doesn't specify a `Comparator`, so it uses the `Comparable` object's `compareTo()` method. This sorts by the `text` instance variable. The `t2` object does specify a `Comparator` when calling the constructor, so it uses the `compare()` method, which sorts by the `int`. This gives us option C as the answer. **Türkçe: C.** Bu soru aynı object üzerinde hem `Comparable` hem `Comparator` tanımladığı için zordur. `t1` object'i bir `Comparator` belirtmez; bu yüzden `Comparable` object'inin `compareTo()` method'unu kullanır ve `text` instance variable'a göre sıralar. `t2` object'i constructor çağırısında bir `Comparator` belirtir; dolayısıyla `int`e göre sıralayan `compare()` method'unu kullanır. Böylece cevap C olur.

Official Answer 13

English: A. When using `binarySearch()`, the `List` must be sorted in the same order that the `Comparator` uses. Since the `binarySearch()` method does not specify a `Comparator` explicitly, the default sort order is used. Only `c2` uses that sort order and correctly identifies that the value `2` is at index `0`. Therefore, option A is correct. The other two comparators sort in descending order. Therefore, the precondition for `binarySearch()` is not met, and the result is undefined for those two. The two calls to `reverse()` are just there to distract you; they cancel each other out. **Türkçe: A.** `binarySearch()` kullanılırken `List`, `Comparator`ın kullandığı order ile aynı biçimde sorted olmalıdır. `binarySearch()` method'u açıkça bir `Comparator` belirtmediği için default sort order kullanılır. Yalnız `c2` bu order'ı kullanır ve `2` value'sunun index `0`da olduğunu doğru belirler; bu nedenle A doğrudur. Diğer iki comparator descending sıralar. Bunlarda `binarySearch()` precondition'ı sağlanmadığından sonuç undefined'dir. İki `reverse()` çağırısı dikkat dağıtmak içindir; birbirini götürür.

Official Answer 14

English: A, B. `Y` is both a class and a type parameter. This means that within the class `Z`, when we refer to `Y`, it uses the type parameter. All of the choices that mention class `Y` are incorrect because it no longer means the class `Y`. Only options A and B are correct. **Türkçe: A, B.** `Y` hem class hem type parameter'dır. Bu nedenle `Z` class'ı içinde `Y`ye başvurulduğunda type parameter kullanılır. Artık `Y` class'ını ifade etmediğinden class `Y`yi kullanan bütün seçenekler yanlıştır. Yalnız A ve B doğrudur.

Official Answer 15

English: A, C. A `LinkedList` implements both `List` and `Queue`. The `List` interface has a method to remove by index. Since this method exists, Java does not autobox to call the other method, making the output `[10]` and option A correct. Similarly, option C is correct because the method to remove an element by index is available on a `LinkedList<Object>` (which is what `var` represents here). By contrast, `Queue` has only the remove-by-object method, so Java does autobox there. Since the number `1` is not in the list, Java does not remove anything for the `Queue`, and the output is `[10, 12]`. **Türkçe: A, C.** `LinkedList` hem `List` hem `Queue` implement eder. `List` interface'i index'e göre kaldıran bir method'a sahiptir. Bu method bulunduğundan Java diğer method'u çağırmak için autoboxing yapmaz; çıktı `[10]` olur ve A doğrudur. Benzer biçimde `var`ın burada temsil ettiği `LinkedList<Object>` üzerinde index'e göre element kaldıran method bulunduğundan C doğrudur. Buna karşılık `Queue` yalnız object'e göre remove method'una sahiptir; Java burada autoboxing yapar. `1` list'te bulunmadığı için `Queue`dan hiçbir şey kaldırılmaz ve çıktı `[10, 12]` olur.

Official Answer 16

English: E. This question looks like it is about generics, but it's not. It is trying to see whether you noticed that `Map` does not have a `contains()` method. It has `containsKey()` and `containsValue()` instead, making option E the answer. If `containsKey()` were called, the answer would be `false` because `123` is an `Integer` key in the `Map`, rather than a `String`. **Türkçe: E.** Soru generics hakkındaymış gibi görünür, fakat değildir. `Map`in `contains()` method'una sahip olmadığını fark edip etmediğinizi ölçer. Bunun yerine `containsKey()` ve `containsValue()` bulunduğu için cevap E'dir. `containsKey()` çağrılıysaydı sonuç `false` olurdu; çünkü `map`'teki `123` key'i `String` değil `Integer`dir.

Official Answer 17

English: A, E. The key to this question is keeping track of the types. Line 48 is a `Map<Integer, Integer>`. Line 49 builds a `List` out of a `Set` of `Entry` objects, giving us `List<Entry<Integer, Integer>>`. This causes a compiler error on line 56 since we can't multiply an `Entry` object by two.

Türkçe: A, E. Bu sorunun anahtarı type'ları izlemektir. Line 48 bir `Map<Integer, Integer>` dir. Line 49, `Entry` object'lerinden oluşan bir `Set`ten `List` oluşturur; sonuç `List<Entry<Integer, Integer>>` olur. Bir `Entry` object'i ikiyle çarpılamayacağından line 56 compiler error üretir.

English: Lines 51–54 are all of type `List<Integer>`. The first three are immutable, and the one on line 54 is mutable. This means line 57 throws an `UnsupportedOperationException` since we attempt to modify the list. Line 58 would work if we could get to it. Since there is one compiler error and one runtime error, options A and E are correct. **Türkçe:** Lines 51–54'ün tümü `List<Integer>` type'ındadır. İlk üçü immutable, line 54'teki ise mutable'dır. List'i değiştirmeye çalıştığımız için line 57

`UnsupportedOperationException` fırlatır. Execution o noktaya ulaşabilseydi line 58 çalışırdı. Bir compiler error ve bir runtime error bulunduğundan A ve E doğrudur.

Official Answer 18

English: B. When using generic types in a method, the generic specification goes before the return type, and option B is correct. **Türkçe: B.** Bir method'da generic type kullanılırken generic specification return type'tan önce gelir; bu nedenle B doğrudur.

Official Answer 19

English: F. The first call to `merge()` calls the mapping function and adds the numbers to get 13. It then updates the map. The second call to `merge()` sees that the map currently has a `null` value for that key. It does not call the mapping function but instead replaces it with the new value of 3. Therefore, option F is correct. **Türkçe: F.** İlk `merge()` çağırısı mapping function'ı çağırır ve number'ları toplayarak 13 elde eder; ardından map'i günceller. İkinci `merge()` çağırısı, map'te o key için mevcut value'nun `null` olduğunu görür. Mapping function'ı çağırmaz; onun yerine value'yu yeni 3 value'suyla değiştirir. Bu nedenle F doğrudur.

Official Answer 20

English: B, D, F. The `java.lang.Comparable` interface is implemented on the object to compare. It specifies the `compareTo()` method, which takes one parameter. The `java.util.Comparator` interface specifies the `compare()` method, which takes two parameters. This gives us options B, D, and F as the answers. **Türkçe: B, D, F.** `java.lang.Comparable` interface'i karşılaştırılacak object üzerinde implement edilir. Bir parameter alan `compareTo()` method'unu bildirir. `java.util.Comparator` interface'i ise iki parameter alan `compare()` method'unu bildirir. Böylece cevaplar B, D ve F olur.